Lean Proof Strategy Proposal for the Scheduling Rules Document

Stuart Swan, Platform Architect, Siemens EDA

24 June 2026

0. Goal and scope

The goal is to formalize the scheduling rules document's mathematical proof core in Lean: trace compatibility, witness construction, cutpoint correspondence, elaboration, and one-way liveness/deadlock preservation. The Lean proof would certify the formalized theorem stack under the stated assumptions, not every informal prose statement in the document.

The scheduling rules document's own framing is conditional: the formal results depend on assumptions, observation-boundary choices, witness-selection conditions, and liveness/progress premises.

The Lean development should therefore be organized around explicit theorem instances and assumption bundles, rather than hidden global axioms.

Notation and signature status. The Lean-like declarations in this strategy are intended to fix proof architecture, theorem dependencies, and object domains. Unless a declaration is explicitly identified as a final Lean signature, it should be read as schematic pseudo-Lean. In particular, channel, process, side, prefix, and bucket types must be unified in the concrete development through the selected ComparisonInstance, the side-parametric SystemOfSide map, and the shared proof-internal alphabet C.Sigma. This note is not a relaxation of the proof obligations; it prevents schematic notation such as Channel, C.Sys_ref.Channel, Prefix, and InfiniteExecutionBucketed from being mistaken for already type-checking Lean code.

Version alignment note. This revision updates the Lean proof strategy to align with the scheduling-rules v45 Appendix K changes, while retaining the earlier v42-aligned strategy as the base. It retains the v42 alignment points: (i) Appendix G/J separates the informational causal-dependency graph from the strict Appendix J required-order relation ≺_J; (ii) Theorem J.1 produces an explicitly selected Sys_ref witness prefix and is organized along the fixed RTL-time ideal chain rather than by arbitrary ≺_J-minimal extension; (iii) Corollary J.1 and Appendix K/R cutpoint transfers use an explicit RTL_B-to-source abstraction relation J.Abs and a corresponding implementation obligation J.AbsConf; and

(iv) Appendix K A5 is now the conservative serial-blocking source-liveness assumption, while Lemma K.2b supplies the serial-source dominance bridge from an RTL-matched / overlapped Sys_ref witness deadlock to a serial-source deadlock.

The retained v42-specific alignment points are: (v) R2C combinational signal IO is now modeled as component-local, stutter-canonical fixed-point signal-observation units with selected observable slices Obs(C_comb), not as an unconditional per-cycle observable bucket; R2C matching is by component identity and local R2C occurrence ordinal, with cycle-locked components handled as an explicit stronger observation mode. (vi) ChannelEnabled is eligibility/availability for scheduler or arbiter selection, not same-cycle commit by itself; E4 is the safety/legality rule for committed transfers, while B2 supplies eventual selection and commit for continuously ChannelEnabled actions. (vii) hidden arbitration, grant, or back-pressure may not be smuggled in as an extra E4 availability conjunct at an ordinary boundary; it must be represented as fair selection among ChannelEnabled actions or as explicit Sys_B^elab boundary structure. (viii) K.0/K.0a frontier reasoning now explicitly proves full NextFront minimality over synchronization, anchored signal/R2C, and message items, and rendezvous endpoint-request transfer relies on J.Abs/J.AbsConf raw endpoint-request facts. (ix) K.1 uses fair drain-normalization, rather than pure idle-stutter, to justify Exec_post applicability at the relevant deadlock cutpoint.

Revision update note (v42 base plus v45 Appendix K refinement pass). This revision is a refinement pass against the v42-aligned strategy plus the scheduling-rules v45 Appendix K update. It retains the earlier v42 items and repairs several internal inconsistencies: (x) the Common Formal Definitions side-condition discharge record is given an explicit Lean-side audit structure whose WC field proves the actual WC predicate, with NB-CF, Appendix L snooping, direct construction, and mixed per-site coverage recorded as proof-producing provenance routes; Sys_B^elab(E), B-faithfulness, J.AbsConf, and Kε retain their theorem-scoped discharge evidence (Section 1.1); (xi) B-faithfulness of each back-annotated capacity B(c) is recorded as a named tool/RTL-flow compliance hypothesis alongside E4 channel-realization conformance and J.AbsConf, with its formal structure and assumption threading in Section 7, its audit discharge in Section 1.1, and its concrete-flow checklist entry in Section 20; (xii) the v42 B3 modularity note is reflected: for a single explicit abstract boundary with ordinary E4 buffered or rendezvous semantics, the B3 P_push/P_pop discharge is derivable from B2 plus E4, and Lean should prove that derivation while retaining B3 as the modular per-channel obligation checked against channel realizations and elaborated Sys_B^elab boundary chains (Section 6); (xiii) the Appendix K terminated-process convention is made an explicit derived theorem together with its scope warning: waiting on a terminated peer is not, by itself, an internal message-passing deadlock in the

K.1 sense, and intended termination/EOF/done protocols must be modeled explicitly or verified outside Appendix K (Section 18); (xiv) initial channel occupancy is aligned with A10: document-aligned theorem instances use reset-empty occupancy (initialOcc = 0), and accounted nonzero initial occupancy is flagged as an extension outside the main document's empty-FIFO model (Section 7); (xv) RULE 1 / RULE 2 source well-formedness is recorded as part of the R2 obligation for proof-backed flows (Section 9); (xvi) ObsSigma gains the R2C component-local observation-unit slice already required by the document's Definition I.Obs (Section 15); (xvii) JTraceAssumptions gains the elaboration-projection well-formedness field that the J.1 proof sketch already consumed, with corresponding K-to-J trace and cutpoint projection constructors (Sections 17 and 18); (xviii) ProcessStalledAtFront is unified with FrontierBlocked so the Appendix K layer uses the Section 13 conjunction form of "blocked on message passing" (Section 18); (xix) RequiredJOrder acyclicity is proved exactly as in the v42 Theorem J.1 argument, by the per-execution commit-cycle measure over the fixed legal prefix (Section 17); (xx) A5 is stated over finite conservative-serial source prefixes, and Theorem K.1 uses Lemma K.2b to convert the selected finite liberal Sys_ref prefix into a finite conservative-serial deadlocking prefix before A5 is invoked (Section 18); (xxi) START_P / FINISH_P anchors are read as per-reset-epoch dynamic occurrences, matching the v42 dynamic-reset treatment of direct inputs (Section 3); (xxii) Corollary L.4 (cadence-sensitive polling handled by isolation) is given an explicit named theorem alias (Section 14); (xxiii) a scope note records that the formalization targets the single-global-clock semantics of Appendices G–K (below); and (xxiv) the v45 Lemma K.2b proof spine is formalized as an eager-count reduction: serial reference executions are explicitly liberal/well-formed reference executions plus the no-eager restriction, and the one-step reduction is supported by endpoint-source-order prefix suppression, joint value/control consistency, occupancy/disabledness domination, no-third-party-enable monotonicity, and terminal-SCC persistence.

M5 alignment note (standing B1 in the main theorem stack). The latest scheduling-rules Appendix G/J wording treats B1 as a standing RTL_B determinism assumption for the main Appendix G, Theorem J.1, Corollary J.1, and Appendix K theorem instances under the fixed stimulus. Lean therefore treats PostDeterministicSigmaTrace C as carried by the B-rule bundle used by JTraceAssumptions and KAssumptions, not as an optional premise introduced only when uniqueness is invoked. B4/B5 remain separate additional premises used for ε-normalization, ε-quiescent endpoint reasoning, and uniqueness up to finite ε-stutter.

22 June 2026 correction pass. This revision additionally: (xxv) makes Q_Ω a fixed witness-indexed dynamic universe rather than a side/state-dependent object; (xxvi) defines

BFaithful and carries per-channel B-faithfulness through ImplementationAssumptions and the side-condition discharge record; (xxvii) states A5 directly over admissible finite conservative-serial source prefixes and makes K.2b return such a prefix, avoiding an unproved fair-extension step; (xxviii) separates the deadlock shape predicate from its reachability wrapper; (xxix) removes the duplicate placeholder K.0 interface in favor of K0_front_nextfront_classification; and (xxx) derives per-process post-side trace-projection determinism from system-level B1 without asserting isolated process determinism.

Second 22 June 2026 correction pass. This revision additionally: (xxxi) removes the residual finite-prefix/infinite-execution type split from the J.1-to-K.2b path; (xxxii) gives J1_selected_ref_prefix_is_liberal the complete selected post-prefix and reference-prefix signature; (xxxiii) places each process dynamic universe directly in WitnessBundle and separates witness-dependent universe well-formedness from universe data; (xxxiv) states the terminated-peer result against the canonical closed, drain-closed WFG-component conjunction; (xxxv) packages the exact selected post prefix, reference prefix, cutpoints, compatibility, normalization, and J.Abs correspondence in SelectedMatchedCutpoint; (xxxvi) strengthens B-faithfulness from a capacity upper bound to exact E4 boundary-behavior refinement; and (xxxvii) replaces free-standing status labels with method-indexed discharge objects whose constructors carry the corresponding syntactic, tool/RTL, or witness evidence.

Third 22 June 2026 correction pass. This revision additionally: (xxxviii) makes ClosedDrainClosedWFGComponent the single canonical component used by ExistsClosedDrainClosedWFGCycle, InternalMPDeadlockShapeAt, process-participation, and the terminated-peer corollaries, with ProcessStalledAtFront defined literally as FrontierBlocked; (xxxix) performs the K.2b eager-count reduction over semantic Prefix C.Sys_ref objects throughout, using RefPrefixBuckets only as a derived bucket projection and requiring each one-step reduction to construct an actual admissible reference prefix; and (xl) updates the earlier B-faithfulness revision note to identify its Section 1.1 audit, Section 7 formal structure/assumption threading, and Section 20 concrete-flow checklist locations.

Fourth 22 June 2026 correction pass. This revision additionally: (xli) makes the side-condition audit theorem-scoped and requires its WC field to discharge the actual predicate WC C W, with NB-CF, snooping, and mixed per-site coverage represented as proof-producing provenance routes rather than as alternatives inside the audited proposition; (xlii) replaces process-only START_P / FINISH_P anchors with explicit reset-epoch-indexed dynamic synchronization occurrences, bounds terminated-process idle padding by the next reset affecting the process, and states the terminated-peer corollaries only for peers terminated in the current reset epoch at the examined cutpoint; and (xliii) removes duplicate A2–A4 fields from DocKStructuralAssumptions and exposes those document-numbered premises as projections of the B4, B2, and B3 fields already carried by A1's BRules component.

23 June 2026 alignment and proof-architecture pass. This revision additionally: (xliv) separates trace/admissibility assumptions from B4/B5 normalization by introducing TraceBRules and NormalizationBRules, using them in Appendix I, and then introducing JTraceAssumptions and JCutpointAssumptions; (xlv) makes preservation of Exec_ref under each K.2b eager deferral an explicit construction theorem, including the obligation to build an admissible fair/progress-satisfying reference extension for the edited finite prefix rather than treating semantic-prefix realizability as automatic; (xlvi) restructures the former monolithic Milestone B into trace, cutpoint-normalization, and WFG/liveness submilestones; (xlvii) aligns the audit summaries with the scheduling-rules treatment of NB-CF and snooping as provenance routes for WC; (xlviii) targets the attached 23 June 2026 Scheduling Rules document, including its B5 environmental-stability and strict earlier-cycle A5 clarifications; and (xlix) makes Appendix I deterministic replay operate over complete P-local ObservableUnit steps rather than individual Σ labels, so atomic rendezvous, synchronization, and stutter-canonical R2C units are never split; the R2C congruence case now derives canonical emission and fixed-point agreement from the prior r2c_unit_slice, R2C_WF, the fixed stimulus, and the applicable first-observation/change/observer/cycle-locked condition rather than from a common label alone.

24 June 2026 scheduling-alignment pass. This revision additionally: (l) aligns Appendix I/J with the corrected 24 June 2026 Scheduling Rules treatment of deterministic replay by separating canonical immediate post-unit replay from explicitly ε-normalized replay; (li) introduces IDR_immediate_post_unit_deterministic_replay for the plain J.1 ideal-chain construction and retains IDR_deterministic_replay for ε-quiescent cutpoint theorems; (lii) makes the finite-P-local-event branch of I.5 explicit: B6 may be used only after a matched global Sys_ref prefix reaches a B5/global fixed point, while the non-fixed-point branch is obtained by projecting an admissible infinite global Sys_ref continuation supplied through Exec_ref; (liii) records the immediate replay-state invariant inside SelectedJWitnessEvidence so J.1 imports no B4/B5 normalization premise; (liv) names the side-parametric finite fair drain-normalization helper consumed by the K.2b eager-deferral reduction and by the Theorem K.1 Exec_post bridge; and (lv) preserves the document result as Theorem R.2 while using the typed Lean identifier R_theorem2_closed_cycle_and_one_way_drain_transfer.

Terminology note on v42/v45 labels. References to “v42” and “v45” in this strategy identify alignment decisions inherited from the earlier review passes. Unless explicitly stated otherwise, the theorem instances, assumptions, and appendix references in this revision target the attached 24 June 2026 Scheduling Rules document.

Scope note (single clock). The formal core targeted by this strategy is the single-global-clock semantics of Appendices G–K of the scheduling-rules document. Appendix O of that document is an informative multi-clock sketch only. A multi-clock Lean development would additionally require the global event order or partial order over local-domain cycles, precise CDC commit semantics, cross-domain occupancy sampling rules, synchronizer fairness/metastability abstractions, and CDC progress obligations that Appendix O explicitly defers; those layers are out of scope for this strategy unless separately supplied.

The primary scheduling rules document on which this doc is based is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

The latest version of this Lean proof strategy document is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/Lean_proof_strategy.pdf

The Matchlib examples kit that contains training slides, examples, etc., is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/tree/master

1. Central comparison object

The proof should be parameterized by a selected comparison instance:

```
structure ComparisonInstance where
Sys_ref : System
RTL_B : System
-- Shared proof-internal observable alphabet for the Sys_ref / RTL_B comparison.
-- Both sides emit events in this same Σ schema.
Sigma : Alphabet
-- Optional outer DUT/testbench-visible projection alphabet.
-- In ordinary cases this may be the identity projection from Sigma.
Sigma_DUT : Alphabet
sigmaDUTProjection : ProjectionMap Sigma Sigma_DUT
```

env : ReactiveEnvironment Sigma
refChoice : SysRefChoice
-- The explicit Sys_ref field and the reference-choice record must name the
-- same source reference system. This prevents theorem instances from using
-- C.Sys_ref in one definition and C.refChoice.Sys_ref in another while silently
-- referring to different systems.
sys_ref_consistent :
Sys_ref = refChoice.Sys_ref

-- Optional modular combinational model used by R2C.
-- The v42 R2C semantics are stated over explicit combinational observation
-- components. Each component has a selected observable slice Obs(C_comb), the
-- composed network has a unique ε-fixed point μ_t in each compared cycle, and
-- Σ-visible R2C events are emitted only as stutter-canonical fixed-point
-- signal-observation units. Matching is by component identity and local R2C
-- occurrence ordinal, not by global cycle number or by equality of unrelated
-- CycleBucket contents.
combModel? : Option CombComposition

assumptions : AssumptionBundle
contracts : ModelingContracts

The comparison relation between Sys_ref and RTL_B is defined over the single shared proof-internal alphabet C.Sigma. The reference and post-HLS executions may have different internal ε-structure, bucket decomposition, or implementation state, but their observable events are encoded in the same Σ event schema before CompatibleP / CompatibleSys is applied.

Sigma_DUT is not a second post-side alphabet. It is only the chosen outer DUT/testbench-visible projection of C.Sigma. In elaborated cases, introduced proof-internal staged/bundle/join events may be hidden or accounted for by sigmaDUTProjection / Π_elab when projecting to Sigma_DUT, while the proof-internal compatibility theorem still ranges over C.Sigma.

Sys_ref is the chosen source reference model for the theorem instance:
ordinary bounded-FIFO case: Sys_ref = Sys_B;
elaborated case: Sys_ref = Sys_B^elab(E) for a well-formed elaboration package E.

The refChoice field records why this Sys_ref is the prescribed source reference model, and sys_ref_consistent enforces that C.Sys_ref and C.refChoice.Sys_ref are the same system. This follows the updated Appendix Q direction: the formal theorems do not require a unique algorithm that constructs Sys_B^elab; instead, the chosen Sys_ref is part of the theorem instance, but the theorem instance may not contain two inconsistent reference systems.

1.1 Side-condition discharge record

The scheduling-rules Common Formal Definitions require an explicit side-condition discharge record: before Theorem J.1, Corollary J.1 cutpoint transfer, or the Appendix K liveness-preservation theorem is applied to a concrete design, the verification flow shall state, for each named side condition, whether the condition is inapplicable, satisfied by a simple syntactic/design restriction, discharged by tool/RTL evidence, or discharged by a source/RTL witness construction. The named conditions are WC, NB-CF, the Appendix L snooping / witness selector W, Sys_B^elab, per-channel B-faithfulness, J.AbsConf, and Kε. Because the required subset depends on the theorem being applied, the Lean audit object is indexed by theorem scope. The record is not an additional semantic assumption beyond the premises already used by Appendices I–K; its purpose is to make theorem application auditable and to ensure that a condition required by the selected theorem cannot be marked inapplicable.

The Lean encoding should be a bookkeeping structure over the same evidence objects the strategy already defines, so that an instantiated theorem instance carries its own audit trail:

```
structure NotRequired (P : Prop) : Prop where
reason : SideConditionNotApplicable P

structure SyntacticEvidence (P : Prop) : Prop where
proof : P
restriction : NamedSyntacticOrDesignRestriction P

structure ToolOrRTLEvidence (P : Prop) : Prop where
proof : P
certificate : NamedToolRTLCertificateOrCheck P

structure WitnessConstructionEvidence (P : Prop) : Prop where
proof : P
construction : NamedSourceRTLWitnessConstruction P
```

```
inductive Discharge (P : Prop) : Type
| inapplicable (evidence : NotRequired P)
| syntacticRestriction (evidence : SyntacticEvidence P)
| toolOrRTLEvidence (evidence : ToolOrRTLEvidence P)
| witnessConstruction (evidence : WitnessConstructionEvidence P)

-- RequiredDischarge has no inapplicable constructor. It is used where the
-- selected theorem instance necessarily requires the side condition.
inductive RequiredDischarge (P : Prop) : Type
| syntacticRestriction (evidence : SyntacticEvidence P)
| toolOrRTLEvidence (evidence : ToolOrRTLEvidence P)
| witnessConstruction (evidence : WitnessConstructionEvidence P)

inductive TheoremScope
| j1Trace
| j1Cutpoint
| k1Liveness

namespace TheoremScope

def requiresCutpointTransfer : TheoremScope → Bool
| .j1Trace => false
| .j1Cutpoint => true
| .k1Liveness => true

def requiresKepsilon : TheoremScope → Bool
| .j1Trace => false
| .j1Cutpoint => false
| .k1Liveness => true

end TheoremScope

-- ScopeDischarge prevents a required condition from being marked inapplicable.
def ScopeDischarge (required : Bool) (P : Prop) : Type :=
if required then RequiredDischarge P else Discharge P

-- Each WC route carries a proof of the actual theorem premise WC C W.
```

```
-- NB-CF and snooping are provenance for that proof, not alternative propositions
-- that may replace WC in the theorem assumption bundle.
structure SnoopingWCDischarge
(C : ComparisonInstance)
(W : WitnessBundle C) where
assumptions : SnoopingWrapperAssumptions C W
wc_proof : WC C W
construction : NamedSourceRTLWitnessConstruction (WC C W)

structure NBCFWCDischarge
(C : ComparisonInstance)
(W : WitnessBundle C) where
witness_is_identity_or_arbitrary :
W = IdentityOrArbitraryNBWitnessBundle C
nb_cf : ∀ P, NBCF C P
local_protocols :
LatencySensitiveLocalProtocolWC
  C W C.contracts.latencySensitiveLocalProtocols
wc_proof : WC C W

structure MixedWCDischarge
(C : ComparisonInstance)
(W : WitnessBundle C) where
partition : NBSiteDischargePartition C W
complete : AllRelevantNBSitesCovered C W partition
wc_proof : WC C W
construction : NamedSourceRTLWitnessConstruction (WC C W)

inductive WCDischarge
(C : ComparisonInstance)
(W : WitnessBundle C) : Type
| direct (evidence : WitnessConstructionEvidence (WC C W))
| fromSnooping (evidence : SnoopingWCDischarge C W)
| fromNBCF (evidence : NBCFWCDischarge C W)
| mixed (evidence : MixedWCDischarge C W)

def WCDischarge.proof
(C : ComparisonInstance)
```

```
(W : WitnessBundle C) :
WCDischarge C W → WC C W
| .direct evidence => evidence.proof
| .fromSnooping evidence => evidence.wc_proof
| .fromNBCF evidence => evidence.wc_proof
| .mixed evidence => evidence.wc_proof

structure SideConditionDischargeRecord
(C : ComparisonInstance)
(W : WitnessBundle C)
(scope : TheoremScope) where
-- This field discharges the actual WC premise consumed by the theorem stack.
-- Its constructor records whether the proof came from direct witness
-- construction, snooping, global NB-CF plus the required local-protocol facts,
-- or a complete mixed per-site discharge.
wc : WCDischarge C W

-- Ordinary Sys_B instances discharge this as inapplicable with a proof that no
-- elaboration package is required. Elaborated instances carry the package proof.
elaboration : Discharge
  (∃ E, C.refChoice.elab? = some E ∧ WellFormedElaborationPackage E)

-- Every selected back-annotated explicit abstract channel requires evidence;
-- therefore this field cannot be marked inapplicable.
bfaithfulness :
∀ c,
BackAnnotatedExplicitAbstractChannel C c →
RequiredDischarge (BFaithful C c)

-- J.AbsConf is required for cutpoint transfer and Appendix K. Kε is required
-- for Appendix K. The theorem-scope index enforces those distinctions.
jabsconf : ScopeDischarge
  (TheoremScope.requiresCutpointTransfer scope)
  (Nonempty (JAbsConf C W))
kepsilon : ScopeDischarge
  (TheoremScope.requiresKepsilon scope)
  (A11_Kepsilon_WFGInert C)
```

The discharge record still duplicates no semantics: every evidence route carries a proof of the same predicate consumed by the theorem stack. In particular, the wc field proves WC C W itself. NB-CF, snooping, and mixed per-site coverage are recorded as provenance for constructing that proof; they are not disjunctive substitutes for WC. The global NB-CF route also records the identity/arbitrary witness selection and the latency-sensitive local-protocol premise required by Corollary L.3. A mixed route carries a complete partition of all relevant NB sites. The scope index makes J.AbsConf mandatory for cutpoint-transfer and Appendix K applications and makes Kε mandatory for Appendix K. Per-channel B-faithfulness continues to use RequiredDischarge, so a back-annotated explicit abstract channel cannot be marked inapplicable. This makes the record a genuine theorem-application audit object rather than an unconstrained label attached to unrelated evidence.

2. Foundation layer: bucketed executions, not flat traces

The primary execution model should be cycle-bucketed, not a flat sequence of eps | tick | sigma e. The reason is that the document allows multiple same-cycle Σ-events without imposing artificial order among independent same-cycle events. The earlier review correctly noted that a flat trace would either impose a fake linearization or require an added bucket abstraction anyway.

Use:

```
structure State where
  cycle  : Nat
  epsPos : Nat
  store  : Store
```

with explicit advancement rules:

```
-- Intra-cycle ε-normalization steps advance epsPos only.
-- Same-cycle Σ-events advance epsPos only.
-- Clock ticks advance cycle and reset epsPos.
-- Cycle-advancing silent progress is represented at the bucket level: a
-- CycleBucket with sigmaEvents = ∅ may still consume a real clock cycle and
-- may represent HLS-inserted FSM latency, arbitration/bookkeeping latency,
-- failed non-blocking polls, or pipeline advances that emit no Σ-event.
```

Define buckets:

```
structure CycleBucket (Σ : Type) where
```

```
cycle : Nat
startState : State
epsPrefix : List EpsStep
-- The Σ-events committed in this clock cycle.
-- This is an unordered finite set. A CycleBucket does not carry an intrinsic
-- sequence or bucket-local partial order among same-cycle events.
sigmaEvents : Finset Σ
epsSuffix : List EpsStep
endState : State
endQuiescent : Prop
def ExecutionBucketed (Σ : Type) :=
List (CycleBucket Σ)
def InfiniteExecutionBucketed (Σ : Type) :=
Nat → CycleBucket Σ
```

Bucket well-formedness predicates, not the Finset representation alone, enforce per-cycle scalar constraints such as at most one Push_c and at most one Pop_c per channel in a cycle. Required ordering is not stored in CycleBucket. It is derived globally from observable units and from the Appendix J required-order relation ≺_J, not from bucket membership or bucket-local sequencing.

The scheduling-rules v42 text intentionally separates two relations:
causal-dependency graph (→), an information-flow graph over observable IO actions that is useful for explaining functional dependencies but need not be acyclic in every legal execution; and
RequiredJOrder / ≺_J, the well-founded strict order over observable units used by the Appendix J finite-ideal induction and by CompatibleP / CompatibleSys.

This provides the right foundation for:
ε-quiescent cutpoints;
rendezvous same-cycle commits;
R2C component-local fixed-point signal-observation units;
J.1 ideal induction over observable units and the mandated ≺_J relation;
B2/B3 temporal reasoning over infinite bucketed executions.

Important separation: ExecutionBucketed is the semantic carrier for state advancement, ε-quiescence, occupancy updates, R2C fixed points, R2C stutter-canonical unit emission, and fairness/progress reasoning. It is not itself the cross-model trace-compatibility predicate.

In particular, system-level compatibility τ_ref,system ≈_sys τ_post,system must not be defined as equality of CycleBucket lists, equality of bucket cycle numbers, or bucket-by-bucket equality of sigmaEvents. The document's Appendix J compatibility relation is an ordered cross-model observable-compatibility predicate. It preserves the explicitly required ≺_J constraints, atomic multi-endpoint units, E1–E5 obligations, FIFO/synchronization/signal-anchor obligations, and matched value labels, while allowing independent observable units to commute or to be grouped into different cycle buckets in Sys_ref and RTL_B. Same-cycle membership in a CycleBucket is therefore a timing fact for one execution, not a source of intrinsic order.

Thus:
bucket equality may be required inside explicitly atomic units, such as rendezvous Push/Pop pairs and SyncChannel/ac_sync handshakes;
bucket decomposition may be used to prove local state-transition and enablement facts;
but CompatibleSys / ≈_sys is defined over observable units and the mandated ≺_J relation, not over identical bucket decompositions.

3. Events, operation instances, and source order

Define dynamic operation universes:

```
inductive MsgKind
| Push
| Pop
| PushNB
| PopNB

def MsgKind.isBlocking : MsgKind → Bool
| MsgKind.Push => true
| MsgKind.Pop => true
| MsgKind.PushNB => false
| MsgKind.PopNB => false

structure DynMsgInstance where
proc : Process
channel : Channel
```

kind : MsgKind -- Push, Pop, PushNB, PopNB
sourceLoc : SourceLoc
instanceId : Nat

def Blocking (q : DynMsgInstance) : Prop :=
q.kind.isBlocking = true

def NonBlocking (q : DynMsgInstance) : Prop :=
q.kind.isBlocking = false

```
structure FrontierItem where
  proc      : Process
  itemKind   : FrontierItemKind
  sourceLoc  : SourceLoc
  instanceId : Nat
```

Core source-order objects are witness-specific. They range only over dynamic operation instances that actually occur in the compared execution / witness. Untaken branches and unexecuted loop iterations are not members of this dynamic universe.

Qexec : Process → Set DynMsgInstance
QOmega : Process → Set DynMsgInstance
Omega : Process → Set FrontierItem

Qexec,P is the execution-level dynamic-instance universe. It includes executed failed non-blocking polls that may produce no Σ-event.

Omega,P is the frontier/order universe used by Front, NextFront, WFG, E3/E5, BoundaryReq attribution, and source-order reasoning. Failed NB-only executed polls are not included in Omega,P merely because they executed.

QOmega,P denotes the executed dynamic message-operation-instance slice induced by Omega,P. Since Qexec,P is a set of DynMsgInstance and Omega,P is a set of FrontierItem, this is not a direct set intersection. It is defined through the message-instance projection from frontier items:

QOmega,P :=
{ q : DynMsgInstance |

```
q ∈ Qexec,P ∧
∃ x : FrontierItem,
x ∈ Omega,P ∧
MsgInstanceOfFrontierItem C P x = some q }

def MsgInstanceOfFrontierItem
(C : ComparisonInstance)
(P : Process)
(x : FrontierItem) : Option DynMsgInstance :=
...

-- R2/E2 signal anchors include explicit synchronization calls and the
-- process-boundary START_P / FINISH_P events. Every anchor is a dynamic
-- occurrence in a particular reset epoch; process identity alone is not enough
-- to identify an anchor when a process can be reset and restarted.
structure ResetEpochId where
index : Nat
deriving DecidableEq

inductive SyncOccurrenceKind
| start
| call (s : SyncCall)
| finish

structure SyncOccurrence where
proc : Process
epoch : ResetEpochId
kind : SyncOccurrenceKind
instanceId : Nat

abbrev SyncAnchor := SyncOccurrence

def SyncAnchorClock
(C : ComparisonInstance)
(W : WitnessBundle C)
(a : SyncAnchor) : Cycle :=
SyncOccurrenceClock C W a
```

```
def SyncAnchorRealizedInUniverse
(C : ComparisonInstance)
(P : Process)
(W : WitnessBundle C)
(Ω : Set FrontierItem)
(a : SyncAnchor) : Prop :=
a.proc = P ∧
SyncOccurrenceWellFormed C W a ∧
∃ x,
x ∈ Ω ∧
SyncOccurrenceFrontierItem C P a x ∧
ExecutedInWitness C W P x

-- Pure universe data. It contains no current state and no side parameter.
structure DynamicUniverseData
(C : ComparisonInstance)
(P : Process) where
Q_exec : Set DynMsgInstance
Ω : Set FrontierItem
Q_Ω : Set DynMsgInstance
qomega_characterization :
∀ q,
q ∈ Q_Ω ↔
q ∈ Q_exec ∧
∃ x, x ∈ Ω ∧ MsgInstanceOfFrontierItem C P x = some q
message_frontier_items_have_exec_instance :
∀ x q,
x ∈ Ω →
MsgInstanceOfFrontierItem C P x = some q →
q ∈ Q_exec ∧ q ∈ Q_Ω
qomega_items_have_frontier_item :
∀ q,
q ∈ Q_Ω →
∃ x, x ∈ Ω ∧ MsgInstanceOfFrontierItem C P x = some q
leP :
{x : FrontierItem // x ∈ Ω} →
{y : FrontierItem // y ∈ Ω} → Prop
leP_partial_order : PartialOrder leP
```

```
prefS :
SyncCall → Set {x : FrontierItem // x ∈ Ω}
suffS :
SyncCall → Set {x : FrontierItem // x ∈ Ω}
pred_S :
{x : FrontierItem // x ∈ Ω} → Option SyncAnchor
succ_S :
{x : FrontierItem // x ∈ Ω} → Option SyncAnchor
read_anchor_defined :
∀ x,
SequentialSignalReadItem x.val → pred_S x ≠ none ∧ succ_S x = none
write_anchor_defined :
∀ x,
SequentialSignalWriteItem x.val → succ_S x ≠ none ∧ pred_S x = none

-- Witness-dependent facts are kept outside DynamicUniverseData. This breaks the
-- former circularity while still proving that W.universe P is exactly the
-- executed dynamic universe selected by W.
structure WitnessUniverseWF
(C : ComparisonInstance)
(W : WitnessBundle C)
(P : Process) where
executed_only :
∀ x, x ∈ (W.universe P).Ω → ExecutedInWitness C W P x
pred_S_anchor_realized :
∀ x a,
SequentialSignalReadItem x.val →
(W.universe P).pred_S x = some a →
SyncAnchorRealizedInUniverse C P W (W.universe P).Ω a
succ_S_anchor_realized :
∀ x a,
SequentialSignalWriteItem x.val →
(W.universe P).succ_S x = some a →
SyncAnchorRealizedInUniverse C P W (W.universe P).Ω a
pred_S_is_closest_preceding_anchor :
∀ x a,
```

SequentialSignalReadItem x.val →
(W.universe P).pred_S x = some a →
ClosestPrecedingDynamicSyncAnchor C P W a x.val
succ_S_is_closest_succeeding_anchor :
∀ x a,
SequentialSignalWriteItem x.val →
(W.universe P).succ_S x = some a →
ClosestSucceedingDynamicSyncAnchor C P W a x.val
signal_read_ordered_at_pred_S :
∀ x a,
SequentialSignalReadItem x.val →
(W.universe P).pred_S x = some a →
SignalItemClock C W x.val = SyncAnchorClock C W a
signal_write_ordered_at_succ_S :
∀ x a,
SequentialSignalWriteItem x.val →
(W.universe P).succ_S x = some a →
SignalItemClock C W x.val = SyncAnchorClock C W a

def QOmegaOfProcess
(C : ComparisonInstance)
(W : WitnessBundle C)
(P : Process) : Set DynMsgInstance :=
(W.universe P).Q_Ω

MsgInstanceOfFrontierItem is defined only for message frontier items. Pure synchronization and signal-anchor items map to none. The pred_S and succ_S maps in W.universe P place ordinary sequential signal reads and writes at their realized dynamic synchronization anchors. START_P and FINISH_P are explicitly represented here as distinct reset-epoch-indexed dynamic occurrences; a later reset creates a new epoch and new start/finish occurrence identifiers.

The Q_Ω field is therefore fixed by W.universe P. It is not recomputed from a state, and the reference and post sides do not choose independent universes. State-dependent predicates such as NextFront, BoundaryReq, ChannelEnabled, and ChannelDisabled range over this same fixed witness universe.

Non-blocking calls need explicit modeling:
· failed PushNB / PopNB polls are executed dynamic instances in Qexec, but produce no Σ-event;
· successful non-blocking calls produce the corresponding committed Push_c(v) / Pop_c(v) event;
· repeated NB calls are distinct dynamic source instances even if a request signal remains physically asserted.
This matches the document's issue/commit treatment of NB calls.

structure NonBlockingInstanceWF
(C : ComparisonInstance)
(P : Process)
(W : WitnessBundle C)
(U : DynamicUniverseData C P) where

nb_instances_are_nonpersistent :
∀ q,
q ∈ U.Q_exec →
NonBlocking q →
¬ PersistentBlockingRequestInstance C P q

nb_reexecution_distinct_instances :
∀ q_1 q_2 t_1 t_2,
q_1 ∈ U.Q_exec →
q_2 ∈ U.Q_exec →
NonBlocking q_1 →
NonBlocking q_2 →
SameSourceNBPollSite q_1 q_2 →
ExecutedAtCycle C W q_1 t_1 →
ExecutedAtCycle C W q_2 t_2 →
$t_1 \neq t_2$ →
$q_1 \neq q_2$

nb_physical_assertion_does_not_identify_instances :
∀ q_1 q_2 t_1 t_2,
q_1 ∈ U.Q_exec →
q_2 ∈ U.Q_exec →
NonBlocking q_1 →

```
NonBlocking q₂ →
SameEndpoint q₁ q₂ →
EndpointRequestContinuouslyAsserted C W q₁.channel t₁ t₂ →
ExecutedAtCycle C W q₁ t₁ →
ExecutedAtCycle C W q₂ t₂ →
t₁ ≠ t₂ →
q₁ ≠ q₂

nb_not_fair_action :
∀ q,
q ∈ U.Q_exec →
NonBlocking q →
¬ ∃ h, FairAction.push q h ∨ FairAction.pop q h
```

The last field is only a type-level sanity condition for the fairness layer: B2 fairness ranges over persistent blocking Push/Pop requests and real synchronization calls. A successful NB call may still contribute the ordinary committed Push_c / Pop_c Σ-event, but the NB call instance itself does not become a persistent FairAction.

4. Reactive environment and modeling contracts

4.1 Reactive environment

"Fixed stimulus" should be modeled as a causal global strategy over Σ-history, not merely as a fixed stream of per-process inputs.

```
structure ReactiveEnvironment (Σ : Type) where
  inputAt : SigmaHistory Σ → ExternalInputs
  causal :
   ∀ h₁ h₂,
    SamePrefix h₁ h₂ →
    inputAt h₁ = inputAt h₂
```

This directly addresses the earlier issue that reactive stimulus must be interpreted as the same global Σ-causal behavior, not a function of local process history.

4.2 Modeling contracts

Direct-input contracts, array-access contracts, and latency-sensitive local-protocol isolation contracts should be parallel top-level modeling contracts, not partly folded into RRules and partly left separate.

The scheduling-rules document treats cadence-sensitive retry/timeout polling as latency-sensitive local protocol logic. In the Lean development this should be an explicit modeling contract, not a hidden theorem-side convention. Failed non-blocking polls still remain ε-steps with no committed Push_c / Pop_c Σ-event; the contract instead states that a polling site whose externally visible behavior depends on the number or cadence of failed polls is outside the ordinary latency-insensitive Appendix I/J source core unless it has been isolated, snoop-aligned, or supplied with an equivalent explicit cycle-accurate local protocol model at the chosen boundary.

```
inductive LocalProtocolHandling
| ordinaryLatencyInsensitiveCore
| isolatedBoundary
| snoopAligned
| explicitCycleAccurateModel

structure LatencySensitiveLocalProtocolContract
(C : ComparisonInstance) where
handlingOf :
Process → SourceLoc → LocalProtocolHandling

cadence_sensitive_nb_sites_handled :
∀ P loc,
CadenceSensitiveNBPollingSite C P loc →
handlingOf P loc = LocalProtocolHandling.isolatedBoundary ∨
handlingOf P loc = LocalProtocolHandling.snoopAligned ∨
handlingOf P loc = LocalProtocolHandling.explicitCycleAccurateModel

ordinary_core_excludes_unhandled_cadence :
∀ P loc,
handlingOf P loc = LocalProtocolHandling.ordinaryLatencyInsensitiveCore →
¬ CadenceSensitiveNBPollingSite C P loc

isolated_boundary_has_stable_observable_interface :
∀ P loc,
handlingOf P loc = LocalProtocolHandling.isolatedBoundary →
LatencyInsensitiveOrExplicitlyModeledBoundary C P loc
```

snoop_aligned_sites_have_admissible_witness :
∀ P loc,
handlingOf P loc = LocalProtocolHandling.snoopAligned →
SnoopAlignedLocalProtocolSite C P loc

cycle_accurate_sites_have_explicit_model :
∀ P loc,
handlingOf P loc = LocalProtocolHandling.explicitCycleAccurateModel →
ExplicitCycleAccurateLocalProtocolModel C P loc

structure ModelingContracts (C : ComparisonInstance) where
directInputs : DirectInputContract C
arrays : ArrayAccessMappingRules C
latencySensitiveLocalProtocols : LatencySensitiveLocalProtocolContract C

This keeps RRules close to the document's actual R-rule set and treats direct-input, external-array, and latency-sensitive local-protocol obligations as environment/modeling contracts. In particular, timeout/retry/poll-arbiter cadence is not added to the ordinary Σ alphabet; it is either local to an isolated block, selected by a snooping witness for that block, or covered by an explicit cycle-accurate local protocol model.

5. Direct-input contract and late sampling

The direct-input contract should contain only primitive environmental stability/update obligations. The fact that late sampling preserves the logical anchor should be a derived theorem, not a separate hypothesis.

The document says hls_direct_input permits late physical sampling because the environment holds the signal stable after reset, while hls_direct_input_sync permits controlled updates only at the designated synchronization handshake. The document also states that logical signal Read/Write events are timestamped at the E2 anchor, not at any later physical RTL sampling cycle; late sampling is ε-internal.

Define:

The direct-input stability rule applies only within a single reset epoch, and only to the effective static direct-input discipline after pragma precedence has been resolved. The scheduling document explicitly permits dynamic resetting during normal HW operation, including resetting of direct-input-controlled regions. It also permits a signal covered by hls_direct_input to be governed instead by an hls_direct_input_sync update contract. In that case the signal remains a direct input for late-sampling purposes, but it is not required to be stable for the entire reset epoch; it is required to change only at the designated sync-controlled update point and to remain stable between such permitted updates.

```
def NoRelevantResetBetween
(C : ComparisonInstance)
(sig : Signal)
(t₁ t₂ : Cycle) : Prop :=
¬ ∃ r,
t₁ < r ∧
r ≤ t₂ ∧
ReceiverResetAffectsSignal C sig r

-- DirectInput is the umbrella property: the signal may be physically sampled
-- late by the implementation. EffectiveStaticDirectInput is the ordinary
-- hls_direct_input case after hls_direct_input_sync precedence has been
-- resolved.
def EffectiveStaticDirectInput
(C : ComparisonInstance)
(sig : Signal) : Prop :=
DirectInput sig ∧ ¬ ∃ s, DirectInputSyncControlled sig s

-- A sync-controlled direct input is still a DirectInput, but its stability
-- obligation is the hls_direct_input_sync discipline rather than whole-epoch
-- stability. The indexed form records the controlling synchronization call;
-- the existential form is a convenient umbrella predicate.
def EffectiveSyncControlledDirectInputBy
(C : ComparisonInstance)
(sig : Signal)
(s : SyncCall) : Prop :=
DirectInput sig ∧ DirectInputSyncControlled sig s

def EffectiveSyncControlledDirectInput
(C : ComparisonInstance)
(sig : Signal) : Prop :=
∃ s, EffectiveSyncControlledDirectInputBy C sig s
-- The field syncControlledUpdateOnlyAtSync records the global discipline that
-- any value change of a sync-controlled direct input must occur at its
-- controlling synchronization handshake. The proof of late-sampling preservation
-- must not assume the desired no-value-change window as an external side
-- condition. Instead, it derives that local no-change fact from the sync-update
-- discipline plus the fact that no permitted controlling-sync update occurs in
-- the anchor-to-sample window.
```

def NoSyncControlledValueChangeBetween
(C : ComparisonInstance)
(sig : Signal)
(t_1 t_2 : Cycle) : Prop :=
¬ ∃ t,
t_1 < t ∧
t ≤ t_2 ∧
ValueChangesAt C.env sig t

def NoControllingSyncHandshakeBetween
(C : ComparisonInstance)
(s : SyncCall)
(t_1 t_2 : Cycle) : Prop :=
¬ ∃ t,
t_1 < t ∧
t ≤ t_2 ∧
SyncHandshakeOccursAt C s t

-- For a late physical sample selected for a sync-controlled direct input, the
-- anchor window must exclude any permitted update through the controlling sync.
-- This is the window fact supplied by DirectInputAnchorControlledBySync /
-- DirectInputAnchorWindow, not a raw value-stability hypothesis.
def NoPermittedSyncControlledUpdateInAnchorWindow
(C : ComparisonInstance)
(sig : Signal)
(s : SyncCall)
(anchor sampleT : Cycle) : Prop :=
NoControllingSyncHandshakeBetween C s anchor sampleT
structure DirectInputContract (C : ComparisonInstance) where
staticStableWithinResetEpoch :
∀ sig t_1 t_2,
EffectiveStaticDirectInput C sig →
AfterAllReceiversOutOfReset C sig t_1 →
t_1 ≤ t_2 →
NoRelevantResetBetween C sig t_1 t_2 →
InputValue C.env sig t_1 = InputValue C.env sig t_2

syncControlledUpdateOnlyAtSync :
∀ sig s t,
DirectInputSyncControlled sig s →
ValueChangesAt C.env sig t →
SyncHandshakeOccursAt C s t

syncControlledStableBetweenUpdates :

```
∀ sig s t₁ t₂,
EffectiveSyncControlledDirectInputBy C sig s →
AfterAllReceiversOutOfReset C sig t₁ →
t₁ ≤ t₂ →
NoRelevantResetBetween C sig t₁ t₂ →
NoSyncControlledValueChangeBetween C sig t₁ t₂ →
InputValue C.env sig t₁ = InputValue C.env sig t₂

theorem no_sync_controlled_value_change_between_from_contract
(C : ComparisonInstance)
(hDI : DirectInputContract C)
(sig : Signal)
(s : SyncCall)
(t₁ t₂ : Cycle)
(hSync : EffectiveSyncControlledDirectInputBy C sig s)
(hNoPermitted :
NoPermittedSyncControlledUpdateInAnchorWindow C sig s t₁ t₂) :
NoSyncControlledValueChangeBetween C sig t₁ t₂ :=
by
-- If the value changed in the window, syncControlledUpdateOnlyAtSync would place
-- that change at the controlling sync handshake, contradicting hNoPermitted.
intro hChange
rcases hChange with ⟨t, hLt, hLe, hValChange⟩
have hCtrl : DirectInputSyncControlled sig s := hSync.2
have hHandshake : SyncHandshakeOccursAt C s t :=
hDI.syncControlledUpdateOnlyAtSync sig s t hCtrl hValChange
exact hNoPermitted ⟨t, hLt, hLe, hHandshake⟩
```

Then prove the late-sampling preservation theorem by cases on the effective direct-input discipline. The ordinary static case uses reset-epoch stability. The sync-controlled case first derives the local no-value-change window from the controlling-sync update discipline and the absence of a permitted controlling sync update in the anchor-to-sample window; it does not take raw value stability as an independent side condition.

```
theorem lateSamplingPreservesLogicalAnchor_static
(C : ComparisonInstance)
(hDI : DirectInputContract C)
(sig : Signal)
(anchor sampleT : Cycle)
(hStatic : EffectiveStaticDirectInput C sig)
(hOutOfReset :
AfterAllReceiversOutOfReset C sig anchor)
(hWindow : DirectInputAnchorWindow C sig anchor sampleT)
(hLe : anchor ≤ sampleT)
(hNoReset :
```

```
NoRelevantResetBetween C sig anchor sampleT) :
InputValue C.env sig anchor =
InputValue C.env sig sampleT :=
hDI.staticStableWithinResetEpoch
sig
anchor
sampleT
hStatic
hOutOfReset
hLe
hNoReset
theorem lateSamplingPreservesLogicalAnchor_syncControlled
(C : ComparisonInstance)
(hDI : DirectInputContract C)
(sig : Signal)
(s : SyncCall)
(anchor sampleT : Cycle)
(hSync : EffectiveSyncControlledDirectInputBy C sig s)
(hOutOfReset :
AfterAllReceiversOutOfReset C sig anchor)
(hAnchor :
DirectInputAnchorControlledBySync C sig anchor s)
(hWindow : DirectInputAnchorWindow C sig anchor sampleT)
(hLe : anchor ≤ sampleT)
(hNoReset :
NoRelevantResetBetween C sig anchor sampleT)
(hNoPermitted :
NoPermittedSyncControlledUpdateInAnchorWindow C sig s anchor sampleT) :
InputValue C.env sig anchor =
InputValue C.env sig sampleT :=
by
have hNoChange :
NoSyncControlledValueChangeBetween C sig anchor sampleT :=
no_sync_controlled_value_change_between_from_contract
C hDI sig s anchor sampleT hSync hNoPermitted
exact
hDI.syncControlledStableBetweenUpdates
sig
s
anchor
sampleT
hSync
hOutOfReset
hLe
```

```
hNoReset
hNoChange
theorem lateSamplingPreservesLogicalAnchor
(C : ComparisonInstance)
(hDI : DirectInputContract C)
(sig : Signal)
(anchor sampleT : Cycle)
(hIsDI : DirectInput sig)
(hOutOfReset :
AfterAllReceiversOutOfReset C sig anchor)
(hWindow : DirectInputAnchorWindow C sig anchor sampleT)
(hLe : anchor ≤ sampleT)
(hNoReset :
NoRelevantResetBetween C sig anchor sampleT)
(hEffective :
EffectiveStaticDirectInput C sig ∨
∃ s,
EffectiveSyncControlledDirectInputBy C sig s ∧
DirectInputAnchorControlledBySync C sig anchor s ∧
NoPermittedSyncControlledUpdateInAnchorWindow C sig s anchor sampleT) :
InputValue C.env sig anchor =
InputValue C.env sig sampleT :=
by
cases hEffective with
| inl hStatic =>
exact
lateSamplingPreservesLogicalAnchor_static
C hDI sig anchor sampleT hStatic hOutOfReset hWindow hLe hNoReset
| inr hSyncCase =>
rcases hSyncCase with ⟨s, hSync, hAnchor, hNoPermitted⟩
exact
lateSamplingPreservesLogicalAnchor_syncControlled
C hDI sig s anchor sampleT hSync hOutOfReset hAnchor hWindow hLe hNoReset
hNoPermitted
```

Bucket-location rule:
For sequential processes, the logical direct-input Read(sig,val) / Write(sig,val) event belongs only to the E2 anchor bucket β_anchor. A physical late sample, if modeled, is an ε-step in a later bucket. The late ε-step creates no new Σ-event. lateSamplingPreservesLogicalAnchor is used only to justify the value recorded in the logical sequential anchor bucket.

This sequential anchoring rule is not applied to combinational-process signal IO.

Combinational processes do not have pred_S / succ_S synchronization anchors. Their observable direct-input Read/Write labels, when any direct input participates in combinational signal IO, are included only inside an emitted R2C fixed-point signal-observation unit for the relevant combinational component. The unit is emitted according to the v42 R2C unit-generation convention: first defined observation, later observable-slice value changes, or an explicitly modeled observer/sampler requirement. Intermediate physical samples, delta-cycle reads/writes, redundant re-evaluations, transient values before the fixed point, and R2C stutter cycles that emit no unit are ε/stutter and are not separately Σ-visible.

```
-- These anchor predicates are derived from the per-process DynamicUniverse
-- fields pred_S and succ_S. For a sequential read item x,
-- SequentialSignalReadAnchoredAt C (LogicalReadEvent sig val) anchor means
-- that x is the corresponding read frontier item and pred_S x = some anchor.
-- For a sequential write item x,
-- SequentialSignalWriteAnchoredAt C (LogicalWriteEvent sig val) anchor means
-- that x is the corresponding write frontier item and succ_S x = some anchor.
-- Thus the anchor used below is the event's actual E2/R2 anchor, not an
-- unconstrained universally quantified synchronization point.

structure DirectInputBucketWF (C : ComparisonInstance) where
sequential_read_recorded_only_at_pred_anchor :
∀ sig val anchor sampleT β,
DirectInput sig →
SequentialSignalReadEvent (LogicalReadEvent sig val) →
LogicalReadEvent sig val ∈ BucketSigmaEvents C β →
SequentialSignalReadAnchoredAt C (LogicalReadEvent sig val) anchor →
DirectInputAnchorWindow C sig anchor sampleT →
β = AnchorBucket C anchor
```

The late-sample value agreement field is reset-epoch local. It is derived from lateSamplingPreservesLogicalAnchor, not directly from whole-reset-epoch stability. It may be used only when no relevant dynamic reset occurs between the logical anchor cycle and the physical sample cycle, and when the signal's effective direct-input discipline supplies either ordinary static stability or sync-controlled stability for the anchor/sample window.

sequential_write_recorded_only_at_succ_anchor :
∀ sig val anchor β,
DirectInput sig →
SequentialSignalWriteEvent (LogicalWriteEvent sig val) →
LogicalWriteEvent sig val ∈ BucketSigmaEvents C β →
SequentialSignalWriteAnchoredAt C (LogicalWriteEvent sig val) anchor →
β = AnchorBucket C anchor

late_sample_is_epsilon_only :
∀ sig anchor sampleT,
DirectInputAnchorWindow C sig anchor sampleT →
PhysicalLateSample C sig sampleT →
EpsilonStepOnly C sig sampleT ∧
¬ ∃ val β,
PhysicalSampleEvent sig val sampleT ∈ BucketSigmaEvents C β

late_sample_value_matches_anchor :
∀ sig anchor sampleT,
DirectInput sig →
AfterAllReceiversOutOfReset C sig anchor →
DirectInputAnchorWindow C sig anchor sampleT →
anchor ≤ sampleT →
NoRelevantResetBetween C sig anchor sampleT →
(EffectiveStaticDirectInput C sig ∨
 ∃ s,
 EffectiveSyncControlledDirectInputBy C sig s ∧
 DirectInputAnchorControlledBySync C sig anchor s ∧
 NoSyncControlledValueChangeBetween C sig anchor sampleT) →
InputValue C.env sig anchor = InputValue C.env sig sampleT

This explicitly reconciles the bucket model with direct-input late sampling: CompatibleP and CompatibleSys compare the logical Σ-visible direct-input Read/Write events at the appropriate logical unit — the E2 anchor bucket for sequential-process signal IO, or the emitted component-local R2C fixed-point signal-observation unit for combinational-process signal IO — never the later physical sample micro-step.

Combinational-process signal IO uses R2C, not the direct-input late-sampling contract. The R2C rule is therefore a general combinational-process well-formedness obligation. It applies to every combinational-process signal Read/Write event in the compared Σ trace, regardless of whether the signal is marked DirectInput.

structure R2CObservationUnitWF (C : ComparisonInstance) where
selected_observable_slice_defined :
∀ comp,
R2CComponent C comp →
∃ obs, ObservableSlice C comp obs

unit_emitted_only_at_canonical_occurrence :
∀ comp unit t,
R2CUnitOfComponent C comp unit →
R2CUnitEmittedAt C unit t →
FirstDefinedR2CObservation C comp t ∨
ObservableSliceChangedSincePreviousUnit C comp t ∨
ExplicitObserverRequiresFreshR2CUnit C comp t ∨
CycleLockedR2CComponent C comp

unit_uses_unique_fixed_point :
∀ comp unit t μ,
R2CUnitOfComponent C comp unit →
R2CUnitEmittedAt C unit t →
CombFixedPointAtComponentCycle C comp t μ →
∀ e,
e ∈ unit.events →
EventReadsWritesStore e μ

unit_not_split :
∀ comp unit e_1 e_2,
R2CUnitOfComponent C comp unit →
e_1 ∈ unit.events →
e_2 ∈ unit.events →
SameObservableUnit C .ref unit e_1 e_2 ∧ SameObservableUnit C .post unit e_1 e_2

stutter_cycles_emit_no_unit :
∀ comp t,
R2CStutterCycle C comp t →
¬ ∃ unit, R2CUnitOfComponent C comp unit ∧ R2CUnitEmittedAt C unit t

matched_by_component_local_ordinal :
∀ comp k u_ref u_post,
KthR2CUnit C .ref comp k u_ref →
KthR2CUnit C .post comp k u_post →
R2CUnitValuesAgree C comp u_ref u_post

multi_input_component_has_anchor_or_confluence_proof :
∀ comp,
MultiInputR2CComponent C comp →
SingleExplicitAnchoringDomain C comp ∨
AllAdmissibleInterleavingsInduceSameR2CUnitSequence C comp ∨
R2CComponentHiddenBehindExplicitSynchronizationOrTransactor C comp

6. Temporal logic layer for B2/B3/B4/B5/B6

B2 and B3 should not be opaque Prop fields. They are temporal assumptions over infinite executions.

The document defines B2 as weak fairness over enabled actions, but v42 clarifies the enabled/selected/committed split. For message operations, ChannelEnabled means that the abstract boundary permits the operation to be selected; it does not by itself mean the transfer has already committed in that cycle. B2 therefore says: if an action remains continuously ChannelEnabled / eligible from some cycle onward, the scheduler or arbiter eventually selects it, and the selected action commits in the selected cycle. B3 remains the P_push(c) / P_pop(c) channel-progress obligation; it may be needed to discharge full/empty/rendezvous-blocked conditions before a pending transfer becomes ChannelEnabled.

Define temporal operators:

```
def AlwaysFrom
  (τ : InfiniteExecutionBucketed Σ)
  (t : Nat)
  (P : Nat → Prop) : Prop :=
  ∀ n, t ≤ n → P n

def EventuallyFrom
  (τ : InfiniteExecutionBucketed Σ)
```

```
 (t : Nat)
 (P : Nat → Prop) : Prop :=
 ∃ n, t ≤ n ∧ P n
```

Concrete fairness actions:

```
-- B2 fairness ranges only over scheduler-selectable operations.
-- For message operations, this means persistent blocking Push and Pop boundary
-- requests. Non-blocking PushNB/PopNB polls are not FairAction cases: failed NB
-- polls are ε-steps, and successful NB polls contribute the ordinary committed
-- Push/Pop Σ-event without creating a persistent scheduler-fairness obligation.
-- For synchronization, fairness ranges only over real synchronization calls
-- such as wait(), ac_wait, ac_sync, or SyncChannel calls. It does not range over
-- the auxiliary process-boundary anchors START_P and FINISH_P.
inductive FairAction
| push (q : DynMsgInstance) (h : q.kind = MsgKind.Push)
| pop (q : DynMsgInstance) (h : q.kind = MsgKind.Pop)
| sync (s : SyncCall)
```

Weak fairness:

Weak fairness is a side-parametric temporal assumption over the FairAction type above. For message operations, FairAction contains only blocking Push and blocking Pop dynamic instances. For synchronization operations, FairAction contains only real SyncCall instances, not arbitrary SyncAnchor values. Thus START_P and FINISH_P may still be used as formal R2/E2 signal anchors when they are realized in the dynamic universe, but they are not scheduler-fairness actions. Non-blocking PushNB/PopNB instances are excluded at the type level; they are handled through the NB execution/witness machinery and through ordinary committed Push/Pop Σ-events when successful.

Weak fairness applies to any valid infinite execution of the selected comparison side. In particular, it is not a post-only premise: the reference-side execution used by Exec_ref and the post-side execution used by RTL_B both carry the same WF/B2 obligation whenever that side is being used in a liveness/progress theorem.

For message actions, FairActionEnabled delegates to ChannelEnabledSide and therefore denotes abstract-boundary eligibility for selection, not an already committed transfer. Selection is a separate scheduler/arbiter fact, and the B2 commit-soundness clause below states that a selected enabled message action, or selected enabled rendezvous pair, commits in that selected cycle.

```
def FairActionCommitsAt
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(a : FairAction)
(n : Cycle) : Prop :=
FairActionSelected C side τ a n ∧
SelectedFairActionCommitsInCycle C side τ a n

def WeakFairness
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma) : Prop :=
∀ a t,
AlwaysFrom τ t (fun n => FairActionEnabled C side τ a n) →
EventuallyFrom τ t (fun n => FairActionCommitsAt C side τ a n)
```

B3 channel-progress predicates should be defined in terms of continuous endpoint-level ready/valid requests, with operation attribution supplied separately. They should not require that the complementary endpoint's continuous request be witnessed by one persistent dynamic message-operation instance.

A blocking request is persistent from cycle t through its commit cycle inclusive when the same dynamic operation instance q keeps its BoundaryReq asserted throughout that interval. For Push operations, persistence also includes payload stability while the request is outstanding, including the commit cycle itself:

```
def PersistentRequestFrom
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(q : DynMsgInstance)
(t : Nat) : Prop :=
Blocking q ∧
∀ n,
t ≤ n →
¬ CommitsBefore C side τ q n →
BoundaryReqAt C side τ q n ∧
(q.kind = MsgKind.Push →
PayloadAt C side τ q n = PayloadAt C side τ q t)
```

PersistentRequestFrom remains the right predicate for a single outstanding blocking Push or Pop. It is deliberately not used to collapse a sequence of executed non-blocking polls into one dynamic instance. Repeated PushNB/PopNB polls are distinct source-level dynamic instances, even if the concrete endpoint request signal remains asserted across adjacent cycles.

```
inductive EndpointDirection
| push
| pop

def DirectionOf
(q : DynMsgInstance) : EndpointDirection :=
match q.kind with
| MsgKind.Push   => EndpointDirection.push
| MsgKind.PushNB => EndpointDirection.push
| MsgKind.Pop    => EndpointDirection.pop
| MsgKind.PopNB  => EndpointDirection.pop

def IsPushKind
(k : MsgKind) : Prop :=
k = MsgKind.Push ∨ k = MsgKind.PushNB

def IsPopKind
(k : MsgKind) : Prop :=
k = MsgKind.Pop ∨ k = MsgKind.PopNB

def EndpointRequestAssertedAt
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(dir : EndpointDirection)
(n : Nat) : Prop :=
match dir with
| EndpointDirection.push => EndpointValidAssertedAt C side τ c n
| EndpointDirection.pop  => EndpointReadyAssertedAt C side τ c n


structure EndpointRequestWitness
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
```

```
(c : Channel)
(dir : EndpointDirection)
(n : Nat) where
endpoint_asserted :
EndpointRequestAssertedAt C side τ c dir n
source_attributed :
∃ q,
q.channel = c ∧
DirectionOf q = dir ∧
BoundaryReqAt C side τ q n
payload_present_if_push :
dir = EndpointDirection.push →
∃ v, EndpointPayloadAt C side τ c n = some v

-- Endpoint request continuity is not a forever-after-t predicate. It is scoped
-- to the interval before the relevant discharge event D. After D occurs, the
-- endpoint may drop the request or change payload for the next operation.
def EndpointRequestHoldsUntilDischarged
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(dir : EndpointDirection)
(t : Nat)
(D : Nat → Prop) : Prop :=
HoldsUntilDischarged τ t
(fun n =>
EndpointRequestWitness C side τ c dir n)
D

-- Push payload stability is also scoped only to the outstanding interval. A
-- successful transfer may be followed by a deasserted request or a different
-- payload for a later operation.
def PushPayloadStableUntilDischarged
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(t : Nat)
(D : Nat → Prop) : Prop :=
∃ v,
HoldsUntilDischarged τ t
(fun n =>
EndpointPayloadAt C side τ c n = some v)
```

D

```
def ContinuousPushRequestUntilDischarged
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(t : Nat)
(D : Nat → Prop) : Prop :=
EndpointRequestHoldsUntilDischarged C side τ c EndpointDirection.push t D ∧
PushPayloadStableUntilDischarged C side τ c t D

def ContinuousPopRequestUntilDischarged
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(t : Nat)
(D : Nat → Prop) : Prop :=
EndpointRequestHoldsUntilDischarged C side τ c EndpointDirection.pop t D

def BufferedChannel
(C : ComparisonInstance)
(c : Channel) : Prop :=
0 < capacity C c

def RendezvousChannel
(C : ComparisonInstance)
(c : Channel) : Prop :=
capacity C c = 0

def SomePopCommitOn
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(n : Nat) : Prop :=
∃ qPop,
qPop.channel = c ∧
IsPopKind qPop.kind ∧
CommitAtCycle C side τ qPop n

def SomePushCommitOn
(C : ComparisonInstance)
```

```
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(n : Nat) : Prop :=
∃ qPush,
qPush.channel = c ∧
IsPushKind qPush.kind ∧
CommitAtCycle C side τ qPush n

def MatchingTransferCommitOnChannel
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(n : Nat) : Prop :=
∃ qPush qPop,
qPush.channel = c ∧
qPop.channel = c ∧
IsPushKind qPush.kind ∧
IsPopKind qPop.kind ∧
MatchingTransferCommitOn C side τ qPush qPop n

def BufferedFullBothEndpointsRequesting
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(n : Nat) : Prop :=
BufferedChannel C c ∧
occAt C side τ c n = capacity C c ∧
EndpointRequestAssertedAt C side τ c EndpointDirection.push n ∧
EndpointRequestAssertedAt C side τ c EndpointDirection.pop n

def BufferedEmptyBothEndpointsRequesting
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(n : Nat) : Prop :=
BufferedChannel C c ∧
occAt C side τ c n = 0 ∧
EndpointRequestAssertedAt C side τ c EndpointDirection.pop n ∧
EndpointRequestAssertedAt C side τ c EndpointDirection.push n
```

```
def RendezvousBothEndpointsRequesting
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(n : Nat) : Prop :=
RendezvousChannel C c ∧
EndpointRequestAssertedAt C side τ c EndpointDirection.push n ∧
EndpointRequestAssertedAt C side τ c EndpointDirection.pop n

-- P holds continuously from t until the first discharge cycle.
-- This avoids the vacuity that would result from requiring the stalled condition
-- to remain true forever after the discharge has already occurred.
def HoldsUntilDischarged
(τ : InfiniteExecutionBucketed Σ)
(t : Nat)
(P : Nat → Prop)
(D : Nat → Prop) : Prop :=
∀ n, t ≤ n → (∀ k, t ≤ k → k < n → ¬ D k) → P n

def EventuallyDischargedFrom
(τ : InfiniteExecutionBucketed Σ)
(t : Nat)
(D : Nat → Prop) : Prop :=
∃ n, t ≤ n ∧ D n
```

Then:

```
def P_push
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel) : Prop :=
(BufferedChannel C c →
∀ t,
BufferedFullBothEndpointsRequesting C side τ c t →
ContinuousPushRequestUntilDischarged C side τ c t
(fun n =>
SomePopCommitOn C side τ c n) →
ContinuousPopRequestUntilDischarged C side τ c t
(fun n =>
SomePopCommitOn C side τ c n) →
EventuallyDischargedFrom τ t
(fun n =>
SomePopCommitOn C side τ c n))
```

```
∧
(RendezvousChannel C c →
∀ t,
RendezvousBothEndpointsRequesting C side τ c t →
ContinuousPushRequestUntilDischarged C side τ c t
(fun n =>
MatchingTransferCommitOnChannel C side τ c n) →
ContinuousPopRequestUntilDischarged C side τ c t
(fun n =>
MatchingTransferCommitOnChannel C side τ c n) →
EventuallyDischargedFrom τ t
(fun n =>
MatchingTransferCommitOnChannel C side τ c n))

def P_pop
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel) : Prop :=
(BufferedChannel C c →
∀ t,
BufferedEmptyBothEndpointsRequesting C side τ c t →
ContinuousPopRequestUntilDischarged C side τ c t
(fun n =>
SomePushCommitOn C side τ c n) →
ContinuousPushRequestUntilDischarged C side τ c t
(fun n =>
SomePushCommitOn C side τ c n) →
EventuallyDischargedFrom τ t
(fun n =>
SomePushCommitOn C side τ c n))
∧
(RendezvousChannel C c →
∀ t,
RendezvousBothEndpointsRequesting C side τ c t →
ContinuousPopRequestUntilDischarged C side τ c t
(fun n =>
MatchingTransferCommitOnChannel C side τ c n) →
ContinuousPushRequestUntilDischarged C side τ c t
(fun n =>
MatchingTransferCommitOnChannel C side τ c n) →
EventuallyDischargedFrom τ t
(fun n =>
MatchingTransferCommitOnChannel C side τ c n))
```

The removed HoldsUntilDischarged premises are not independent B3 assumptions. For bounded FIFOs, the needed full/empty persistence facts are derived from E4: if occAt C side τ c t = capacity C c and no Pop_c commits before n, then the channel remains full at n; dually, if occAt C side τ c t = 0 and no Push_c commits before n, then the channel remains empty at n. For rendezvous channels, the corresponding "both endpoints requesting until transfer" fact is exactly the conjunction of the two continuous endpoint-request premises together with RendezvousChannel C c.

Thus P_push and P_pop state only the real B3 progress obligation: once the appropriate blocked condition holds at t and both endpoint requests continue until the matching discharge event, that discharge event must eventually occur. The occupancy-stability facts needed inside proofs should be supplied as E4 derived lemmas rather than duplicated as extra hypotheses in B3.

This matches the document's conditional B3 reading: B3 forces progress only when both endpoints continuously request the matching transfer until the corresponding discharge event. A producer stalled on a full channel is not automatically a B3 violation if the consumer is not yet requesting the matching pop, and a consumer stalled on an empty channel is not automatically a B3 violation if the producer is not yet requesting the matching push.

The continuous-request premise is endpoint-level and interval-scoped. For blocking Push/Pop it may be witnessed by one persistent dynamic operation instance using PersistentRequestFrom through the commit cycle. For non-blocking polling loops it may instead be witnessed by a sequence of distinct executed PushNB/PopNB instances on the same endpoint, provided the endpoint request remains continuously asserted until discharge and each cycle's assertion has BoundaryReq attribution to the appropriate executed dynamic instance.

The payload-stability premise for Push is likewise scoped only until discharge. After the discharge event, the producer may deassert valid or present a different payload for a later operation. Thus B3 tracks the ready/valid progress premise without requiring an endpoint request or payload to remain stable forever and without misidentifying multiple NB polls as one persistent q.

B3 modularity (v42). For a single explicit abstract boundary with ordinary E4 buffered or rendezvous semantics, the corresponding B3 discharge is implied by B2 plus E4 once the complementary endpoint request in the B3 premise remains continuously asserted: in the buffered full case, occ_c = B(c) > 0 makes the complementary Pop_c continuously ChannelEnabled, so B2 forces eventual selection and commit of that Pop_c, freeing space; in the buffered empty case, occ_c = 0 < B(c) makes the complementary Push_c continuously ChannelEnabled, so B2 forces eventual selection and commit of that Push_c, providing data; and in the rendezvous case, the two persistent endpoint requests make the rendezvous pair continuously ChannelEnabled, so B2 supplies eventual rendezvous selection and commit. Lean should prove this derivation explicitly:

```
theorem B3_ordinary_boundary_from_B2_E4
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : Channel)
(hExec : InfiniteExecutionOfSide C side τ)
(hWF : WeakFairness C side τ)
(hE4 : E4OccupancyTraceWF C side τ)
(hOrd : OrdinaryE4Boundary C c) :
P_push C side τ c ∧ P_pop C side τ c
```

Here OrdinaryE4Boundary C c records that c is a single explicit abstract boundary whose ChannelEnabled predicate is exactly the ordinary E4 buffered or rendezvous availability predicate, with no elaborated staged/bundle/join/epoch-gate structure interposed. The proof uses the E4-derived occupancy-stability lemmas (full stays full until a Pop commits; empty stays empty until a Push commits), the continuous complementary endpoint-request premise of P_push / P_pop, and B2 selection-commit soundness for the continuously enabled complementary action or rendezvous pair.

B3 is nevertheless retained as the explicit named B-rule obligation in BRules and DocKStructuralAssumptions, exactly as in the v42 document: Lemma I.2, Appendix K, and the channel-validation methodology consume B3 in that modular per-channel form, and it is the form checked per explicit abstract channel in elaborated Sys_B^elab boundary chains, where the boundary in question may not satisfy OrdinaryE4Boundary. Thus B3 should be read as the modular channel-progress assumption used by the proof and checked against channel realizations, not as a claim that the ordinary single-boundary clauses are logically independent of B2 plus E4. For ordinary instances, B3_ordinary_boundary_from_B2_E4 lets the B3 field of BRules be discharged inside Lean rather than assumed separately.

The latest scheduling-rules document treats B1 as a standing determinism assumption for the post-HLS RTL_B side in the main Appendix G/J/K theorem stack. B1 is not a source determinism assumption: the source-side models Sys / Sys_B / Sys_ref may admit multiple ε- or latency-different executions having the same Σ-projection, and witness selection / snooping resolve the required source-side correspondence when needed.

For a fixed comparison instance — including the fixed initial state and fixed reactive environment / external stimulus — B1 requires uniqueness of the labeled post-side Σ-trace, not uniqueness of the full internal ε-execution. Internal ε-steps, ε-depths, and other non-observable scheduling details may differ between valid post-side executions.

```
def PostDeterministicSigmaTrace
(C : ComparisonInstance) : Prop :=
∀ τ₁ τ₂,
InfinitePostExecution C τ₁ →
InfinitePostExecution C τ₂ →
SigmaTraceOf C .post τ₁ =
SigmaTraceOf C .post τ₂

-- Process-owned or process-attributed projection of the post-side Σ-trace.
def ProcessProjectedPostSigmaTrace
(C : ComparisonInstance)
(P : Process)
(τ : InfiniteExecutionBucketed C.Sigma) : SigmaTrace C.Sigma :=
ProjectProcessOwnedOrAttributedEvents C P (SigmaTraceOf C .post τ)

def DeterministicProjectedPostTrace
(C : ComparisonInstance)
(P : Process) : Prop :=
∀ τ₁ τ₂,
InfinitePostExecution C τ₁ →
InfinitePostExecution C τ₂ →
ProcessProjectedPostSigmaTrace C P τ₁ =
ProcessProjectedPostSigmaTrace C P τ₂

theorem post_deterministic_process_projection
(C : ComparisonInstance)
(hB1 : PostDeterministicSigmaTrace C)
(P : Process) :
DeterministicProjectedPostTrace C P :=
by
intro τ₁ τ₂ hτ₁ hτ₂
unfold ProcessProjectedPostSigmaTrace
rw [hB1 τ₁ τ₂ hτ₁ hτ₂]
```

This is a projection consequence of system-level B1 under the same fixed comparison instance. It does not assert that process P is deterministic in isolation under arbitrary peer behavior, environments, or stimuli. Appendix I should consume this derived projection lemma wherever it needs uniqueness of P-owned or P-attributed post-side observations, rather than adding a separate per-process determinism assumption.

BRules / AssumptionBundle handling of B1. In the concrete Lean development, the B-rule bundle used by Appendix J and Appendix K theorem instances must carry B1 unconditionally for RTL_B under the fixed comparison instance. Equivalently, JTraceAssumptions C W, JCutpointAssumptions C W, and KAssumptions C W must expose a field or projection such as:

postDeterministicSigmaTrace : PostDeterministicSigmaTrace C

This field is the Lean representative of the standing B1 assumption. It is not a separate source-side determinism premise, and it is not conditional only on the phrase ‘when uniqueness is invoked.’ By contrast, B4 and B5 remain separate assumptions or derived structures used when ε-normalization, ε-quiescent endpoint reasoning, or uniqueness up to finite ε-stutter is needed.

Constructive ε-normalization and finite-stutter obligations for B4/B5.
B4 and B5 should not be merely opaque Prop fields when used to mechanize Lemma I.1-style arguments. They are assumptions, but they must expose enough structure to prove that internal ε-evolution cannot hide an infinite internal tail before the next Σ-action or stable wait.

This Lean strategy distinguishes two implementation-level measurements that are both silent with respect to Σ:
• intra-cycle ε-normalization: ε-microsteps inside one CycleBucket, used to reach an ε-quiescent cutpoint within a cycle; and
• cycle-advancing silent progress: one or more real clock cycles whose buckets contain no Σ-event for the process, used to model HLS-inserted FSM latency, arbitration/bookkeeping latency, failed non-blocking polls, and pipeline advances.

This distinction is only a Lean-internal decomposition. It is not a redefinition of B4. The main-document B4 premise remains a single finite ε-stutter bound for each process P: along any consecutive ε-only evolution in which P is internally advancing and is not externally stalled, P must reach either a Σ-action or a stable waiting state within a finite bound D_P.

Accordingly, ProcessEpsilonBound and IntraCycleNormalizationBound may remain separate Lean-internal implementation parameters, but neither one is the main-document B4 bound D_P. FiniteEpsilonDepth must expose a derived unified B4 consequence with one bound per process, and that derived bound is the Lean representative of the main document's D_P.

In this strategy, the identification is:

D_P ≡ MainB4Bound C P.

ProcessEpsilonBound C P is only an internal bound on cycle-advancing silent progress, and IntraCycleNormalizationBound C P is only an internal bound on intra-cycle ε-normalization. They are useful for proof engineering, but together they must imply the single main-document B4 finite-stutter obligation. Any B5 statement of the form "within max_P D_P" is therefore represented in Lean using max_P MainB4Bound C P, not max_P ProcessEpsilonBound C P.

```
def ProcessEpsilonBound
(C : ComparisonInstance)
(P : Process) : Nat :=
pipe_depth C P

def IntraCycleNormalizationBound
(C : ComparisonInstance)
(P : Process) : Nat :=
eps_norm_depth C P
-- Main-document B4 bound D_P.
-- MainB4Bound C P is the single D_P-style finite ε-stutter bound exposed to
-- the theorem stack. ProcessEpsilonBound and IntraCycleNormalizationBound are
-- Lean-internal components only; they are not separate candidates for D_P. The
-- exact arithmetic expression is not important, provided MainB4Bound dominates
-- the Lean-internal cycle-progress and bucket-normalization bounds strongly
-- enough to prove the unified B4 consequence.
def MainB4Bound
(C : ComparisonInstance)
(P : Process) : Nat :=
```

```
(ProcessEpsilonBound C P + 1) *
(2 * IntraCycleNormalizationBound C P + 1)



def IntraCycleEpsilonStep
(C : ComparisonInstance)
(side : Side)
(σ σ' : State) : Prop :=
InternalStep C side σ σ' ∧
SigmaEventsBetween C side σ σ' = ∅ ∧
σ'.cycle = σ.cycle

def IntraCycleEpsilonSegment
(C : ComparisonInstance)
(side : Side)
(σ0 σ1 : State)
(steps : List EpsStep) : Prop :=
EpsilonStepsFromTo C side σ0 steps σ1 ∧
∀ σ σ',
AdjacentInEpsilonSteps C side σ0 steps σ1 σ σ' →
IntraCycleEpsilonStep C side σ σ'

def StableWaitOrSigmaEnabled
(C : ComparisonInstance)
(side : Side)
(σ : State) : Prop :=
StableWaitCutpoint C side σ ∨
∃ e, SigmaEnabledAt C side e σ

-- Event ownership is side-parametric. In simple cases the ownership predicate
-- may reduce to a side-agnostic schema-level owner map over C.Sigma, but the
-- side parameter is still carried here so bucket membership, execution
-- well-formedness, and process attribution are checked in the same side context.
def SigmaCommitsForProcessInBucket
(C : ComparisonInstance)
(side : Side)
(P : Process)
```

```
(β : CycleBucket C.Sigma) : Prop :=
∃ e,
e ∈ β.sigmaEvents ∧
EventOwnedByProcess C side P e

def SigmaCommitsForProcessAt
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : InfiniteExecutionBucketed C.Sigma)
(n : Cycle) : Prop :=
SigmaCommitsForProcessInBucket C side P (τ n)

def ProcessSilentCycle
(C : ComparisonInstance)
(side : Side)
(P : Process)
(β : CycleBucket C.Sigma) : Prop :=
WellFormedCycleBucket C side β ∧
ProcessActiveInBucket C side P β ∧
¬ SigmaCommitsForProcessInBucket C side P β ∧
InternalProgressForProcess C side P β

def SilentCycleSegment
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : InfiniteExecutionBucketed C.Sigma)
(t0 t1 : Cycle) : Prop :=
t0 ≤ t1 ∧
∀ n,
t0 ≤ n →
n < t1 →
ProcessSilentCycle C side P (τ n)

def StableWaitOrSigmaEnabledAtCycle
(C : ComparisonInstance)
(side : Side)
```

```
(P : Process)
(τ : InfiniteExecutionBucketed C.Sigma)
(n : Cycle) : Prop :=
StableWaitOrSigmaEnabled C side ((τ n).endState) ∨
∃ e,
EventOwnedByProcess C side P e ∧
SigmaEnabledAt C side e ((τ n).endState)


structure FiniteEpsilonDepth (C : ComparisonInstance) where
-- MainB4Bound is the Lean representative of the main-document D_P. These two
-- fields record that the Lean-internal component bounds are dominated by that
-- single exposed D_P bound.
main_B4_bound_dominates_cycle_bound :
∀ P,
ProcessEpsilonBound C P ≤ MainB4Bound C P

main_B4_bound_dominates_normalization_bound :
∀ P,
IntraCycleNormalizationBound C P ≤ MainB4Bound C P

-- Cycle-level internal consequence used by Lemma I.1-style arguments.
-- This bound is intentionally stated using the Lean-internal
-- ProcessEpsilonBound. It is not the main-document D_P; it is one component
-- used to prove the unified D_P bound below.
silent_cycle_bound :
∀ side τ P t₀ t₁,
InfiniteExecutionOfSide C side τ →
SilentCycleSegment C side P τ t₀ t₁ →
(∀ n,
t₀ ≤ n →
n < t₁ →
¬ StableWaitOrSigmaEnabledAtCycle C side P τ n) →
t₁ - t₀ ≤ ProcessEpsilonBound C P

-- Main-document B4 consequence: all consecutive ε-only evolution, including
-- any intra-cycle normalization ε-steps represented inside CycleBucket, is
-- bounded by the single D_P bound before the process either commits a Σ-action
```

-- or reaches stable wait. In this Lean strategy, that D_P bound is
-- MainB4Bound C P.
unified_epsilon_stutter_bound :
∀ side τ P s,
InfiniteExecutionOfSide C side τ →
ConsecutiveEpsilonOnlySegment C side P τ s →
ProcessInternallyAdvancingThroughout C side P τ s →
NoExternalStallThroughout C side P τ s →
NoSigmaCommitOrStableWaitBeforeEnd C side P τ s →
EpsilonStepCount C side P τ s ≤ MainB4Bound C P

normalizes_or_reaches_sigma_or_wait :
∀ side τ P t_0,
InfiniteExecutionOfSide C side τ →
ProcessActiveInBucket C side P (τ t_0) →
BeginsInternalProgressTowardNextObservable C side P τ t_0 →
∃ n,
t_0 ≤ n ∧
n ≤ t_0 + MainB4Bound C P ∧
(SigmaCommitsForProcessAt C side P τ n ∨
StableWaitOrSigmaEnabledAtCycle C side P τ n)

-- Counts only ε-steps attributed to process P. A global CycleBucket may contain
-- ε-steps from several processes, so the per-process normalization bound must
-- be stated over the P-owned slice of the ε-prefix / ε-suffix, not over the
-- entire global list.
def ProcessEpsilonStepsIn
(C : ComparisonInstance)
(side : Side)
(P : Process)
(steps : List EpsStep) : Nat :=
CountEpsStepsOwnedByProcess C side P steps

-- Equality of the non-clock state components that may not change across an
-- idle quiescent bucket. A bucket may still advance the clock, so B5-style
-- idle/quiescent reasoning must not assert β.endState = β.startState unless
-- endState is defined before the tick. However, when the bucket has no
-- Σ-events and no ε-prefix/ε-suffix steps, the intra-cycle ε position must not

```
-- drift silently.
def NonClockStateEq
(C : ComparisonInstance)
(σ₀ σ₁ : State) : Prop :=
σ₀.store = σ₁.store ∧
σ₀.epsPos = σ₁.epsPos

-- Finite process universe used to form the main-document B5 maximum.
-- This is the set of processes in the selected Sys_ref / RTL_B comparison.
def ProcessUniverse
(C : ComparisonInstance) : Finset Process :=
ProcessesOfComparison C

-- Lean representation of the main-document B5 bound max_P D_P.
-- Since D_P is represented in this strategy by MainB4Bound C P,
-- SystemQuiescenceBound C is exactly max_P MainB4Bound C P over the finite
-- process universe of the selected comparison instance.
def SystemQuiescenceBound
(C : ComparisonInstance) : Nat :=
(ProcessUniverse C).sup (fun P => MainB4Bound C P)

structure QuiescentNormalization (C : ComparisonInstance) where
bucket_prefix_bounded :
∀ side β P,
WellFormedCycleBucket C side β →
ProcessEpsilonStepsIn C side P β.epsPrefix ≤ IntraCycleNormalizationBound C P
bucket_suffix_bounded :
∀ side β P,
WellFormedCycleBucket C side β →
ProcessEpsilonStepsIn C side P β.epsSuffix ≤ IntraCycleNormalizationBound C P
end_quiescent_sound :
∀ side β,
WellFormedCycleBucket C side β →
β.endQuiescent →
QuiescentAt C side β.endState


-- Starting a bucket at an ε-quiescent state only says that the incoming state
```

```
-- has no pending internal normalization. It does not forbid Σ-events in the
-- bucket, and it does not forbid ε-suffix normalization caused by those Σ-events.
-- The no-further-ε conclusion is valid only for an idle bucket in which no
-- observable action is enabled and no external/environmental change wakes the
-- system.
idle_quiescent_bucket_has_no_epsilon_steps :
∀ side β,
WellFormedCycleBucket C side β →
QuiescentAt C side β.startState →
β.sigmaEvents = ∅ →
NoObservableActionEnabledAtState C side β.startState →
NoExternalChangeInBucket C side β →
β.epsPrefix = [] ∧
β.epsSuffix = [] ∧
NonClockStateEq C β.startState β.endState

bounded_system_quiescence :
∀ side τ t,
InfiniteExecutionOfSide C side τ →
NoObservableActionEnabledAtState C side ((τ t).startState) →
NoExternalChangeFrom C side τ t →
∃ n,
t ≤ n ∧
n ≤ t + SystemQuiescenceBound C ∧
QuiescentAt C side ((τ n).endState) ∧
∀ m,
n ≤ m →
QuiescentAt C side ((τ m).startState) ∧
QuiescentAt C side ((τ m).endState) ∧
((NoExternalChangeFrom C side τ m ∧
NoObservableActionEnabledAtState C side ((τ m).startState)) →
(τ m).sigmaEvents = ∅ ∧
(τ m).epsPrefix = [] ∧
(τ m).epsSuffix = [])
```

The bounded_system_quiescence field is the Lean-level semantic assumption corresponding to B5. It is triggered from a single cycle whose start state has no enabled observable action. If no external/environmental change wakes the system, this assumption says that the system reaches the ε-fixed point within the explicit main-document bound max_P D_P.

In this Lean strategy, D_P means MainB4Bound C P. Therefore the B5 phrase "within max_P D_P" is represented by SystemQuiescenceBound C, defined as the finite maximum of MainB4Bound C P over the process universe of the selected comparison instance. The fact that each MainB4Bound C P is below SystemQuiescenceBound C, and the least-upper-bound property of SystemQuiescenceBound C, are not B5 assumptions; they are Lean lemmas proved from the Finset.sup definition of SystemQuiescenceBound C. ProcessEpsilonBound C P is only an internal cycle-progress component and is not the D_P used by B5.

After that point, later buckets remain idle only while the no-external-change condition continues to hold and no observable action becomes enabled again. Thus B5 is a bounded quiescence-closure premise from an idle start state, not a requirement to prove in advance that no observable action is disabled forever from the original cycle.

The field idle_quiescent_bucket_has_no_epsilon_steps is deliberately weaker than the invalid statement "QuiescentAt β.startState implies β.epsPrefix = [] and β.epsSuffix = [] and β.endState = β.startState." A quiescent start state may still be followed by committed Σ-events, and those Σ-events may update state and create ε-suffix normalization. Also, a bucket may advance the clock, so the idle fixed-point conclusion uses NonClockStateEq rather than literal equality of startState and endState.

B4_finiteEpsilonDepth exposes the main-document B4 consequence through MainB4Bound: each process has a single finite D_P ε-stutter bound before a Σ-action or stable wait, and in this Lean strategy that D_P bound is exactly MainB4Bound C P. ProcessEpsilonBound and IntraCycleNormalizationBound remain useful Lean-internal components, but they are not separate replacements for B4 and must not be used as the D_P in B5. The formalization must prove that the internal decomposition implies the single unified B4 finite-stutter obligation. QuiescentNormalization may still use IntraCycleNormalizationBound for bucket well-formedness and ε-quiescent cutpoint construction, but those bounds are applied per process by counting only the ε-steps owned by that process.

Bundle B-rules:

```
structure BRules (C : ComparisonInstance) where
B1_postDeterminism :
PostDeterministicSigmaTrace C

-- B2 is not post-only. It applies to any valid infinite execution of the
-- selected comparison side when that side participates in a progress/liveness
-- theorem.
B2_weakFairness :
∀ side τ,
InfiniteExecutionOfSide C side τ →
WeakFairness C side τ

-- B3 is likewise side-parametric. The P_push/P_pop obligations are required
-- for the reference-side Sys_ref execution as well as the post-side RTL_B
-- execution when the theorem uses those executions for liveness/deadlock
-- reasoning.
B3_channelProgress :
∀ side τ c,
InfiniteExecutionOfSide C side τ →
P_push C side τ c ∧ P_pop C side τ c

B4_finiteEpsilonDepth :
FiniteEpsilonDepth C
B5_quiescentNormalization :
QuiescentNormalization C
B6_idleStutterTimeDivergence :
IdleStutterTimeDivergence C

-- Theorem-scope projections of BRules. TraceBRules deliberately omits B4/B5;
-- NormalizationBRules contains exactly the finite-ε / quiescence premises used
-- by normalization and cutpoint theorems.
structure TraceBRules (C : ComparisonInstance) where
B1_postDeterminism : PostDeterministicSigmaTrace C
B2_weakFairness :
∀ side τ,
InfiniteExecutionOfSide C side τ →
WeakFairness C side τ
B3_channelProgress :
```

```
∀ side τ c,
InfiniteExecutionOfSide C side τ →
P_push C side τ c ∧ P_pop C side τ c
B6_idleStutterTimeDivergence : IdleStutterTimeDivergence C

structure NormalizationBRules (C : ComparisonInstance) where
B4_finiteEpsilonDepth : FiniteEpsilonDepth C
B5_quiescentNormalization : QuiescentNormalization C

def BRules.toTraceBRules
(C : ComparisonInstance)
(hB : BRules C) : TraceBRules C :=
{
B1_postDeterminism := hB.B1_postDeterminism
B2_weakFairness := hB.B2_weakFairness
B3_channelProgress := hB.B3_channelProgress
B6_idleStutterTimeDivergence := hB.B6_idleStutterTimeDivergence
}

def BRules.toNormalizationBRules
(C : ComparisonInstance)
(hB : BRules C) : NormalizationBRules C :=
{
B4_finiteEpsilonDepth := hB.B4_finiteEpsilonDepth
B5_quiescentNormalization := hB.B5_quiescentNormalization
}
```

B2/B3 remain temporal assumptions usable by K proofs. B4/B5 also remain assumptions, but they are now constructive assumptions: they provide the finite ε-normalization facts needed to bound epsPrefix / epsSuffix lengths in CycleBucket and to prove bounded ε-chain lemmas such as Lemma I.1.

7. Channel semantics and E4

Define channel state over the selected Sys_ref boundary.

The abstract occupancy is cycle-boundary, non-fall-through occupancy. It is indexed by the execution side, the infinite bucketed execution, the channel, and the cycle, not by the full intra-cycle state. It must not be a free bookkeeping function disconnected from the committed trace.

```
capacity : C.Sys_ref.Channel → Nat

-- Number of Σ-visible Push_c commits on channel c strictly before cycle t.
def PushCommitsBefore
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : C.Sys_ref.Channel)
(t : Cycle) : Nat :=
CountSigmaCommitsBefore C side τ t (fun e => IsPushCommitOnChannel C side e c)

-- Number of Σ-visible Pop_c commits on channel c strictly before cycle t.
def PopCommitsBefore
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : C.Sys_ref.Channel)
(t : Cycle) : Nat :=
CountSigmaCommitsBefore C side τ t (fun e => IsPopCommitOnChannel C side e c)

-- Initial occupancy at the selected comparison boundary. In the usual reset-empty
-- case this is 0. In accounted-initial-occupancy cases it is supplied explicitly
-- by the comparison instance / elaboration package.
initialOcc :
ComparisonInstance → Side → C.Sys_ref.Channel → Nat

-- Commit-history-derived cycle-boundary occupancy.
-- For B(c)=0 rendezvous channels this value is always 0; rendezvous enablement
-- is governed by same-cycle matching endpoint requests, not stored occupancy.
def occAt
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : C.Sys_ref.Channel)
(t : Cycle) : Nat :=
if capacity C c = 0 then
0
```

else
initialOcc C side c

+ PushCommitsBefore C side τ c t

- PopCommitsBefore C side τ c t

For convenience, one may define a state-indexed abbreviation only by projecting the state to its cycle, and only relative to the execution whose state is being inspected:

def occAtState
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : C.Sys_ref.Channel)
(σ : State) : Nat :=
occAt C side τ c σ.cycle

This abbreviation is not a second occupancy notion. In particular, occupancy is invariant under ε-steps and under same-cycle Σ-events because it is defined from commits strictly before the cycle boundary:

theorem occ_invariant_within_cycle
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma)
(c : C.Sys_ref.Channel)
(σ_1 σ_2 : State) :
σ_1.cycle = σ_2.cycle →
occAtState C side τ c σ_1 = occAtState C side τ c σ_2

The definition above is the canonical occupancy used by E4, ChannelEnabled, ChannelDisabled, WFG construction, B3 progress reasoning, and drain-closure reasoning. If the concrete Lean development keeps occAt opaque for proof-engineering reasons, it must instead assume the following well-formedness condition and use it everywhere occAt is consumed:

structure E4OccupancyTraceWF
(C : ComparisonInstance)
(side : Side)
(τ : InfiniteExecutionBucketed C.Sigma) where
occ_matches_committed_prefix :

```
∀ c t,
occAt C side τ c t =
if capacity C c = 0 then
0
else
initialOcc C side c

+ PushCommitsBefore C side τ c t

- PopCommitsBefore C side τ c t
initial_occupancy_bounded :
∀ c,
initialOcc C side c ≤ capacity C c
occupancy_bounded :
∀ c t,
occAt C side τ c t ≤ capacity C c
no_underflow :
∀ c t,
PopCommitsBefore C side τ c t ≤
initialOcc C side c + PushCommitsBefore C side τ c t
rendezvous_has_no_stored_occupancy :
∀ c t,
capacity C c = 0 →
occAt C side τ c t = 0
```

Reset-empty alignment (A10). The main document's formal model is reset-empty: after each reset boundary at which the compared execution begins or restarts, every buffered channel with $B(c) > 0$ has logical occupancy 0 at the chosen Σ boundary, and the Appendix J/K occupancy equations and the per-channel Push/Pop precedence lemma are interpreted relative to that reset-empty initial channel state. Document-aligned theorem instances therefore set initialOcc C side c = 0, and A10_reset_empty_fifo / ResetEmptyFifoAssumption C entails exactly that equation at each compared reset boundary. A nonzero accounted initialOcc is an extension outside the main document's empty-FIFO model: per v42, preloaded channel contents must instead be modeled explicitly as post-reset producer actions, or supported by a separately stated initialized-state extension whose obligations are then carried by the comparison instance. The initialOcc parameter is kept in occAt only so that such an explicitly flagged extension does not require re-deriving the occupancy infrastructure; it must not be used to smuggle preloaded contents into a theorem instance that claims the documented A10 assumption.

```
-- Domain for one-cycle issue-time request reasoning.
-- This domain includes every executed source-level message-operation instance,
-- including failed PushNB / PopNB polls that produce no committed Σ-event.
def BoundaryReqIssueDomainSide
(C : ComparisonInstance)
(side : Side)
(q : DynMsgInstance)
(σ : State) : Prop :=
∃ P,
q ∈ QExecSide C side P ∧
MessageOperationInstance q ∧
OwnsMessageEndpoint P q ∧
IssuedAtState C side q σ

-- Domain for persistent operation-attributive request/channel reasoning.
-- This is the domain used by ChannelEnabled, ChannelDisabled, Front, NextFront,
-- automatic flush, and WFG construction. Failed NB-only polls are not included
-- merely because they executed.
def BoundaryReqPersistentDomainSide
(C : ComparisonInstance)
(side : Side)
(q : DynMsgInstance)
(σ : State) : Prop :=
∃ P,
q ∈ QOmegaSide C side P ∧
MessageOperationInstance q ∧
OwnsMessageEndpoint P q

-- General BoundaryReq domain.
-- A request may be meaningful either because q is issuing in the current cycle
-- or because q is a persistent message-operation frontier instance.
def BoundaryReqDomainSide
(C : ComparisonInstance)
(side : Side)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqIssueDomainSide C side q σ ∨
BoundaryReqPersistentDomainSide C side q σ
```

Executed failed non-blocking polls may belong to Qexec,P for replay and witness purposes while remaining absent from QOmega,P and Ω_P. Such polls do acquire the one-cycle BoundaryReq-at-issue obligation when they issue, via BoundaryReqIssueDomainSide. They do not acquire persistent BoundaryReqPersistentDomainSide, ChannelEnabled, ChannelDisabled, Front, NextFront, automatic-flush, or WFG obligations merely because they executed.

```
BoundaryReqSide :
ComparisonInstance → Side → DynMsgInstance → State → Prop

-- ChannelEnabledSide is the v42 E4 availability/eligibility predicate at the
-- chosen abstract boundary. It is not itself a Commit predicate. For B(c)>0 it
-- is BoundaryReq conjoined with the cycle-boundary FIFO availability predicate;
-- for B(c)=0 it is BoundaryReq conjoined with the same-cycle complementary
-- endpoint request predicate. Hidden grant/arbitration/back-pressure must not
-- be added as an extra conjunct here unless it is represented as explicit
-- Sys_B^elab boundary structure.
ChannelEnabledSide :
ComparisonInstance → Side → DynMsgInstance → State → Prop

ChannelSelectedSide :
ComparisonInstance → Side → DynMsgInstance → State → Prop

def ChannelSelectionCommits
(C : ComparisonInstance)
(side : Side) : Prop :=
∀ τ q σ t,
StateAtCycleInPrefix C side τ t σ →
BoundaryReqPersistentDomainSide C side q σ →
ChannelEnabledSide C side q σ →
ChannelSelectedSide C side q σ →
CommitAtCycle C side τ q t

def ChannelDisabledSide
(C : ComparisonInstance)
(side : Side)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqPersistentDomainSide C side q σ ∧
BoundaryReqSide C side q σ ∧
¬ ChannelEnabledSide C side q σ
```

For post-side automatic-flush and issue/commit formulas, use the following abbreviations:

```
def BoundaryReqIssueDomain
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
```

BoundaryReqIssueDomainSide C .post q σ

def BoundaryReqPersistentDomain
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqPersistentDomainSide C .post q σ

def BoundaryReqDomain
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqDomainSide C .post q σ

def BoundaryReq
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
BoundaryReqSide C .post q σ

def ChannelEnabled
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
ChannelEnabledSide C .post q σ

def ChannelDisabled
(C : ComparisonInstance)
(q : DynMsgInstance)
(σ : State) : Prop :=
ChannelDisabledSide C .post q σ

BoundaryReqSide, ChannelEnabledSide, and ChannelDisabledSide are the canonical side-indexed predicates. The shorter BoundaryReq / ChannelEnabled / ChannelDisabled forms are post-side abbreviations used only where the section is already explicitly about RTL_B / post-side behavior.

BoundaryReqSide is used under BoundaryReqDomainSide. This includes both one-cycle issue requests, via BoundaryReqIssueDomainSide, and persistent frontier/channel requests, via BoundaryReqPersistentDomainSide.

ChannelEnabledSide and ChannelDisabledSide are used only under BoundaryReqPersistentDomainSide or its post-side abbreviation BoundaryReqPersistentDomain. Thus failed non-blocking polls in

Q_exec,P \ Ω_P may satisfy the Appendix C one-cycle request-at-issue rule, but they still do not become message-operation frontier items and do not acquire ChannelEnabled, ChannelDisabled, Front, NextFront, automatic-flush, or WFG obligations merely because they executed.

ChannelEnabled is allowed to inspect the settled boundary-request state σ for persistent message-operation frontier instances, but its capacity/occupancy component must use the commit-history-linked value occAt C side τ q.channel σ.cycle. Thus E4 enablement is evaluated against the beginning-of-cycle abstract occupancy, with no same-cycle fall-through through ε-settling or same-cycle Σ-events.

Capacity cases:

inductive CapacityKind
| rendezvous -- B(c) = 0
| buffered -- B(c) > 0

Derived E4 lemmas:
BufferedPushEnabled
BufferedPopEnabled
RendezvousEnabled
FIFOOrderPreservation
NoDropNoDuplicate
OccupancyUpdatePush
OccupancyUpdatePop

The occupancy-update lemmas are derived from the commit-history definition of occAt, or from E4OccupancyTraceWF if occAt is kept opaque. They are not independent facts about a free primitive. In particular, OccupancyUpdatePush and OccupancyUpdatePop must prove that moving from cycle t to t+1 changes occAt exactly according to the Push_c / Pop_c commits in cycle t, subject to the non-fall-through rule and the single-transfer-per-cycle endpoint constraints.

E4 should be parameterized by Sys_ref, so in elaborated cases it ranges over explicit elaborated channels, not merely outer channels.

B-faithfulness is a distinct implementation-side obligation, not a consequence of E4 or trace compatibility. It is stated per back-annotated explicit abstract channel as a behavioral refinement of the selected concrete boundary by the explicit E4 abstract channel:

```
structure BFaithful
(C : ComparisonInstance)
(c : Channel) : Prop where
boundary_complete :
∀ e,
RTLVisibleTransferOnChannel C c e →
CrossesChosenAbstractBoundaryAsCommittedPushOrPop C c e
capacity_upper_bound :
∀ τ t,
RealizedUnpoppedItemsAtChosenBoundary C τ c t ≤ capacity C c
boundary_behavior_exact :
∀ τ t action,
RealizedBoundaryActionPermitted C τ c t action ↔
E4BoundaryActionPermitted C τ c t action
no_hidden_storage_or_credit :
NoUnmodeledStorageOrCreditBeyondChosenBoundary C c
no_hidden_enablement :
NoHiddenGrantArbitrationOrBackpressureAtBoundary C c ∨
RepresentedByExplicitSysBElabBoundaryStructure C c
internal_motion_epsilon :
InternalChannelMovementBetweenBoundaryCommitsIsEpsilon C c
evidence :
BFaithfulnessEvidence C c
```

The capacity field is intentionally named capacity_upper_bound: by itself it proves only that concrete occupancy does not exceed B(c). Exactness is supplied by boundary_behavior_exact together with boundary completeness, absence of hidden storage/credit, absence or explicit elaboration of hidden enablement, and Σ-opacity of internal movement. The concrete implementation-assumption bundle carries the full behavioral obligation:

```
structure ImplementationAssumptions
(C : ComparisonInstance) where
e4ChannelRealization : E4ChannelRealizationConformance C
bFaithful :
∀ c,
BackAnnotatedExplicitAbstractChannel C c → BFaithful C c
otherNamedImplementationObligations : OtherImplementationAssumptions C
```

Consequently, JTraceAssumptions, JCutpointAssumptions, JSigmaDUTSafetyAssumptions, IConstructionAssumptions, and KStructuralAssumptions import B-faithfulness mechanically through ImplementationAssumptions. J.AbsConf remains separate because it is witness- and selected-cutpoint-dependent.

8. Appendix Q elaboration interface

Appendix Q should be introduced early. In the updated document, elaboration is not a late proof afterthought: in elaborated cases, Sys_ref = Sys_B^elab(E), and all references to B, occ, BoundaryReq, ChannelEnabled/Disabled, Front_P, and WFG are interpreted over the explicit channels of the elaborated model.

As in the ordinary E4 semantics, elaborated-channel occupancy is cycle-boundary, non-fall-through occupancy. Thus occ_E(d,t) and A_E(c,t) are indexed by cycle t. State-indexed forms such as occ_E(d,σ) or A_E(c,σ), when used informally at ε-quiescent cutpoints, are only abbreviations for occ_E(d,σ.cycle) and A_E(c,σ.cycle).

Chan_outer should be a parameter fixed by the chosen outer Sys_B, not a field chosen internally by the elaboration package.

The channel-level elaboration projection follows Appendix Q's single-valued optional form. A proof-internal elaborated channel is either hidden from the outer DUT/testbench boundary or assigned to one outer channel. Separately, a proof-internal token/event may account for multiple outer Σ_DUT-visible channel effects in bundled or joined cases.

```
structure ElaborationPackage
(Sys_B : System)
(Chan_outer : Type)
(Σ_DUT : Type) where
Sys_elab : System
Chan_elab : Type
finite_chan_elab : Fintype Chan_elab
ElabToken : Type
B_E : Chan_elab → Nat
occ_E : Chan_elab → Cycle → Nat

-- Channel-level projection/accounting support.
```

```
-- Π_elab d = some c means that elaborated channel d belongs to the
-- channel-level fiber for outer channel c. Π_elab d = none corresponds to
-- the ⊥ case in Appendix Q: d is proof-internal and hidden from the outer
-- DUT/testbench boundary.
Pi_elab : Chan_elab → Option Chan_outer

-- Token/event-level accounting support.
-- Π_token_elab tok is the set of outer channels accounted for by one
-- proof-internal token/event. This may contain multiple outer channels for
-- bundled or joined work items. Token-level multiplicity does not make the
-- channel-level Π_elab relation multi-valued.
Pi_token_elab : ElabToken → Set Chan_outer
tokenOfSigma : Sys_elab.Sigma → Option ElabToken

-- Bucketed outer-projection of the elaborated proof-internal execution.
-- This replaces any flat Trace-level projection. The projection may hide
-- proof-internal staged/bundle/join events, but it must preserve the
-- cycle-bucket structure needed for ε-quiescent cutpoints, same-cycle atomic
-- units, and temporal B2/B3 reasoning.
piSigmaBucket :
CycleBucket Sys_elab.Sigma →
CycleBucket Σ_DUT

piSigmaFinite :
ExecutionBucketed Sys_elab.Sigma →
ExecutionBucketed Σ_DUT

piSigmaInfinite :
InfiniteExecutionBucketed Sys_elab.Sigma →
InfiniteExecutionBucketed Σ_DUT

piSigma_bucket_wf : Prop
piSigma_finite_agrees_with_bucket : Prop
piSigma_infinite_agrees_with_bucket : Prop

A_E : Chan_outer → Cycle → Nat
outer_bucketed_execution_preservation : Prop
occupancy_preservation : Prop
```

internal_refinement : Prop
no_hidden_cross_channel_disablement : Prop
wfg_visibility : Prop
combinational_wellformedness : Prop

For each outer channel c, the channel-level fiber of the elaboration is the single-valued optional fiber

Π_elab^{-1}(c) = { d : Chan_elab | Pi_elab d = some c }.

This channel-level optional map matches Appendix Q's Π_elab : Chan_elab(E) → Chan_outer ∪ {⊥}. The token-level projection Pi_token_elab is a Lean-side refinement used to make bundled/joined multi-channel accounting explicit when one proof-internal staged/bundle/join token accounts for several outer Σ_DUT-visible channel effects. It is not an additional named component of Appendix Q.1; the scheduling-rules document folds this accounting into A_E and the corresponding Q.1 accounting clauses. The bucketed projection piSigmaBucket, together with piSigmaFinite and piSigmaInfinite, and the occupancy/accounting map A_E must be consistent with the channel-level optional projection Pi_elab and with this Lean-side token-level accounting refinement. In particular, elaboration projection is not a flat event-list projection: it preserves the cycle-bucket decomposition needed for ε-quiescent cutpoints, same-cycle atomic units, R2C fixed-point buckets, and B2/B3 temporal reasoning, while allowing proof-internal staged/bundle/join events to be hidden from the outer Σ_DUT view.

For convenience, the elaboration package may also define state-indexed abbreviations by projecting the state to its cycle:

occ_E_atState : Chan_elab → State → Nat
occ_E_atState d σ := occ_E d σ.cycle

A_E_atState : Chan_outer → State → Nat
A_E_atState c σ := A_E c σ.cycle

These abbreviations are not separate occupancy or accounting notions. They are invariant under ε-steps and same-cycle Σ-events:

occ_E_invariant_within_cycle :

$\forall$ d σ_1 σ_2,
σ_1.cycle = σ_2.cycle →
occ_E_atState d σ_1 = occ_E_atState d σ_2

A_E_invariant_within_cycle :
$\forall$ c σ_1 σ_2,
σ_1.cycle = σ_2.cycle →
A_E_atState c σ_1 = A_E_atState c σ_2

Reference choice:
inductive ReferenceKind
| ordinarySysB
| elaborated

structure SysRefChoice where
outerSysB : System
Chan_outer : Type
kind : ReferenceKind
elab? : Option (ElaborationPackage outerSysB Chan_outer Σ_DUT)
Sys_ref : System
sys_ref_matches_kind : Prop

For ordinarySysB, sys_ref_matches_kind entails Sys_ref = outerSysB.
For elaborated, sys_ref_matches_kind entails that there exists an elaboration package E with elab? = some E and Sys_ref = E.Sys_elab.

When SysRefChoice is embedded in a ComparisonInstance C, the field C.sys_ref_consistent additionally requires C.Sys_ref = C.refChoice.Sys_ref. Thus SysRefChoice records the construction/justification of the selected reference system, while ComparisonInstance.Sys_ref is the single reference system used by the theorem statements. There is no freedom for these two references to diverge.

This also handles sync-boundary epoch gates: if proof-relevant withholding exists, it must be represented by explicit gate channels in Sys_B^elab(E), not by hidden gating on an otherwise E4-enabled outer channel. The earlier review correctly flagged this as a point that must be explicit in the Lean strategy.

Locality note. Although ElaborationPackage is presented as a single structure parameterizing the theorem instance, its components (Sys_elab, Chan_elab, Π_elab, Π_token,elab, π_Σ,E, A_E) are constructed locally per process and per channel boundary. The channel-level projection Π_elab is single-valued and optional, matching Appendix Q's map Chan_elab(E) → Chan_outer ∪ {⊥}. The token-level projection Π_token,elab is the object allowed to map one proof-internal staged/bundle/join token to multiple outer Σ_DUT-visible channels. Each per-process elaboration is determined by that process's local channel structure (number of inputs, sync-boundary discipline, pipelining configuration) and is independent of the elaboration chosen for any other process. The system-level ElaborationPackage is the disjoint composition of these per-process pieces. Treating E as a theorem-instance parameter is the right Lean encoding of "the verifier has chosen per-process elaborations and the proof takes them as given"; it is not a claim that the verifier must reason globally to construct E.

9. R-rules, E-rules, and modeling contracts

The R-rule bundle should include only the document's actual R-rules plus an optional Lean-side array-rule pointer if desired. It should not include a fictional R5_syncBoundaryNoCrossing; sync-boundary crossing is E5. The document explicitly labels "Messages cannot cross their immediate synchronization boundary" as E5.

However, R5 itself is a real R-rule: R5 (Channel capacity semantics; Sys vs. Sys_B). In the Lean strategy, R5 should record the chosen source/reference-model capacity convention: raw Sys uses rendezvous capacity, while Sys_B / Sys_ref uses the selected capacity B(c), with B(c)=0 interpreted as rendezvous and B(c)>0 interpreted as cycle-boundary, non-fall-through bounded FIFO. The concrete enablement and occupancy-update obligations remain the E4 channel-capacity semantics; R5 records which source/reference capacity semantics are being used by the selected comparison instance.

The predicates RendezvousChannel C c and BufferedChannel C c are definitional abbreviations for the zero-capacity and positive-capacity cases. Therefore the substantive R5 obligation is not the tautological case split
capacity C c = 0 → RendezvousChannel C c and
0 < capacity C c → BufferedChannel C c. The substantive obligation is that the selected source reference model uses the intended capacity convention for the comparison instance: in the raw Sys case this means rendezvous capacity; in

Sys_B / Sys_B^elab this means the selected B(c) capacities, with E4 supplying the corresponding ChannelEnabled / ChannelDisabled and occupancy-update semantics.

```
def R2CCombinationalFixedPointRule
(C : ComparisonInstance) : Prop :=
SequentialOnly C ∨
∃ hR2C : R2C_WF C,
R2CEventsObservedOnlyInStutterCanonicalUnits C hR2C ∧
R2CUnitsMatchedByComponentLocalOrdinal C hR2C

def R5ChannelCapacitySemantics
(C : ComparisonInstance) : Prop :=
SysRefUsesSelectedCapacitySemantics C

structure RRules (C : ComparisonInstance) where
R1_syncOrder : Prop
R2_signalAnchorSequential : Prop
R2C_combinationalFixedPoint :
R2CCombinationalFixedPointRule C
R3_syncSemantics : Prop
R4_messageIssueOrder : Prop
R5_channelCapacitySemantics :
R5ChannelCapacitySemantics C
```

R4_messageIssueOrder is deliberately named only as an issue-order rule. It records the source-induced no-reverse discipline for message-operation issue. It does not impose a separate commit-order rule; commit timing is constrained by E4 channel availability, FIFO/rendezvous semantics, backpressure, and the chosen Sys_ref / RTL_B comparison instance.

R2 source well-formedness (RULE 1 / RULE 2). In v42, RULE 1 conformance is a source well-formedness obligation for proof-backed flows, not merely a recommended diagnostic. R2_signalAnchorSequential therefore carries, in addition to the anchoring equations, the following obligations: every observable sequential-process signal Read/Write has a unique real bounding synchronization anchor on each dynamic path on which it occurs (with a particular START_P occurrence usable only when the modeling convention includes that reset-epoch start as a real synchronization event in S, and a particular FINISH_P occurrence usable only for an actually terminating process in the same reset epoch); no observable Read/Write event is assigned clock time ∞; and if a blocking message-passing operation separates a signal read or write from its corresponding synchronization anchor, the theorem instance is admissible only if the model is rewritten so the signal IO is adjacent to its intended anchor, or an explicit source-level well-formedness discharge shows that the signal IO has a unique unconditional real anchor on every dynamic path and cannot observe a value inconsistent with that anchor. In the Lean development these facts are part of C.WellFormed together with the read_anchor_defined / write_anchor_defined / reset-epoch-indexed anchor-realized fields of DynamicUniverse; an unresolved tool warning is not by itself a discharge.

E-rules:

structure ERules (C : ComparisonInstance) where
E1_sync_order_preservation : Prop
E2_signal_anchor_preservation : Prop
E3_message_order_preservation : Prop
E4_channel_capacity_semantics : Prop
E5_sync_boundary_preservation : Prop

Modeling contracts live in the single ModelingContracts structure defined in Section 4.2. That structure includes direct-input contracts, array-access mapping rules, and latency-sensitive local-protocol handling:

structure ModelingContracts (C : ComparisonInstance) where
directInputs : DirectInputContract C
arrays : ArrayAccessMappingRules C
latencySensitiveLocalProtocols : LatencySensitiveLocalProtocolContract C

Section 9 should not introduce a second, narrower definition of ModelingContracts. The same contract bundle is used by the R/E-rule theorem instances, Appendix I/J witness construction, Appendix L snooping/WC, direct-input late-sampling proofs, external-array proofs, and the cadence-sensitive local-protocol isolation obligations.

10. Array access mapping

External arrays are not merely internal Store. The document states that external arrays are mapped onto IO operations by an array access mapping layer, and that array accesses before a synchronization call must commit no later than the sync commit, while accesses after the sync must commit strictly after it. It also permits transformations such as caching, merging, splitting, and reordering when allowed by the mapping layer.

Define:

```
inductive MemoryEvent
| read  (array : ArrayId) (addr : Addr) (value : Value)
| write (array : ArrayId) (addr : Addr) (value : Value)

structure ArrayAccessMapping where
 sourceArrayAccess : SourceArrayAccess → List CoreIOEvent
 commitTime       : SourceArrayAccess → Cycle
 fixedLatency     : SourceArrayAccess → Nat

 conflictFreeReorderAllowed :
  SourceArrayAccess → SourceArrayAccess → Prop

 transformedEquivalent :
  SourceArrayAccess → List SourceArrayAccess → Prop
```

Rules:

Array accesses are constrained by the same dynamic synchronization intervals as the ordinary source-order frontier, but SourceArrayAccess values are not themselves members of prefS(s) or suffS(s). The array-access mapping layer must therefore provide an explicit bridge from source array accesses to the dynamic frontier/synchronization interval structure.

```
structure ArraySyncIntervalBridge
(C : ComparisonInstance) where
arrayFrontierItemOf :
SourceArrayAccess → Option FrontierItem

array_item_in_dynamic_universe :
∀ a x,
arrayFrontierItemOf a = some x →
∃ P W,
x ∈ (W.universe P).Ω

array_before_sync_matches_prefS :
```

```
∀ a s x,
arrayFrontierItemOf a = some x →
ArrayAccessBeforeSync C a s ↔
FrontierItemInPrefS C x s

array_after_sync_matches_suffS :
∀ a s x,
arrayFrontierItemOf a = some x →
ArrayAccessAfterSync C a s ↔
FrontierItemInSuffS C x s

array_commit_cycle_is_mapping_commit_cycle :
∀ a,
ArrayCommitCycle C a = ArrayCommitTime a

-- External/shared array accesses are interpreted through the array access
-- mapping layer, not as ordinary internal Store updates. The mapping layer
-- supplies the externally relevant memory operation, the addressed storage
-- location, the commit cycle at which that storage is accessed, and the
-- declared memory semantics used to compare Sys_ref and RTL_B.

inductive ArrayOpKind
| read
| write

structure ExternalStorageLocation where
storage : ExternalArrayStorage
addr : ArrayAddress

def ArrayMappedOpKind
(C : ComparisonInstance)
(a : ArrayAccess) : ArrayOpKind :=
...

def ArrayMappedLocation
(C : ComparisonInstance)
(a : ArrayAccess) : ExternalStorageLocation :=
...
```

```
def ArrayWriteValue
(C : ComparisonInstance)
(a : ArrayAccess) : Option Value :=
...

def ArrayReadValue
(C : ComparisonInstance)
(a : ArrayAccess) : Option Value :=
...

def SameExternalStorageLocation
(C : ComparisonInstance)
(a₁ a₂ : ArrayAccess) : Prop :=
ArrayMappedLocation C a₁ = ArrayMappedLocation C a₂

-- The mapping layer may declare that a transformation splits, merges, caches,
-- or otherwise rewrites source array accesses. Such transformations are legal
-- only when the mapped external memory behavior is equivalent under the
-- declared memory semantics.
def MappingEquivalentUnderSharedMemorySemantics
(C : ComparisonInstance)
(a : ArrayAccess)
(transformed : ArrayAccess) : Prop :=
MappingEquivalent a transformed ∧
PreservesExternalMemoryBehavior C a transformed

-- Serialization order for accesses that touch the same externally visible
-- storage location and are not proven conflict-free. This is a semantic order
-- over mapped memory commits, not a new source-order rule for ordinary core IO.
def SharedStorageSerializedBefore
(C : ComparisonInstance)
(a₁ a₂ : ArrayAccess) : Prop :=
SameExternalStorageLocation C a₁ a₂ ∧
ArrayCommitTime a₁ < ArrayCommitTime a₂

def SharedStorageSameCycleCompatible
(C : ComparisonInstance)
```

(a_1 a_2 : ArrayAccess) : Prop :=
SameExternalStorageLocation C a_1 a_2 ∧
ArrayCommitTime a_1 = ArrayCommitTime a_2 ∧
DeclaredSameCycleMemoryPolicyAllows C a_1 a_2

def SharedStorageConflict
(C : ComparisonInstance)
(a_1 a_2 : ArrayAccess) : Prop :=
SameExternalStorageLocation C a_1 a_2 ∧
¬ conflictFreeReorderAllowed a_1 a_2

def SharedStorageCoherent
(C : ComparisonInstance) : Prop :=
∀ aRead aWrite,
ArrayMappedOpKind C aRead = ArrayOpKind.read →
ArrayMappedOpKind C aWrite = ArrayOpKind.write →
SameExternalStorageLocation C aWrite aRead →
ArrayCommitTime aWrite < ArrayCommitTime aRead →
NoInterveningWriteToSameLocation C aWrite aRead →
ArrayReadValue C aRead = ArrayWriteValue C aWrite

structure ArrayAccessMappingRules (C : ComparisonInstance) where
syncIntervalBridge :
ArraySyncIntervalBridge C
beforeSyncCommitsNoLater :
∀ a s,
ArrayAccessBeforeSync C a s →
ArrayCommitTime a ≤ SyncCommitTime s
afterSyncCommitsAfter :
∀ a s,
ArrayAccessAfterSync C a s →
SyncCommitTime s < ArrayCommitTime a
issueConsistentWithFixedLatency :
∀ a,
ArrayCommitTime a =
ArrayIssueTime a + ArrayFixedLatency a

-- Transformed operations must preserve the memory behavior declared by the

-- array access mapping layer, including any externally visible shared-memory
-- effects.
transformedOpsRespectMapping :
∀ a transformed,
TransformedFrom a transformed →
MappingEquivalentUnderSharedMemorySemantics C a transformed

-- Reordering of external array/memory accesses is legal only when the mapping
-- layer declares the reordering conflict-free.
conflictFreeReorderingOnly :
∀ a_1 a_2,
Reordered a_1 a_2 →
conflictFreeReorderAllowed a_1 a_2

-- If two mapped accesses touch the same externally visible storage location and
-- are not covered by a declared conflict-free reordering proof, the mapping
-- layer must provide a serialization relation or an explicit same-cycle memory
-- policy. This is the shared-storage rule required by the main scheduling
-- document for external/shared memories.
sharedStorageConflictsAreSerializedOrPolicyCompatible :
∀ a_1 a_2,
$a_1 \neq a_2$ →
SharedStorageConflict C a_1 a_2 →
SharedStorageSerializedBefore C a_1 a_2 ∨
SharedStorageSerializedBefore C a_2 a_1 ∨
SharedStorageSameCycleCompatible C a_1 a_2

-- Reads observe the value determined by the declared serialization/coherence
-- semantics of the shared external storage.
sharedStorageCoherence :
SharedStorageCoherent C

For early milestones, it is acceptable to assume:

AllExternalArrayAccessesAlreadyMappedToCoreIO C

but the full proof strategy should include array mapping.

11. R2C combinational fixed point

ComparisonInstance now has:
combModel? : Option CombComposition

R2C should use that field when combinational processes are in scope. In v42, combinational Read/Write events are not emitted unconditionally in every observable cycle bucket. Instead, each explicitly modeled combinational process or combinational network component C_comb has a selected observable slice Obs(C_comb). The composed combinational network still has a unique ε-quiescent fixed-point valuation μ_t in each compared cycle, but a Σ-visible R2C fixed-point signal-observation unit is emitted only at canonical observation occurrences: (1) the first cycle in the compared execution in which the component's observable slice is defined and observed; (2) a later cycle in which the vector of fixed-point values on Obs(C_comb) differs from the most recent emitted unit for that component; (3) a cycle in which an explicitly modeled observer/sampler requires a fresh observation unit; or (4) every relevant cycle for an explicitly cycle-locked R2C component.

The Lean model should therefore preserve four facts:
1. R2C fixed-point values are read from the composed valuation μ_t.
2. The composed valuation is obtained from explicit local combinational process components, rather than from an unexplained monolithic global object.
3. Σ-visible R2C units are stutter-canonicalized per component unless the component is explicitly cycle-locked.
4. Cross-model matching is by component identity and local R2C occurrence ordinal, not by equality of global cycle bucket number or by co-bucketing with unrelated events.

Define:
structure R2CObservableLabel where
proc : Process
sig : Signal
direction : SignalDirection -- read or write

structure LocalCombNetwork where
proc : Process
combStepLocal : Store → Store → Prop

```
reads : Finset Signal
writes : Finset Signal
obsSlice : Finset R2CObservableLabel
localDeterministic : Prop

structure CombNetwork where
combStep : Store → Store → Prop
deterministic : Prop

structure CombComposition where
components :
Process → Option LocalCombNetwork

composed :
CombNetwork

component_processes_are_combinational :
∀ P N,
components P = some N →
CombinationalProcess P

component_owner_matches_process :
∀ P N,
components P = some N →
N.proc = P

observable_slice_well_formed :
∀ P N label,
components P = some N →
label ∈ N.obsSlice →
label.proc = P ∧ (label.sig ∈ N.reads ∨ label.sig ∈ N.writes)

local_step_embeds_in_composed_step :
∀ P N σ σ',
components P = some N →
N.combStepLocal σ σ' →
composed.combStep σ σ'
```

composed_step_generated_by_components :
∀ σ σ',
composed.combStep σ σ' →
∃ P N,
components P = some N ∧
N.combStepLocal σ σ'

resolved_driver_or_disjoint_write_policy :
∀ P_1 P_2 N_1 N_2 sig,
components P_1 = some N_1 →
components P_2 = some N_2 →
sig ∈ N_1.writes →
sig ∈ N_2.writes →
$P_1 \neq P_2$ →
ResolvedMultiDriverPolicy composed sig

composed_deterministic_from_components :
(∀ P N, components P = some N → N.localDeterministic) →
composed.deterministic

The scheduling document's R2C rule requires deterministic ε-settling to a unique observable fixed point. It does not globally require structural combinational acyclicity for every admissible combinational network. Pure combinational oscillation, ambiguous unresolved multi-driver behavior, intentional combinational-loop behavior outside a unique fixed-point interpretation, or non-convergent ε-behavior are excluded through the unique-fixed-point obligation below unless they are represented by explicit state or synchronization elsewhere.

Structural acyclicity may still be imposed as a separate well-formedness requirement for specific elaboration/coupler constructions in Appendix Q, but it is not a universal requirement of LocalCombNetwork, CombNetwork, or CombComposition itself.

def CombQuiescent (N : CombNetwork) (σ : State) : Prop :=
¬ ∃ σ', N.combStep σ.store σ'.store

def CombFixedPointAtCycle
(N : CombNetwork)

```
(t : Cycle)
(μ : Store) : Prop :=
ReachesByCombSteps N t μ ∧
IsFixedPoint N μ

def ComponentParticipatesInCycle
(C : ComparisonInstance)
(M : CombComposition)
(P : Process)
(t : Cycle) : Prop :=
∃ N,
M.components P = some N ∧
CombinationalProcessActiveAt C P t

def R2CUnitVector
(C : ComparisonInstance)
(M : CombComposition)
(P : Process)
(t : Cycle)
(μ : Store) : Vector Value :=
ValuesOfObservableSlice C M P t μ

def R2CUnitEmissionCanonical
(C : ComparisonInstance)
(M : CombComposition)
(P : Process)
(t : Cycle) : Prop :=
FirstDefinedR2CObservation C M P t ∨
R2CUnitVectorChangedSincePreviousEmission C M P t ∨
ExplicitObserverRequiresFreshR2CUnit C M P t ∨
CycleLockedR2CComponent C M P
```

Well-formedness:

```
structure R2C_WF (C : ComparisonInstance) where
composition_present :
C.combModel?.isSome

uniqueComposedFixedPoint :
```

```
∀ t M,
C.combModel? = some M →
∃! μ, CombFixedPointAtCycle M.composed t μ

component_events_use_composed_fixed_point :
∀ P e t μ M,
C.combModel? = some M →
ComponentParticipatesInCycle C M P t →
CombReadWriteEventOfProcess C P e →
e ∈ R2CUnitEventsAt C P t →
CombFixedPointAtCycle M.composed t μ →
EventReadsWritesStore e μ

all_comb_events_have_component :
∀ P e t M,
C.combModel? = some M →
CombReadWriteEventOfProcess C P e →
e ∈ R2CUnitEventsAt C P t →
∃ N,
M.components P = some N

units_emit_only_at_canonical_occurrences :
∀ P t M,
C.combModel? = some M →
R2CUnitEmitted C P t →
R2CUnitEmissionCanonical C M P t

stutter_cycles_have_no_sigma_unit :
∀ P t M,
C.combModel? = some M →
¬ R2CUnitEmissionCanonical C M P t →
¬ R2CUnitEmitted C P t

component_local_occurrence_matching :
∀ P k u_ref u_post,
KthR2CUnit C .ref P k u_ref →
KthR2CUnit C .post P k u_post →
R2CUnitValuesAgree C P u_ref u_post
```

```
multi_input_well_formed :
∀ P M,
C.combModel? = some M →
MultiInputR2CComponent C M P →
SingleExplicitAnchoringDomain C M P ∨
AllAdmissibleInterleavingsInduceSameR2CUnitSequence C M P ∨
R2CComponentHiddenBehindExplicitSynchronizationOrTransactor C M P

def R2CEventsObservedOnlyInStutterCanonicalUnits
(C : ComparisonInstance)
(hR2C : R2C_WF C) : Prop :=
∀ P e t,
CombReadWriteEventOfProcess C P e →
e ∈ BucketSigmaEvents C t →
∃ M μ unit,
C.combModel? = some M ∧
ComponentParticipatesInCycle C M P t ∧
CombFixedPointAtCycle M.composed t μ ∧
R2CUnitEmitted C P t ∧
e ∈ unit.events ∧
EventReadsWritesStore e μ

def R2CUnitsMatchedByComponentLocalOrdinal
(C : ComparisonInstance)
(hR2C : R2C_WF C) : Prop :=
∀ P k u_ref u_post,
KthR2CUnit C .ref P k u_ref →
KthR2CUnit C .post P k u_post →
R2CUnitValuesAgree C P u_ref u_post
```

This formulation keeps the semantic fixed point global where R2C needs it, but makes the proof obligation modular and stutter-canonical: each combinational process contributes a local component and selected observable slice, the composed network used for μ_t is justified by the CombComposition fields, and the emitted observable sequence is compared by component-local occurrence ordinal. Thus R2C fixed-point equality remains a system-level statement over μ_t, while conformance remains checkable process-by-process plus an explicit composition/driverresolution and multi-input anchoring/confluence obligation.

If combinational processes are out of scope for an early milestone, use:
SequentialOnly C
and postpone R2C_WF.

12. Issue/Commit well-formedness

Issue and Commit should be relational plus existence/uniqueness, not necessarily computable total functions.

The Issue/Commit layer is side-parametric and uses the same bucketed execution carrier as the rest of the strategy. It must not introduce a separate flat Trace type.

```
def SystemOfSide
(C : ComparisonInstance) : Side → System
| Side.ref => C.Sys_ref
| Side.post => C.RTL_B

abbrev SidePrefix
(C : ComparisonInstance)
(side : Side) : Type :=
Prefix (SystemOfSide C side)

abbrev SideInfiniteExecution
(C : ComparisonInstance)
(side : Side) : Type :=
InfiniteExecutionBucketed C.Sigma

IssueAt :
(C : ComparisonInstance) →
(side : Side) →
SidePrefix C side →
DynMsgInstance →
Cycle →
Prop
```

```
CommitAt :
(C : ComparisonInstance) →
(side : Side) →
SidePrefix C side →
DynMsgInstance →
Cycle →
Payload →
Prop

def CommitAtCycle
(C : ComparisonInstance)
(side : Side)
(τ : SidePrefix C side)
(q : DynMsgInstance)
(t : Cycle) : Prop :=
∃ v, CommitAt C side τ q t v
```

Well-formedness:

```
def BoundaryRequestAtIssue
(C : ComparisonInstance) : Prop :=
∀ side τ q tIssue σ,
IssuedInPrefix C side τ q →
IssueAt C side τ q tIssue →
StateAtCycleInPrefix C side τ tIssue σ →
BoundaryReqIssueDomainSide C side q σ ∧
BoundaryReqSide C side q σ ∧
((q.kind = MsgKind.Push ∨ q.kind = MsgKind.PushNB) →
PayloadAtState C side q σ = PayloadAtIssue C side τ q tIssue)

def BlockingBoundaryRequestPersistence
(C : ComparisonInstance) : Prop :=
∀ side τ q tIssue t σ,
IssuedInPrefix C side τ q →
IssueAt C side τ q tIssue →
StateAtCycleInPrefix C side τ t σ →
tIssue ≤ t →
¬ CommitsBefore C side τ q t →
Blocking q →
```

BoundaryReqPersistentDomainSide C side q σ ∧
BoundaryReqSide C side q σ ∧
(q.kind = MsgKind.Push →
PayloadAtState C side q σ = PayloadAtIssue C side τ q tIssue)

```
def BoundaryRequestSoundness
(C : ComparisonInstance) : Prop :=
BoundaryRequestAtIssue C ∧
BlockingBoundaryRequestPersistence C
```

```
-- SameEndpointDirection means the same concrete ready/valid endpoint, not
-- merely the same Push-vs-Pop direction. In particular, two Push operations on
-- different channels are not the same endpoint direction and may be outstanding
-- concurrently.
def SameEndpointDirection
(q₀ q₁ : DynMsgInstance) : Prop :=
q₀.channel = q₁.channel ∧
DirectionOf q₀ = DirectionOf q₁
```

```
def StableAttribution
(C : ComparisonInstance) : Prop :=
∀ side τ q₀ q₁ t σt σnext,
SameEndpointDirection q₀ q₁ →
q₀ ≠ q₁ →
Blocking q₀ →
StateAtCycleInPrefix C side τ t σt →
StateAtCycleInPrefix C side τ (t + 1) σnext →
BoundaryReqDomainSide C side q₀ σt →
BoundaryReqSide C side q₀ σt →
BoundaryReqDomainSide C side q₁ σnext →
BoundaryReqSide C side q₁ σnext →
¬ EndpointCommitAt C side τ q₀.endpoint t →
¬ EndpointCommitAt C side τ q₁.endpoint t →
False
```

Equivalently, the first premise may be written as q_0.endpoint = q_1.endpoint if the concrete Lean development defines endpoint as the pair (q.channel, DirectionOf q). The important point is that StableAttribution forbids retagging one still-outstanding request to a different operation on the same physical endpoint; it does not forbid independent blocking pushes or pops on different channels.

BoundaryRequestSoundness has two parts. BoundaryRequestAtIssue applies to every issued source-level message-operation instance q, including PushNB and PopNB instances that fail and therefore produce no committed Σ-event. Such a non-blocking poll must still assert the appropriate endpoint request in its issue cycle. BlockingBoundaryRequestPersistence is the additional non-withdrawal rule for blocking Push/Pop instances: after issue and before commit, the same dynamic operation instance keeps its request asserted, and a Push keeps its payload stable.

StableAttribution applies when the earlier outstanding request q_0 is a blocking Push/Pop. In that case, attribution cannot change to any distinct same-endpoint operation before a commit on that endpoint direction, whether the later candidate q_1 is blocking or non-blocking. For non-blocking PushNB / PopNB calls, request assertion at the issue cycle does not by itself imply persistence beyond that issue cycle, and continuous assertion of the endpoint request signal does not by itself collapse later executed non-blocking call instances into the same dynamic instance q. Repeated executed NB polls remain distinct source-level dynamic instances unless the witness explicitly identifies the same still-outstanding request instance.

```
-- Dynamic source interval between synchronization anchors.
-- This is witness-specific: only executed dynamic instances in the selected
-- witness universe are classified. Untaken branches and unexecuted loop
-- iterations have no SyncIntervalId.

structure SyncInterval where
proc : Process
pred? : Option SyncCall
succ? : Option SyncCall
intervalId : Nat

def SyncIntervalId
(C : ComparisonInstance)
(W : WitnessBundle C)
```

(q : DynMsgInstance) : Option SyncInterval :=
DynamicSourceIntervalOf C W q

structure SyncIntervalWF
(C : ComparisonInstance)
(W : WitnessBundle C) where
interval_defined_for_frontier_messages :
∀ P q,
q ∈ (W.universe P).Q_Ω →
∃ I, SyncIntervalId C W q = some I

same_interval_iff_same_anchors :
∀ q_0 q_1 I_0 I_1,
SyncIntervalId C W q_0 = some I_0 →
SyncIntervalId C W q_1 = some I_1 →
I_0 = I_1 ↔
I_0.proc = I_1.proc ∧
I_0.pred? = I_1.pred? ∧
I_0.succ? = I_1.succ? ∧
I_0.intervalId = I_1.intervalId

pref_suff_consistent :
∀ s q,
q ∈ prefS s ∨ q ∈ suffS s →
∃ I, SyncIntervalId C W q = some I

def SameSyncInterval
(C : ComparisonInstance)
(W : WitnessBundle C)
(q_0 q_1 : DynMsgInstance) : Prop :=
∃ I,
SyncIntervalId C W q_0 = some I ∧
SyncIntervalId C W q_1 = some I

def SourceIssuesBackToBackAfterCommit
(C : ComparisonInstance)
(W : WitnessBundle C)

(side : Side)
(τ : SidePrefix C side)
(q_0 q_1 : DynMsgInstance)
(t : Cycle) : Prop :=
SameEndpointDirection q_0 q_1 ∧
SameSyncInterval C W q_0 q_1 ∧
CommitAtCycle C side τ q_0 t ∧
NextSourceMessageOpOnEndpoint C q_0 q_1 ∧
IssuedInPrefix C side τ q_1 ∧
SourceControlReachesAndIssuesAt C W side τ q_1 (t + 1)

def BackToBackIssueWF
(C : ComparisonInstance)
(W : WitnessBundle C) : Prop :=
∀ side τ q_0 q_1 t,
SourceIssuesBackToBackAfterCommit C W side τ q_0 q_1 t →
EndpointRequestContinuouslyAsserted C side τ q_0.endpoint t (t + 1) →
IssueAt C side τ q_1 (t + 1)

The SameSyncInterval component of SourceIssuesBackToBackAfterCommit prevents the back-to-back issuance rule from carrying a continuously asserted endpoint request across an explicit wait(), SyncChannel, ac_sync, or other synchronization boundary. Cross-boundary issue timing is governed instead by SyncBoundaryIssueDiscipline / E5.

The continuous physical endpoint assertion is not what causes q_1 to issue. It only explains why the boundary signal may remain asserted across the q_0/q_1 transition without a visible deassertion bubble. The source-level cause is the separate SourceControlReachesAndIssuesAt premise: q_1 has actually been reached and issued as the next same-endpoint operation in the same synchronization interval. Without that premise, a held valid/ready signal could be incorrectly used to infer issuance of a source operation that control flow has not yet reached.

def SyncBoundaryIssueDiscipline
(C : ComparisonInstance) : Prop :=
∀ side τ q_0 q_1 s tSync t_0 t_1,

q_0 ∈ prefS s →
q_1 ∈ suffS s →
SyncCommitAt C side τ s tSync →
IssueAt C side τ q_0 t_0 →
IssueAt C side τ q_1 t_1 →
t_0 ≤ tSync ∧ tSync < t_1

def NBDynamicInstanceDiscipline
(C : ComparisonInstance) : Prop :=
∀ P q_0 q_1,
IsNB q_0 →
IsNB q_1 →
ExecutedInProcess C P q_0 →
ExecutedInProcess C P q_1 →
SameSourceCallSiteButDifferentExecution q_0 q_1 →
$q_0 \neq q_1$

structure IssueCommitWF
(C : ComparisonInstance)
(W : WitnessBundle C) where
issue_exists :
∀ side τ q,
IssuedInPrefix C side τ q →
∃ t, IssueAt C side τ q t
issue_unique :
∀ side τ q t_1 t_2,
IssueAt C side τ q t_1 →
IssueAt C side τ q t_2 →
$t_1 = t_2$
commit_exists_unique :
∀ side τ q,
CommitsInPrefix C side τ q →
∃! tp : Cycle × Payload,
CommitAt C side τ q tp.1 tp.2
boundary_request_soundness :
BoundaryRequestSoundness C
stable_attribution :
StableAttribution C

back_to_back_issue :
BackToBackIssueWF C W
sync_boundary_discipline :
SyncBoundaryIssueDiscipline C
nb_dynamic_instance_discipline :
NBDynamicInstanceDiscipline C

Derived timestamp:
noncomputable def IssueTime
(hWF : IssueCommitWF C W)
(h : IssuedInPrefix C side τ q) : Cycle :=
Classical.choose (hWF.issue_exists side τ q h)

This preserves Appendix C's use of Issue(q) as an auxiliary timestamp while avoiding an unnecessary computability requirement.

13. Automatic flush and sync-boundary policy

Define:
Pending P σ
Front P σ
NextFront P σ
BlockingMsgNextFront P σ
Done P σ
Remain P σ

Automatic flush must be stated over the same dynamic universe and source-order objects used by Appendix I/J/K. In particular, it must not be an opaque Prop bundle.

Section 13 reasons primarily about FrontierItem values, while Section 7's BoundaryReqSide / ChannelEnabledSide / ChannelDisabledSide predicates are operation-attributive predicates over DynMsgInstance values. Therefore Section 13 uses the following item-level wrappers. These wrappers are not new semantic predicates; they merely read a message-operation frontier item through its owning dynamic message instance.

```
def MsgOfFrontierItemSide
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem) : Option DynMsgInstance :=
MsgInstanceOfFrontierItem C x.proc x

def BoundaryReqItemSide
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem)
(σ : State) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
BoundaryReqPersistentDomainSide C side q σ ∧
BoundaryReqSide C side q σ

def ChannelEnabledItemSide
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem)
(σ : State) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
ChannelEnabledSide C side q σ

def ChannelDisabledItemSide
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem)
(σ : State) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
ChannelDisabledSide C side q σ

def PayloadAtFrontierItemState
(C : ComparisonInstance)
(side : Side)
```

(x : FrontierItem)
(σ : State) : Payload :=
PayloadAtState C side (Classical.choose (MsgOfFrontierItemSide_isSome C side x)) σ

def FrontierItemIssuedAt
(C : ComparisonInstance)
(side : Side)
(τ : SidePrefix C side)
(x : FrontierItem)
(t : Cycle) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
IssueAt C side τ q t

def FrontierItemCommitsBeforeCycle
(C : ComparisonInstance)
(side : Side)
(τ : SidePrefix C side)
(x : FrontierItem)
(t : Cycle) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
CommitsBefore C side τ q t

def FrontierItemCommitsBetween
(C : ComparisonInstance)
(side : Side)
(τ : SidePrefix C side)
(x : FrontierItem)
(t_0 t : Cycle) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
CommitsBetween C side τ q t_0 t

def FrontierItemCommitsAtCycle
(C : ComparisonInstance)
(side : Side)
(τ : SidePrefix C side)

(x : FrontierItem)
(t : Cycle) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
CommitAtCycle C side τ q t

def FrontierBlocked
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State) : Prop :=
Front C side P σ ≠ ∅ ∧
∀ x,
x ∈ Front C side P σ →
BlockingMessageFrontierItem x ∧
BoundaryReqItemSide C side x σ ∧
ChannelDisabledItemSide C side x σ

def LaterThanBlockedFrontier
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State)
(y : FrontierItem) : Prop :=
∃ x,
x ∈ Front C side P σ ∧
SourceOrderLt C side P x y

def FrontierRemainsBlockedOver
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side)
(t_0 t : Cycle) : Prop :=
∀ n σn,
t_0 ≤ n →
n ≤ t →

```
StateAtCycleInPrefix C side τ n σn →
FrontierBlocked C side P σn

def NoNewLaterWorkWhileBlocked
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) : Prop :=
∀ t0 t σ0 y,
StateAtCycleInPrefix C side τ t0 σ0 →
FrontierBlocked C side P σ0 →
FrontierRemainsBlockedOver C side P τ t0 t →
LaterThanBlockedFrontier C side P σ0 y →
¬ FrontierItemIssuedAt C side τ y t

def NoWithdrawalWhileBlocked
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) : Prop :=
∀ t0 t σ0 σt x,
StateAtCycleInPrefix C side τ t0 σ0 →
StateAtCycleInPrefix C side τ t σt →
t0 ≤ t →
x ∈ Front C side P σ0 →
BoundaryReqItemSide C side x σ0 →
¬ FrontierItemCommitsBetween C side τ x t0 t →
BoundaryReqItemSide C side x σt ∧
(IsPushFrontierItem x →
PayloadAtFrontierItemState C side x σt =
PayloadAtFrontierItemState C side x σ0)

def FrontierMinimality
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) : Prop :=
∀ t σ x y,
```

StateAtCycleInPrefix C side τ t σ →
x ∈ Front C side P σ →
y ∈ Remain C side P σ →
SourceOrderLt C side P y x →
False

The next predicates separate two obligations that should not be conflated.

First, there is the Kε / A11-style WFG-inertness obligation. It applies only to intervals that contain no observable Σ-commit. It says that pure internal / ε-only movement while a process remains blocked at the same frontier cannot create a new wait-for cause.

Second, there is the active drain-only automatic-flush discipline. It permits already-issued downstream work to commit while the process is blocked at a minimal frontier. Such drain commits may be observable Σ-events, so they must not be excluded by an InternalOrEpsilonOnlyBetween premise. The discipline is instead that no new later source work is issued past the blocked frontier, already asserted requests are not withdrawn before commit, and any observable commit owned by the blocked process during the blocked interval is attributable to an already-issued drainable downstream request.

def WFGInertForProcess
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ_0 σt : State) : Prop :=
∀ Q,
WFGEdge C side σ_0 P Q ↔ WFGEdge C side σt P Q

def NoNewBlockingFrontierCause
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ_0 σt : State) : Prop :=
∀ x,
x ∈ Front C side P σt →

BlockingMessageFrontierItem x →
ChannelDisabledItemSide C side x σt →
∃ y,
y ∈ Front C side P σ_0 ∧
BlockingMessageFrontierItem y ∧
SameBlockingCause C side P y x

def WFGInertUnderInternalOnlyBetween
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) : Prop :=
∀ t_0 t σ_0 σt,
StateAtCycleInPrefix C side τ t_0 σ_0 →
StateAtCycleInPrefix C side τ t σt →
t_0 ≤ t →
FrontierBlocked C side P σ_0 →
FrontierRemainsBlockedOver C side P τ t_0 t →
InternalOrEpsilonOnlyBetween C side τ t_0 t →
WFGInertForProcess C side P σ_0 σt ∧
NoNewBlockingFrontierCause C side P σ_0 σt

def DrainableDownstreamRequest
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ_0 : State)
(x : FrontierItem) : Prop :=
AlreadyIssuedBeforeBlockedCutpoint C side P σ_0 x ∧
DownstreamOfBlockedFrontier C side P σ_0 x ∧
BlockingMessageFrontierItem x ∧
BoundaryReqItemSide C side x σ_0

def DrainOnlyCommitDiscipline
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) : Prop :=

∀ t_0 t σ_0 e,
StateAtCycleInPrefix C side τ t_0 σ_0 →
t_0 ≤ t →
FrontierBlocked C side P σ_0 →
FrontierRemainsBlockedOver C side P τ t_0 t →
e ∈ SigmaEventsAtCycleInPrefix C side τ t →
EventOwnedByProcess C side P e →
ObservableCommitEvent e →
∃ x,
DrainableDownstreamRequest C side P σ_0 x ∧
EventCommitsFrontierItem C side x e ∧
¬ FrontierItemCommitsBeforeCycle C side τ x t_0 ∧
FrontierItemCommitsAtCycle C side τ x t

def DrainOnlyDownstreamProgress
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) : Prop :=
WFGInertUnderInternalOnlyBetween C side P τ ∧
DrainOnlyCommitDiscipline C side P τ

structure AutomaticFlush
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side) where
block_point :
∀ t σ,
StateAtCycleInPrefix C side τ t σ →
FrontierBlocked C side P σ →
FlushBlockedPoint C side P τ t σ

no_new_later_work :
NoNewLaterWorkWhileBlocked C side P τ

no_withdrawal :
NoWithdrawalWhileBlocked C side P τ

frontier_minimality :
FrontierMinimality C side P τ

drain_only_downstream_progress :
DrainOnlyDownstreamProgress C side P τ

sync_boundary_policy :
SyncBoundaryPolicy C side P τ

The sync-boundary policy must explicitly split the ordinary and elaborated cases. In the elaborated case, it must state a universal coverage obligation: it is not enough to exhibit one gate for one synchronization call. Every proof-relevant withholding case in which capacity belongs only to suff_S(s) and can affect producer-facing availability while s is pending must be represented by an explicit sync-boundary / epoch-gate abstract channel, and the resulting refusal must appear as ordinary ChannelDisabled behavior at that explicit boundary.

def SyncPendingAt
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side)
(s : SyncCall)
(t : Cycle) : Prop :=
SyncIssuedButNotCommitted C side P τ s t

def FutureEpochCapacityFor
(C : ComparisonInstance)
(side : Side)
(P : Process)
(s : SyncCall)
(gateChannel : Channel)
(q : DynMsgInstance) : Prop :=
q ∈ suffS_msg C side P s ∧
MessageOperationInstance q ∧
TransferWouldUseCapacityBehindGate C side P s gateChannel q

def RequiresProducerFacingWithholding

```
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side)
(s : SyncCall)
(q : DynMsgInstance)
(t : Cycle)
(σ : State) : Prop :=
StateAtCycleInPrefix C side τ t σ ∧
SyncPendingAt C side P τ s t ∧
q ∈ suffS_msg C side P s ∧
MessageOperationInstance q ∧
CapacityBelongsOnlyToPostSyncInterval C side P s q ∧
BoundaryReqPersistentDomainSide C side q σ ∧
BoundaryReqSide C side q σ

def SyncBoundaryGateCorrect
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side)
(E : ElaborationPackage C.refChoice.outerSysB C.refChoice.Chan_outer C.Sigma_DUT)
(gateChannel : Channel)
(s : SyncCall)
(q : DynMsgInstance) : Prop :=
GateChannelOf E gateChannel ∧
FutureEpochCapacityFor C side P s gateChannel q ∧
∀ t σ,
StateAtCycleInPrefix C side τ t σ →
SyncPendingAt C side P τ s t →
RequiresProducerFacingWithholding C side P τ s q t σ →
ChannelDisabledSide C side q σ

structure SyncBoundaryWithholdingCoverage
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side)
```

```
(E : ElaborationPackage C.refChoice.outerSysB C.refChoice.Chan_outer C.Sigma_DUT)
where
covered :
∀ s q t σ,
RequiresProducerFacingWithholding C side P τ s q t σ →
∃ gateChannel,
SyncBoundaryGateCorrect C side P τ E gateChannel s q

inductive SyncBoundaryPolicy
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ : SidePrefix C side)
| no_withholding_in_ordinary_SysB :
C.refChoice.kind = ReferenceKind.ordinarySysB →
NoProofRelevantSyncBoundaryWithholding C side P τ →
SyncBoundaryPolicy C side P τ
| explicit_gates_in_elab :
∀ E,
C.refChoice.kind = ReferenceKind.elaborated →
SyncBoundaryWithholdingCoverage C side P τ E →
SyncBoundaryPolicy C side P τ
```

Here suffS_msg C side P s is the message-operation-instance projection of suffS(s). The sync-boundary gate rule is stated over DynMsgInstance values because BoundaryReqSide, ChannelEnabledSide, and ChannelDisabledSide are obligations of attributed message-operation instances, not of arbitrary FrontierItem records.

In the ordinary Sys_B case, the sync-boundary clause does not add hidden gating. It asserts that no proof-relevant sync-boundary withholding is present. If such withholding is needed, the theorem instance must use Sys_B^elab(E), where every proof-relevant withholding case is covered by explicit gate structure and appears as ordinary ChannelDisabled behavior at an explicit sync-boundary / epoch-gate abstract channel.

14. Witness selector and Appendix L

The uploaded document's witness selector is a partial strategy over ε-quiescent source cutpoints and ε-modeled choices whose outcomes may affect later Σ-visible behavior; it must be causally available from the matched RTL prefix and global Σ-causal history.

Avoid circularity by defining it over the partial construction so far:

```
structure WitnessSelector (C : ComparisonInstance) where
 choose :
 (τ_post_prefix : Prefix C.RTL_B) →
 (τ_ref_so_far : Prefix C.Sys_ref) →
 (cp : ChoicePoint τ_ref_so_far) →
 Option ChoiceOutcome
 causal :
 ∀ τp τr cp out,
 choose τp τr cp = some out →
 DependsOnlyOnAvailablePrefix τp τr cp out
 consistent_with_post_prefix :
 ∀ τp τr cp out,
 choose τp τr cp = some out →
 ConsistentWithRTLPrefix τp τr cp out
```

Non-blocking witness mapping.
Failed PushNB / PopNB polls are executed dynamic instances in Q_exec but produce no Σ-event. Therefore the Post → Ref construction cannot reconstruct them from the Σ-trace alone. The witness bundle must carry an explicit partial dynamic-instance correspondence for such ε-modeled NB instances.

```
structure NBInstanceMap (C : ComparisonInstance) where
μ_Q :
DynMsgInstance → Option DynMsgInstance

maps_only_executed_nb :
∀ q_post q_ref,
μ_Q q_post = some q_ref →
ExecutedInPost C q_post ∧
ExecutedInRef C q_ref ∧
IsNB q_post ∧ IsNB q_ref
```

preserves_owner_and_interface :
∀ q_post q_ref,
μ_Q q_post = some q_ref →
q_post.proc = q_ref.proc ∧
q_post.channel = q_ref.channel ∧
q_post.kind = q_ref.kind

preserves_nb_outcome :
∀ q_post q_ref,
μ_Q q_post = some q_ref →
NBOutcomePost C q_post = NBOutcomeRef C q_ref

covers_non_cadence_control_relevant_failed_polls :
∀ q_post,
ExecutedInPost C q_post →
IsFailedNB q_post →
AffectsLaterSigmaOrFrontier C q_post →
¬ CadenceSensitiveNBInstance C q_post →
∃ q_ref, μ_Q q_post = some q_ref

cadence_sensitive_failed_polls_are_not_unhandled_core :
∀ q_post,
ExecutedInPost C q_post →
IsFailedNB q_post →
CadenceSensitiveNBInstance C q_post →
HandledByLatencySensitiveLocalProtocolContract C q_post

source_order_position_preserved :
∀ q_post q_ref,
μ_Q q_post = some q_ref →
SourceOrderKeyOf C q_post = SourceOrderKeyOf C q_ref

respects_source_order_slice :
∀ P q_1_post q_2_post q_1_ref q_2_ref,
μ_Q q_1_post = some q_1_ref →
μ_Q q_2_post = some q_2_ref →
SourceOrderLe C P (SourceOrderKeyOf C q_1_post) (SourceOrderKeyOf C q_2_post) ↔
SourceOrderLe C P (SourceOrderKeyOf C q_1_ref) (SourceOrderKeyOf C q_2_ref)

```
structure WitnessBundle (C : ComparisonInstance) where
selector : WitnessSelector C
nbMap : NBInstanceMap C
universe : ∀ P, DynamicUniverseData C P

structure WitnessBundleWF
(C : ComparisonInstance)
(W : WitnessBundle C) where
universe_wf : ∀ P, WitnessUniverseWF C W P
```

Witness compatibility:

```
def WC
(C : ComparisonInstance)
(W : WitnessBundle C) : Prop :=
(∀ constructionPrefix,
WitnessChoicesRespectMatchedPrefix C W.selector constructionPrefix) ∧
WitnessNBMapRespectsMatchedPrefix C W.nbMap ∧
WitnessChoicesAndNBMapAgree C W.selector W.nbMap ∧
LatencySensitiveLocalProtocolWC C W C.contracts.latencySensitiveLocalProtocols
```

Thus WC is the Lean-side packaging of witness selection, the μ_Q matching obligation, and the modeling-contract obligation that cadence-sensitive failed-poll behavior is not left as an unconstrained hidden timing dependence inside the ordinary source core. When E3/E5 reasoning or the Post → Ref construction refers to failed ε-modeled non-blocking call instances in $Q_{exec,P}$, the required μ_Q correspondence is supplied by W.nbMap as part of WC for non-cadence-sensitive or already boundary-selected cases. If a failed-poll sequence is cadence-sensitive because externally visible behavior depends on the number or cadence of failures, then WC is applied at the chosen observable boundary of the isolated/snoop-aligned local protocol block, not to the raw internal failed-poll cadence.

Snooping assumptions:

```
structure SnoopingWrapperAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
covers_eps_choices_affecting_boundary :
∀ P choice,
EpsChoiceAffectsLaterSigmaOrFrontier C P choice →
ChoiceCoveredBySnoopingOrWitness C W P choice
```

does_not_relabel_boundary_sigma :
∀ τ,
SnoopedExecution C W τ →
BoundarySigmaTracePreserved C W τ

selected_choices_prefix_causal :
∀ constructionPrefix,
WitnessChoicesRespectMatchedPrefix C W.selector constructionPrefix

failed_polls_remain_epsilon :
∀ q,
IsFailedNB q →
¬ ∃ e, FailedPollContributesCommittedSigmaEvent C q e

cadence_sensitive_sites_isolated_or_snoop_aligned :
∀ P loc,
CadenceSensitiveNBPollingSite C P loc →
LocalProtocolSiteHandledByContract C W C.contracts.latencySensitiveLocalProtocols P loc

Lemma L.2:

theorem L2_snooping_establishes_WC
(C : ComparisonInstance)
(W : WitnessBundle C)
(hSnoop : SnoopingWrapperAssumptions C W) :
WC C W

This theorem does not turn failed PushNB/PopNB polls into committed Σ-events. Failed polls remain ε-steps. For ordinary NB-CF sites, the identity/arbitrary witness suffices. For non-NB-CF but non-cadence-sensitive sites, the witness selects the ε-modeled outcomes needed for the matched prefix. For cadence-sensitive retry/timeout/poll-arbiter sites, the theorem may be used only after the local protocol block has been isolated, snoop-aligned, or given an explicit cycle-accurate model, and the comparison is made at that block's chosen observable boundary.

IdentityOrArbitraryNBWitnessBundle C is the witness bundle used in the NB-CF case. Its selector uses identity choices for ε-modeled non-blocking outcomes that are fixed by the matched prefix, and arbitrary choices for non-blocking outcomes that NB-CF proves cannot affect later Σ-visible control flow or NextFront behavior. Its nbMap maps matched executed NB instances to the corresponding same dynamic source-call instance when such a match is relevant, and is arbitrary on failed NB instances that NB-CF proves irrelevant. It is not a discharge for cadence-sensitive retry/timeout/poll-arbiter sites unless those sites have first been shown not to affect later Σ-visible behavior, or have been handled by the latency-sensitive local-protocol contract.

```
def IdentityOrArbitraryNBWitnessBundle
(C : ComparisonInstance) : WitnessBundle C :=
{
selector := IdentityOrArbitraryNBSelector C,
nbMap := IdentityOrArbitraryNBMap C
}
```

Corollary L.3:
The document-level WC predicate is global for the selected comparison instance and now includes the latency-sensitive local-protocol contract. Therefore a theorem returning global WC must not be stated from a single-process NB-CF premise alone.

Use a global theorem when proving WC for the full comparison instance:

```
theorem L3_nb_cf_identity_witness
(C : ComparisonInstance)
(hNBCF : ∀ P, NBCF C P)
(hLocalProtocols :
LatencySensitiveLocalProtocolWC
C
(IdentityOrArbitraryNBWitnessBundle C)
C.contracts.latencySensitiveLocalProtocols) :
WC C (IdentityOrArbitraryNBWitnessBundle C)
```

If a per-process theorem is useful, state it as a local witness fact rather than as global WC:

```
theorem L3_nb_cf_identity_witness_local
(C : ComparisonInstance)
(P : Process)
(hNBCF : NBCF C P) :
LocalWCForProcess C (IdentityOrArbitraryNBWitnessBundle C) P
```

The global theorem is then obtained by combining the local theorem over all processes with the latency-sensitive local-protocol contract:

```
theorem L3_nb_cf_identity_witness_from_local
(C : ComparisonInstance)
(hLocal : ∀ P, LocalWCForProcess C (IdentityOrArbitraryNBWitnessBundle C) P)
(hLocalProtocols :
LatencySensitiveLocalProtocolWC
C
(IdentityOrArbitraryNBWitnessBundle C)
C.contracts.latencySensitiveLocalProtocols) :
WC C (IdentityOrArbitraryNBWitnessBundle C)
```

Corollary L.4 (cadence-sensitive polling handled by isolation). The scheduling-rules document names this fact Corollary L.4; the strategy already enforces its content through the LatencySensitiveLocalProtocolContract and the WC definition, but the Lean development should also expose it under its document name:

```
theorem L4_cadence_sensitive_isolated_or_out_of_scope
(C : ComparisonInstance)
(P : Process)
(loc : SourceLoc)
(hCadence : CadenceSensitiveNBPollingSite C P loc) :
HandledByLatencySensitiveLocalProtocolContractAt
C C.contracts.latencySensitiveLocalProtocols P loc ∨
OutsideOrdinaryAppendixIJSourceCoreAt C P loc
```

That is: if NB-CF fails because the source observes the number or cadence of failed PushNB/PopNB polls — retry counters, timeout counters, or arbitration policies whose later Σ-visible behavior depends on how long a channel remained empty or full — then the site is latency-sensitive at that polling site, and it must be isolated, snoop-aligned, or given an explicit cycle-accurate local protocol model under Appendix D / Appendix L before the ordinary Appendix I/J source-core proof is applied. The left disjunct is exactly the contract obligation cadence_sensitive_nb_sites_handled; the right disjunct records that an unhandled site removes the process from the scope of the ordinary trace-compatibility theorem at the chosen observable boundary. Lemma L.2 may then be used only at the chosen observable boundary of the handled block, never against the raw internal failed-poll cadence.

Appendix L Note 1: Σ_DUT-safety-inert local timing choices.

The preceding L.2/L.3 facts are WC-producing facts for the ordinary proof-internal C.Sigma theorem. Appendix L Note 1 is different: it is a projected-safety principle for the outer DUT/testbench alphabet C.Sigma_DUT. It says that an internal latency-sensitive interface need not be snooped for the Σ_DUT-safety fragment when its ε-modeled timing choices cannot change which Σ_DUT-visible observable units occur, cannot change their projected values, and can only permute projected units that are causally independent at the DUT boundary.

This is not a replacement for WC in the full C.Sigma theorem, and it is not an Appendix K deadlock/liveness exemption. A Σ_DUT-safety-inert interface that is also used in Appendix K reasoning must still satisfy the WFG-inert / Kε obligations, such as A11_Kepsilon_WFGInert.

```
-- A proof-internal observable unit whose projection is visible at the chosen
-- DUT/testbench boundary.
SigmaDUTVisibleUnit :
ComparisonInstance → ObservableUnit C.Sigma → Prop

-- Required projected Appendix J order at the DUT boundary.
-- This is the mandated projected ≺_J relation induced by E1–E5, channel
-- semantics, synchronization semantics, atomic-unit constraints,
-- witness-control dependencies that survive projection, and elaboration /
-- σ_DUT-projection constraints. It is not an incidental order that merely
-- appears in one admissible schedule.
ProjectedRequiredJOrderDUT :
ComparisonInstance →
ObservableUnit C.Sigma →
ObservableUnit C.Sigma →
Prop

-- Two proof-internal observable units are ordered at the DUT boundary exactly
-- when their projected Σ_DUT-visible units are related by the required projected
-- Appendix J order.
def DUTOrdered
(C : ComparisonInstance)
(u v : ObservableUnit C.Sigma) : Prop :=
SigmaDUTVisibleUnit C u ∧
SigmaDUTVisibleUnit C v ∧
ProjectedRequiredJOrderDUT C u v

-- A pair is causally comparable if either order direction is required at the
-- projected DUT boundary.
def DUTCausallyComparable
(C : ComparisonInstance)
(u v : ObservableUnit C.Sigma) : Prop :=
DUTOrdered C u v ∨ DUTOrdered C v u
```

```
-- The choice may change the projected DUT-visible presence or value label of u.
ChoiceMayAddRemoveOrRelabelSigmaDUTUnit :
ComparisonInstance → Process → EpsModeledChoice C P → ObservableUnit C.Sigma →
Prop

-- The choice may change the relative order of two projected DUT-visible units.
ChoiceMayReorderSigmaDUTUnits :
ComparisonInstance → Process → EpsModeledChoice C P →
ObservableUnit C.Sigma → ObservableUnit C.Sigma → Prop

-- A single ε-modeled local timing choice is inert for Σ_DUT safety when it
-- neither changes projected event presence/value nor reorders any projected
-- pair whose order is causally required at the DUT boundary.
def EpsChoiceIsSigmaDUTSafetyInert
(C : ComparisonInstance)
(P : Process)
(choice : EpsModeledChoice C P) : Prop :=
(∀ u,
SigmaDUTVisibleUnit C u →
¬ ChoiceMayAddRemoveOrRelabelSigmaDUTUnit C P choice u) ∧
(∀ u v,
ChoiceMayReorderSigmaDUTUnits C P choice u v →
¬ DUTCausallyComparable C u v)

-- A latency-sensitive interface is Σ_DUT-safety-inert when all of the ε-modeled
-- local timing choices owned by the interface are safety-inert in the sense above.
def SigmaDUTSafetyInertInterface
(C : ComparisonInstance)
(I : LatencySensitiveInterface C) : Prop :=
∀ P choice,
ChoiceOwnedByInterface C I P choice →
EpsChoiceIsSigmaDUTSafetyInert C P choice

-- A choice has a Σ_DUT-safety discharge either by ordinary snooping/witness
-- coverage or by belonging to a certified Σ_DUT-safety-inert interface.
def ChoiceHasSigmaDUTSafetyDischarge
(C : ComparisonInstance)
(W : WitnessBundle C)
(P : Process)
(choice : EpsModeledChoice C P) : Prop :=
ChoiceCoveredBySnoopingOrWitness C W P choice ∨
∃ I,
ChoiceOwnedByInterface C I P choice ∧
SigmaDUTSafetyInertInterface C I
```

```
-- The projected-safety obligation is deliberately weaker than WC. It covers
-- only choices that can matter to the Σ_DUT projected safety theorem.
def SigmaDUTSafetyWitnessObligation
(C : ComparisonInstance)
(W : WitnessBundle C) : Prop :=
∀ P choice,
EpsChoiceCanAffectSigmaDUTProjection C P choice →
ChoiceHasSigmaDUTSafetyDischarge C W P choice

-- Local Appendix L Note 1 corollary: a certified safety-inert interface
-- discharges only its own choices. This theorem is intentionally local; it does
-- not claim that an arbitrary identity witness satisfies the global obligation
-- for every other choice in the design.
theorem LNote1_sigmaDUT_safety_inert_interface
(C : ComparisonInstance)
(W : WitnessBundle C)
(I : LatencySensitiveInterface C)
(hI : SigmaDUTSafetyInertInterface C I) :
∀ P choice,
ChoiceOwnedByInterface C I P choice →
ChoiceHasSigmaDUTSafetyDischarge C W P choice

-- No global Appendix L Note 1 assembly theorem is stated here. The global obligation
 -- SigmaDUTSafetyWitnessObligation C W is intentionally an assumption of
 -- JSigmaDUTSafetyAssumptions. Individual certified inert-interface choices are
-- discharged locally by LNote1_sigmaDUT_safety_inert_interface; all other
-- Σ_DUT-relevant choices must be discharged by the ordinary snooping/witness
-- machinery at the point where the global obligation is established.
```

The Appendix L Note 1 projected-safety facts are safety-side counterparts of the WFG-inert ε / Kε idea, but they live in Appendix L/J rather than Appendix K. They are used only for the Σ_DUT-projected safety theorem below. They do not produce WC C W, do not prove CompatibleSys over the full proof-internal C.Sigma alphabet, and do not discharge A11_Kepsilon_WFGInert for deadlock/liveness proofs.

15. Concrete ObsSigma, Obs-congruence, and deterministic replay

Do not define ObsSigma as an informal least fixed point. Define it as a concrete record of the source-state slices required for replay. The record is P-local: it contains the P-owned state, frontier, pending, attribution, signal-anchor, direct-input, array-mapping, and R2C component-local observation-unit facts needed to determine later P-local Σ-visible behavior and Appendix I–K proof predicates. It does not independently store ChannelEnabled / ChannelDisabled as P-local facts; enablement is obtained from the P-local request/frontier/BoundaryReq facts together with the chosen Sys_ref boundary state and E4. For combinational-process signal IO, ObsSigma records the selected component identity, local R2C occurrence ordinal, observable slice, and fixed-point values for emitted R2C units, not every stutter cycle of the combinational network.

```
structure ObsSigma
(C : ComparisonInstance)
(W : WitnessBundle C)
(P : Process)
(σ : State) where
control_pos : ControlPos P
relevant_vars : RelevantVarSlice C P σ
selected_eps_outcomes : SelectedOutcomeSlice C W.selector P σ
nb_witness_slice : NBWitnessSlice C W.nbMap P σ
msg_attribution_slice : MsgAttributionSlice C W P σ
pending_slice : PendingSlice C P σ
front_slice : FrontSlice C P σ
nextfront_slice : NextFrontSlice C P σ
boundary_req_slice : BoundaryReqSlice C P σ
signal_anchor_slice : SignalAnchorSlice C P σ
array_mapping_slice : ArrayMappingSlice C P σ
direct_input_slice : DirectInputSlice C P σ
r2c_unit_slice : R2CUnitSlice C P σ
```

Here msg_attribution_slice records the Q_exec,P / Q_Ω,P attribution facts needed for E3/E5, Issue/Commit replay, WFG construction, and BoundaryReq ownership. It is broader than nb_witness_slice: nb_witness_slice records μ_Q matching for ε-modeled failed NB calls, while msg_attribution_slice records the source-level operation attribution of pending and frontier message-operation instances generally. The r2c_unit_slice field is the previously missing explicit carrier for the combinational-process facts that the document's Definition I.Obs already requires: for each R2C component owned by P, the selected component identity, the local R2C occurrence ordinal of the most recent emitted unit, the selected observable slice Obs(C_comb), and the fixed-point values of emitted units — not every stutter cycle of the combinational network. The combinational-R2C branch of IObs_congruence_one_observable_unit below propagates this agreement at the granularity of a complete atomic observable unit; without the r2c_unit_slice field that branch would have no carrier to transport, which is why earlier revisions could only assume it.

Cadence-sensitive failed-poll behavior is not represented by adding failed-poll cadence events to ObsSigma or to Σ. Failed PushNB/PopNB polls remain ε-steps with no committed transfer event. If a retry counter, timeout counter, polling arbiter, or other local policy computes externally visible behavior from the number or cadence of failed polls, then that site must already be handled by C.contracts.latencySensitiveLocalProtocols. The ordinary source-core ObsSigma record is then applied at the chosen observable boundary of the isolated/snoop-aligned/explicitly modeled block.

def SigmaRelevantEq C W P σ_1 σ_2 :=
ObsSigma C W P σ_1 = ObsSigma C W P σ_2

Two replay modes must be kept distinct. Plain Theorem J.1 advances through canonical immediate post-unit semantic states: the state immediately after the transition, or explicitly atomic transition group, that realizes the next P-local observable unit, before any optional maximal ε-normalization. Corollary J.1 and Appendix K instead compare explicitly ε-normalized cutpoints. Both modes use complete ObservableUnit objects, never individual labels of an atomic rendezvous, paired synchronization, or R2C fixed-point unit. SameObservableUnitImmediateStep states that u is the next P-local unit and that the step stops at the canonical immediate post-unit state. SameObservableUnitCutpointStep additionally carries the finite ε-normalization to the ε-quiescent cutpoint after u.

def CanonicalImmediatePostUnitState
(C : ComparisonInstance)
(W : WitnessBundle C)
(P : Process)
(ρ : ExecutionPrefix C.Sigma)
(k : Nat) : State :=
ImmediateStateAfterKthPLocalObservableUnit C W P ρ k

theorem IObs_congruence_one_immediate_observable_unit
(C : ComparisonInstance)
(W : WitnessBundle C)
(hB : TraceBRules C)
(hR : RRules C)
(hE : ERules C)
(hIC : IssueCommitWF C W)
(hContracts : ModelingContracts C)
(hWC : WC C W)
(P : Process)

(σ_1 σ_2 σ_1' σ_2' : State)
(u : ObservableUnit C.Sigma)
(hObs : SigmaRelevantEq C W P σ_1 σ_2)
($hStep_1$: SameObservableUnitImmediateStep C W P σ_1 u σ_1')
($hStep_2$: SameObservableUnitImmediateStep C W P σ_2 u σ_2')
(hBoundary : ChosenObservableBoundaryWF C W P hContracts)
(hCadence :
AllCadenceSensitiveNBSitesHandled
C W P hContracts.latencySensitiveLocalProtocols) :
SigmaRelevantEq C W P σ_1' σ_2'

Proof obligations for IObs_congruence_one_immediate_observable_unit should be decomposed by source construct. The theorem consumes only TraceBRules: the next complete observable unit, the fixed stimulus, and W determine the finite transition segment through that unit. It does not invoke maximal ε-closure and does not claim that σ_1' or σ_2' is ε-quiescent.

• deterministic local computation: equal relevant variables and control position determine equal successor ObsSigma fields;
• anchored signal reads/writes: the fixed reactive stimulus and the common next observable unit u determine equal latched Read/Write facts; an ordinary signal event is represented by a singleton unit, while an explicitly atomic signal-observation unit is consumed as a whole;
• combinational R2C units: when u is an R2CFixedPointObservationUnit owned by P, hObs supplies equal prior r2c_unit_slice facts. Together with equal relevant component inputs/state, the fixed stimulus, and R2C_WF, this gives the same unique composed fixed point, canonical-emission decision, component identity, next local occurrence ordinal, selected Obs(C_comb) slice, and fixed-point values. Ordinary R2C stutter cycles emit no unit and are not induction steps;

• blocking Push/Pop and synchronization: the common unit u, together with equal pending/frontier/attribution/BoundaryReq facts, determines equal immediate successor ObsSigma fields; rendezvous and paired synchronization units are consumed atomically rather than replayed as separate labels;
• successful PushNB/PopNB: the singleton committed-transfer unit determines equal data, control, attribution, and frontier updates;
• failed PushNB/PopNB: the failed poll is ε and is not itself an observable-unit induction step. NB-CF or WC fixes every failed outcome that can affect the next unit; cadence-sensitive sites are handled only at the isolated/snoop-aligned/explicit protocol boundary

required by hCadence;
• other ε-only choices before the next unit: WC supplies every outcome capable of affecting that unit or later Appendix I–K proof predicates.

The explicitly ε-quiescent variant is obtained by composing the immediate theorem with the constructive B4/B5 normalization supplied by NormalizationBRules:

theorem IObs_congruence_one_observable_unit
(C : ComparisonInstance)
(W : WitnessBundle C)
(hB : TraceBRules C)
(hNorm : NormalizationBRules C)
(hR : RRules C)
(hE : ERules C)
(hIC : IssueCommitWF C W)
(hContracts : ModelingContracts C)
(hWC : WC C W)
(P : Process)
(σ_1 σ_2 σ_1' σ_2' : State)
(u : ObservableUnit C.Sigma)
(hObs : SigmaRelevantEq C W P σ_1 σ_2)
(hStep$_1$: SameObservableUnitCutpointStep C W P σ_1 u σ_1')
(hStep$_2$: SameObservableUnitCutpointStep C W P σ_2 u σ_2')
(hBoundary : ChosenObservableBoundaryWF C W P hContracts)
(hCadence :
AllCadenceSensitiveNBSitesHandled
C W P hContracts.latencySensitiveLocalProtocols) :
SigmaRelevantEq C W P σ_1' σ_2' :=
by
-- Apply IObs_congruence_one_immediate_observable_unit, then use hNorm to
-- construct the corresponding finite ε-normalizations and prove that the
-- normalized ObsSigma slices remain equal.
sorry

Deterministic replay therefore has two theorem forms:

theorem IDR_immediate_post_unit_deterministic_replay
(C : ComparisonInstance)
(W : WitnessBundle C)

(hB : TraceBRules C)
(hR : RRules C)
(hE : ERules C)
(hIC : IssueCommitWF C W)
(hContracts : ModelingContracts C)
(hWC : WC C W)
(P : Process)
(ρ_1 ρ_2 : ExecutionPrefix C.Sigma)
(hInit : SameInitialSourceState C W P ρ_1 ρ_2)
(hSameUnits : SamePLocalObservableUnitSequence C P ρ_1 ρ_2)
(hBoundary : ChosenObservableBoundaryWF C W P hContracts)
(hCadence :
AllCadenceSensitiveNBSitesHandled
C W P hContracts.latencySensitiveLocalProtocols) :
∀ k,
SigmaRelevantEq C W P
(CanonicalImmediatePostUnitState C W P ρ_1 k)
(CanonicalImmediatePostUnitState C W P ρ_2 k)

-- This is the Lean counterpart of scheduling-rules Lemma I.DR'. It is the
-- replay theorem used by plain J.1 and imports no B4/B5 premise.

theorem IDR_deterministic_replay
(C : ComparisonInstance)
(W : WitnessBundle C)
(hB : TraceBRules C)
(hNorm : NormalizationBRules C)
(hR : RRules C)
(hE : ERules C)
(hIC : IssueCommitWF C W)
(hContracts : ModelingContracts C)
(hWC : WC C W)
(P : Process)
(ρ_1 ρ_2 : ExecutionPrefix C.Sigma)
(hInit : SameInitialSourceState C W P ρ_1 ρ_2)
(hSameUnits : SamePLocalObservableUnitSequence C P ρ_1 ρ_2)
(hBoundary : ChosenObservableBoundaryWF C W P hContracts)
(hCadence :

AllCadenceSensitiveNBSitesHandled
C W P hContracts.latencySensitiveLocalProtocols) :
∀ k,
SigmaRelevantEq C W P
(EpsQuiescentCutpointAfterObservableUnit C W P ρ_1 k)
(EpsQuiescentCutpointAfterObservableUnit C W P ρ_2 k)

Proof architecture. Prove IDR_immediate_post_unit_deterministic_replay by induction on k using IObs_congruence_one_immediate_observable_unit. The base case follows from hInit; the successor case consumes the same complete next P-local observable unit in both executions. Prove IDR_deterministic_replay by applying the immediate theorem at each k and then the paired finite ε-normalizations supplied by hNorm. The proof must not identify a raw immediate terminal state with its ε-normalized successor; it proves equality separately at the two explicitly chosen cutpoint modes.

The same ObsSigma carrier is reused in both modes. Plain J1_fixed_RTL_time_selected_witness_construction and J1_post_to_ref use IDR_immediate_post_unit_deterministic_replay and maintain canonical immediate post-unit replay states. Lemma S1, J1_cutpoint_correspondence, Corollary J.1, and Appendix K use IDR_deterministic_replay only after NormalizationBRules has produced the relevant ε-quiescent cutpoints. Atomic rendezvous, synchronization, and R2C units remain indivisible in both modes. For RTL_B-side request-attribution, BoundaryReq, pending, endpoint-request, and enablement facts, J.AbsConf for the same selected (τ_post, τ_ref, ρ_post, σ_ref, W) triple remains necessary; neither replay theorem derives those implementation-side facts from the P-local observable-unit sequence alone.

16. Appendix I per-process construction

Bundle the assumptions:

structure IConstructionAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
wellformed : C.WellFormed
bRules : TraceBRules C
rRules : RRules C
eRules : ERules C
issueCommitWF : IssueCommitWF C W
witnessUniverseWF : WitnessBundleWF C W

```
implementation : ImplementationAssumptions C
witnessCompatible : WC C W
contracts : ModelingContracts C

-- B4/B5 are supplied only to I1_bounded_epsilon_closure and to explicitly
-- normalized replay/cutpoint theorems. I3, I4, the countably-infinite local
-- branch of I5, and plain J.1 use the trace/admissibility bundle above.

Main theorems:
theorem I1_bounded_epsilon_closure
(C : ComparisonInstance)
(hB4 : C.assumptions.B.B4_finiteEpsilonDepth)
(hB5 : C.assumptions.B.B5_quiescentNormalization) :
...

theorem I2_channel_progress_instance
(C : ComparisonInstance)
(hB3 : C.assumptions.B.B3_channelProgress) :
...

theorem I3_finite_extension_step
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W) :
...

theorem I4_finite_prefix_construction
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W) :
...

def NoFurtherPLocalObservableUnitsAfter
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ∞ : InfiniteExecutionBucketed C.Sigma)
(n₀ : Nat) : Prop :=
```

```
∀ n,
n₀ ≤ n →
PLocalObservableUnitsInBucket C side P (τ∞ n) = ∅

structure FinitePLocalMatch
(C : ComparisonInstance)
(W : WitnessBundle C)
(P : Process)
(ρ_post : InfiniteExecutionBucketed C.Sigma) where
cutoff : Nat
τ_post : Prefix C.RTL_B
τ_ref : Prefix C.Sys_ref
post_prefix_of_execution : PrefixOfInfiniteExecution C .post τ_post ρ_post
captures_all_post_P_units :
NoFurtherPLocalObservableUnitsAfter C .post P ρ_post cutoff
compatible_local_prefix : CompatibleP C P τ_ref τ_post

-- The finite-P-local-event branch stops here at the per-process layer. The
-- absence of later P-local units does not authorize an arbitrary B6 suffix.
theorem I5_finite_local_prefix_construction
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W)
(P : Process)
(ρ_post : InfiniteExecutionBucketed C.Sigma)
(hPost : PostAdmissible C ρ_post)
(hFinite : FinitePLocalObservableUnits C .post P ρ_post) :
FinitePLocalMatch C W P ρ_post

-- This theorem covers only the countably-infinite P-local-unit branch.
theorem I5_countably_infinite_local_limit_construction
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W)
(P : Process)
(ρ_post : InfiniteExecutionBucketed C.Sigma)
(hPost : PostAdmissible C ρ_post)
(hInf : CountablyInfinitePLocalObservableUnits C .post P ρ_post) :
```

∃ ρ_ref_local,
InfinitePLocalReferenceTrace C W P ρ_ref_local ∧
CompatiblePInfinite C P ρ_ref_local
  (ProjectPLocalExecution C .post P ρ_post)

-- Backward-compatible alias for the countably-infinite branch only. The finite
-- branch is completed system-wide by the two theorems in Section 17.
theorem I5_infinite_limit_construction :=
I5_countably_infinite_local_limit_construction

I3 should use IssueAt plus derived IssueTime, not an unconstrained guessed issue cycle. For v42 it must also keep the E4/B2 split explicit: matched channel history and B3 can establish that a pending operation eventually becomes continuously ChannelEnabled; B2 then gives eventual scheduler/arbiter selection, and the selection-commit soundness clause gives the actual Commit in the selected cycle. I3 should not collapse ChannelEnabled into immediate same-cycle Commit.

17. Appendix J system-level correspondence

Define execution scopes:
def FairProgressSatisfying
(C : ComparisonInstance)
(side : Side)
(τ∞ : InfiniteExecutionBucketed C.Sigma) : Prop :=
InfiniteExecutionOfSide C side τ∞ ∧
WeakFairness C side τ∞ ∧
∀ c,
P_push C side τ∞ c ∧
P_pop C side τ∞ c

def PostAdmissible
(C : ComparisonInstance)
(τ∞ : InfiniteExecutionBucketed C.Sigma) : Prop :=
InfinitePostExecution C τ∞ ∧
FairProgressSatisfying C .post τ∞

def RefAdmissible
(C : ComparisonInstance)
(τ∞ : InfiniteExecutionBucketed C.Sigma) : Prop :=

InfiniteRefExecution C τ∞ ∧
FairProgressSatisfying C .ref τ∞

def Exec_post (C : ComparisonInstance) (τ : Prefix C.RTL_B) : Prop :=
∃ τ∞,
Extends τ∞ τ ∧
PostAdmissible C τ∞

def Exec_ref (C : ComparisonInstance) (τ : Prefix C.Sys_ref) : Prop :=
∃ τ∞,
Extends τ∞ τ ∧
RefAdmissible C τ∞

Here FairProgressSatisfying is side-parametric. It includes the B2/B3 fairness/progress assumptions for the selected execution side: weak fairness and the P_push / P_pop channel-progress obligations. PostAdmissible is the post-side instance of this condition together with InfinitePostExecution; RefAdmissible is the reference-side instance together with InfiniteRefExecution. Thus Exec_post and Exec_ref are defined analogously, and any τ_ref produced or consumed by J.1 / K.1 carries the ref-side B2/B3 progress assumptions needed by the drain-closure and deadlock-preservation arguments.

B6 is the globally fixed-point idle-stutter theorem, not an arbitrary deadlock-cutpoint extension principle. It is the general time-divergence / idle-stutter convention only for a reachable finite prefix/state at which the system has reached a B5/global fixed point: after no further Σ-action or internal ε-microstep is enabled under the same stimulus, the global clock may continue by infinite idle-stutter ticks. Each idle-stutter bucket has no sigmaEvents and performs no ε-prefix or ε-suffix work. It changes no non-clock state; the only permitted state change is the clock/tick bookkeeping associated with the idle cycle.

def ClockOnlyIdleStep
(C : ComparisonInstance)
(side : Side)
(σ_0 σ_1 : State) : Prop :=
NonClockStateEq σ_1 σ_0 ∧
σ_1.cycle = σ_0.cycle + 1 ∧
σ_1.epsPos = 0

def IdleStutterBucket
(C : ComparisonInstance)

```
(side : Side)
(σ : State)
(β : CycleBucket C.Sigma) : Prop :=
β.startState = AdvanceClockOnly σ β.cycle ∧
β.sigmaEvents = ∅ ∧
β.epsPrefix = [] ∧
β.epsSuffix = [] ∧
ClockOnlyIdleStep C side β.startState β.endState ∧
β.endQuiescent

def FixedPointState
(C : ComparisonInstance)
(side : Side)
(σ : State) : Prop :=
EpsQuiescent σ ∧
NoObservableActionEnabledAtState C side σ ∧
NoInternalEpsilonStepEnabledAtState C side σ

theorem B6_extend_reachable_prefix_by_idle_stutter
(C : ComparisonInstance)
(side : Side)
(π : Prefix (SystemOfSide C side))
(σ : State)
(hReach : TerminalCutpoint π σ)
(hFP : FixedPointState C side σ) :
∃ τ : InfiniteExecutionBucketed C.Sigma,
ExtendsPrefix C side τ π ∧
∀ n,
n ≥ PrefixLength π →
IdleStutterBucket C side σ (τ n) :=
by
-- General B6 theorem: any reachable finite execution prefix/state at a B5/global
-- fixed point is extendable to an infinite execution by appending idle-stutter
-- ticks under the same stimulus. B6 supplies time divergence only. It does not
-- by itself prove the B2/B3 fairness/progress side conditions required for
-- membership in Exec_post or Exec_ref at an arbitrary partial internal-MP
-- deadlock cutpoint.
sorry
```

Thus any broad prose summary of B6 should be read with this B5/global fixed-point side condition.

The v42 K.1 bridge for a partial internal-MP deadlock is therefore not this B6 idle-stutter theorem. It is the separate fair drain-normalization construction in Section 18: already-issued D-owned non-frontier requests that later become ChannelEnabled may commit as finite drain-only progress; outside-D and environment endpoints continue under the ordinary B2/B3 progress premises; and pure idle-stutter is used only for D-owned internal frontier blockers that remain ChannelDisabled inside the closed WFG component.

Finite-P-local-event completion for Appendix I.5. The per-process finite-prefix theorem is completed only after a global Sys_ref continuation has been selected. Define:

The predicate NoFurtherPLocalObservableUnitsAfter was defined in Section 16.

```
def EmptyPLocalProjectionAfter
(C : ComparisonInstance)
(side : Side)
(P : Process)
(τ∞ : InfiniteExecutionBucketed C.Sigma)
(n₀ : Nat) : Prop :=
NoFurtherPLocalObservableUnitsAfter C side P τ∞ n₀

structure RefContinuationWithoutFurtherPUnits
(C : ComparisonInstance)
(P : Process)
(π_ref : Prefix C.Sys_ref)
(n₀ : Nat) where
τ∞ : InfiniteExecutionBucketed C.Sigma
extends_prefix : Extends τ∞ π_ref
admissible : RefAdmissible C τ∞
no_further_P_units :
NoFurtherPLocalObservableUnitsAfter C .ref P τ∞ n₀

-- Non-fixed-point branch: project the selected admissible global continuation.
-- Other processes and the environment may continue.
```

```
theorem I5_finite_local_projection_from_admissible_global_continuation
(C : ComparisonInstance)
(W : WitnessBundle C)
(P : Process)
(π_ref : Prefix C.Sys_ref)
(n₀ : Nat)
(hCont : RefContinuationWithoutFurtherPUnits C P π_ref n₀) :
∃ τ∞,
Extends τ∞ π_ref ∧
RefAdmissible C τ∞ ∧
EmptyPLocalProjectionAfter C .ref P τ∞ n₀ :=
⟨hCont.τ∞,
 hCont.extends_prefix,
 hCont.admissible,
 hCont.no_further_P_units⟩

-- General fixed-point helper: B6 supplies the idle extension, while the B2/B3
-- obligations are vacuous because no observable action or persistent matching
-- channel request is enabled at or after the fixed point.
theorem fixedpoint_idle_extension_ref_admissible
(C : ComparisonInstance)
(W : WitnessBundle C)
(hB : TraceBRules C)
(π_ref : Prefix C.Sys_ref)
(σ_ref : State)
(τ∞ : InfiniteExecutionBucketed C.Sigma)
(hTerm : TerminalCutpoint π_ref σ_ref)
(hFP : FixedPointState C .ref σ_ref)
(hExt : ExtendsPrefix C .ref τ∞ π_ref)
(hIdle :
∀ n,
n ≥ PrefixLength π_ref →
IdleStutterBucket C .ref σ_ref (τ∞ n)) :
RefAdmissible C τ∞ :=
by
-- InfiniteRefExecution follows from the finite prefix and hIdle. Weak fairness
-- and P_push/P_pop hold vacuously on the suffix because hFP and every hIdle
-- bucket contain no enabled or committed observable action and no newly
```

```
-- asserted endpoint request.
sorry

-- Fixed-point branch: B6 is applied only to the complete global state.
theorem I5_finite_local_projection_from_global_fixedpoint
(C : ComparisonInstance)
(W : WitnessBundle C)
(hB : TraceBRules C)
(P : Process)
(π_ref : Prefix C.Sys_ref)
(σ_ref : State)
(n₀ : Nat)
(hTerm : TerminalCutpoint π_ref σ_ref)
(hFP : FixedPointState C .ref σ_ref)
(hNoLaterPInPrefix : NoLaterPLocalUnitAfterIndex C .ref P π_ref n₀) :
∃ τ∞,
Extends τ∞ π_ref ∧
RefAdmissible C τ∞ ∧
EmptyPLocalProjectionAfter C .ref P τ∞ n₀ :=
by
rcases B6_extend_reachable_prefix_by_idle_stutter
  C .ref π_ref σ_ref hTerm hFP with ⟨τ∞, hExt, hIdle⟩
have hAdm : RefAdmissible C τ∞ :=
  fixedpoint_idle_extension_ref_admissible
    C W hB π_ref σ_ref τ∞ hTerm hFP hExt hIdle
exact ⟨τ∞, hExt, hAdm,
  no_further_P_units_in_fixedpoint_idle_extension
    C W P π_ref σ_ref τ∞ n₀ hNoLaterPInPrefix hFP hExt hIdle⟩
```

The Section 17 J.1 construction must choose one of these branches for every process with finitely many local observable units. It may not infer a global idle extension merely from the fact that P itself has stopped emitting units.

```
def BeforeNextResetAffectingProcess
(C : ComparisonInstance)
(side : Side)
(P : Process)
(epoch : ResetEpochId)
```

```
(τ : InfiniteExecutionBucketed C.Sigma)
(n : Cycle) : Prop :=
NoResetAffectingProcessSinceEpochStart C side P epoch τ n

theorem finish_process_idle_padding_until_next_reset
(C : ComparisonInstance)
(side : Side)
(P : Process)
(epoch : ResetEpochId)
(π : Prefix (SystemOfSide C side))
(σ : State)
(hReach : TerminalCutpoint π σ)
(hEpoch : CurrentResetEpoch C side P σ = some epoch)
(hFin : ProcessFinishedInEpoch C side P epoch σ)
(hNoMoreP : NoFurtherProcessStepEnabledInEpoch C side P epoch σ) :
∃ τ : InfiniteExecutionBucketed C.Sigma,
ExtendsPrefix C side τ π ∧
∀ n,
n ≥ PrefixLength π →
BeforeNextResetAffectingProcess C side P epoch τ n →
ProcessIdleStuttered C side P (τ n) :=
by
-- The FINISH_P occurrence remains a realized reset-epoch-indexed SyncAnchor
-- for R2/E2 signal anchoring. Until the next reset affecting P, the process is
-- padded by process-local idle stutter. This padding creates no new process-owned
-- Σ-events, no new BoundaryReq, no new Front/NextFront item, and no new WFG edge
-- for P. A later reset ends this conclusion, creates a new reset epoch, and may
-- be followed by a new START_P occurrence.
sorry
```

The FINISH_P corollary is process-local and reset-epoch-bounded: a process that has finished in epoch e is padded by ε/idle-stutter while the global system may continue executing other processes, but only until the next reset that affects P. This padding must not be counted as additional B4/B5 internal progress by P, because it performs no enabled internal ε-step and emits no Σ-event. After a later reset, the earlier FINISH_P occurrence has no anchoring, ordering, or WFG force in the new epoch.

Directed compatibility:
Do not define CompatibleP or CompatibleSys as equality of bucketed executions.

First define the observable units extracted from a bucketed execution. An observable unit may be: a single ordinary Σ-event; an explicitly atomic multi-endpoint unit, such as a rendezvous Push/Pop pair or a SyncChannel/ac_sync pair; or a component-local R2C fixed-point signal-observation unit emitted under the v42 stutter-canonical R2C convention. Independent same-cycle events are not ordered merely because they share a CycleBucket; any strict required order among observable units is imposed only by RequiredJOrder / ≺_J. The broader causal-dependency graph → may still be recorded for explanatory information-flow reasoning, but it is not the induction order.

```
structure ObservableUnit (Σ : Type) where
events : Finset Σ
atomic : Prop

-- ObservableUnitWF is the well-formedness predicate for the unit extraction used
-- by Appendix J. It is where non-splitting and grouping obligations are checked:
-- rendezvous Push/Pop pairs, paired SyncChannel/ac_sync endpoints, R2C
-- fixed-point observations, and any other explicitly atomic same-cycle
-- transaction must appear as one ObservableUnit rather than as strict internal
-- RequiredJOrder edges. It also records that every Σ-event in the finite prefix
-- belongs to exactly one extracted unit and that independent same-cycle events
-- are not ordered merely by bucket membership.
structure ObservableUnitWF
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side)) where
covers_prefix_events :
∀ e,
SigmaEventInPrefix C side τ e →
∃ u,
u ∈ ObservableUnitsOf C side τ ∧
e ∈ u.events
events_have_unique_unit :
∀ e u v,
u ∈ ObservableUnitsOf C side τ →
v ∈ ObservableUnitsOf C side τ →
e ∈ u.events →
e ∈ v.events →
u = v
```

atomic_rendezvous_grouped :
∀ u,
AtomicRendezvousUnit C side τ u →
u ∈ ObservableUnitsOf C side τ ∧ u.atomic
atomic_sync_grouped :
∀ u,
AtomicSyncUnit C side τ u →
u ∈ ObservableUnitsOf C side τ ∧ u.atomic
r2c_fixed_point_unit_grouped :
∀ u,
R2CFixedPointObservationUnit C side τ u →
u ∈ ObservableUnitsOf C side τ ∧ u.atomic
atomic_units_not_split :
∀ u e_1 e_2,
u ∈ ObservableUnitsOf C side τ →
u.atomic →
e_1 ∈ u.events →
e_2 ∈ u.events →
SameObservableUnit C side τ
(ObservableUnit.singleton e_1)
(ObservableUnit.singleton e_2)
independent_same_cycle_not_ordered_by_bucket :
∀ u v,
u ∈ ObservableUnitsOf C side τ →
v ∈ ObservableUnitsOf C side τ →
u ≠ v →
SameCycleOnly C side τ u v →
¬ RequiredJOrderGenerator C side τ u v ∧
¬ RequiredJOrderGenerator C side τ v u

ObservableUnitsOf extracts Σ-visible events from the finite bucketed prefix and groups them into comparison units as follows:

- an ordinary committed message, signal, or synchronization event may form a singleton observable unit;
- explicitly atomic multi-endpoint events, such as rendezvous Push/Pop pairs and SyncChannel/ac_sync handshakes, form one atomic observable unit and may not be split by CompatibleP or CompatibleSys;
- R2C combinational observations are emitted as component-local fixed-point signal-observation units according to R2C_WF and are matched by local occurrence ordinal;
- independent same-cycle events are unordered merely by virtue of being in the same CycleBucket;
- same-cycle events that are merely coincident in one implementation are not forced to remain same-cycle in the other implementation unless they are

explicitly atomic or otherwise ordered by RequiredJOrder.

Because τ is a finite prefix, ObservableUnitsOf returns a Finset. If a Set view is needed elsewhere, it should be obtained by coercion from this Finset, rather than by introducing a second observable-unit universe.

```
def ObservableUnitsOf
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side)) :
Finset (ObservableUnit (C.Sigma)) :=
ExtractSigmaUnitsFinset
C side τ
(AtomicRendezvousUnits C side τ)
(AtomicSyncUnits C side τ)
(R2CFixedPointObservationUnits C side τ)
(IndependentSameCycleUnits C side τ)
```

```
def ObservableUnitsOfSet
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side)) :
Set (ObservableUnit (C.Sigma)) :=
fun u => u ∈ ObservableUnitsOf C side τ
```

CausalDependency is an information-flow graph. It is useful for explaining that a Pop observes the corresponding Push value, that signal handshakes communicate information, or that a witness-selected ε choice affects later visible behavior. It is not itself the strict partial order used by Theorem J.1, and the Lean development should not assume the unquotiented CausalDependency graph is acyclic. CausalDependency is not part of the core mechanized theorem spine for Theorem J.1, Corollary J.1, Appendix K, or Appendix R. The core proof order is RequiredJOrder / ≺_J over observable units, together with ObservableUnitWF, E1–E5 legality/alignment, witness compatibility, cutpoint correspondence, and the Appendix K WFG/deadlock machinery. If CausalDependency is formalized in Lean, it should be treated as an optional documentation/design-well-formedness layer used to mirror the scheduling-rules document's explanatory causal-dependency relation, not as a dependency of the main trace-compatibility or liveness-preservation theorems.

```
def CausalDependency
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (C.Sigma)) : Prop :=
Relation.TransGen
```

```
(CausalDependencyGenerator C side τ)
u v

-- Intra-process causal trace/source order is part of CausalDependency, because it
-- records source-level information flow and design intent inside one dynamic
-- process execution. It is intentionally non-temporal: by itself it does not
-- assert strict commit-cycle order, and it is not a RequiredJOrder edge unless
-- some explicit required-order generator below also applies.
def IntraProcessCausalTraceOrder
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (C.Sigma)) : Prop :=
SameOwningProcess C side τ u v ∧
SourceTracePrecedes C side τ u v ∧
CausallyRelevantSourceFlow C side τ u v

-- Declared shared-storage / memory-serialization dependence is also part of
-- CausalDependency. It records only dependencies supplied by the array-access
-- mapping layer, explicit synchronization, atomic memory semantics, or declared
-- serialization/coherence rules for mapped shared storage. It does not turn a
-- memory information-flow edge into a proof-order edge unless the selected
-- mapping contract also generates an explicit RequiredJOrder edge or atomic unit.
def DeclaredMemorySerializationDependency
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (C.Sigma)) : Prop :=
SharedStorageConflict C side τ u v ∧
(ArrayMappingSerializes C side τ u v ∨
 ExplicitSyncOrdersMemoryAccess C side τ u v ∨
 DeclaredAtomicMemorySemanticsOrder C side τ u v ∨
 DeclaredCoherenceSerializationOrder C side τ u v)

def CausalDependencyGenerator
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (C.Sigma)) : Prop :=
MessageInformationFlow C side τ u v ∨
SignalInformationFlow C side τ u v ∨
SynchronizationInformationFlow C side τ u v ∨
IntraProcessCausalTraceOrder C side τ u v ∨
DeclaredMemorySerializationDependency C side τ u v ∨
```

WitnessControlDependency C side τ u v ∨
ElaborationInformationFlow C side τ u v

RequiredJOrder is the mandated Appendix J order over observable units. It is narrower than CausalDependency. It contains only strict order constraints that CompatibleP / CompatibleSys must preserve as order. It does not contain mere information-flow dependencies, grouping requirements, or source-lexical order that v42 intentionally treats as non-ordering.

The generator must encode the v42 exclusions explicitly:

- E2 signal anchoring contributes equality/alignment of the corresponding anchor cycle and value label; it never contributes an independent strict RequiredJOrder edge.
- E3 same-interface message operations may impose strict same-interface order.

Strict issue/commit attribution is used by the fixed RTL-time witness construction schedule, not as an independent RequiredJOrder / ≺_J generator. In particular, issue-attribution facts may help choose the next replay unit, but they do not add observable order among units that v42 otherwise leaves unordered.

- E3 distinct-interface lexical source order is only a no-reverse legality constraint. It is not a positive strict edge between the distinct-interface commits. In particular, two distinct-interface operations that v42 permits to issue/commit in parallel do not create u ≺_J v merely because one appears earlier in source text.
- Atomic rendezvous, SyncChannel/ac_sync handshakes, and R2C fixed-point observations are represented as single ObservableUnit objects. Their non-splitting constraints are checked by ObservableUnitWF and CompatibleSys; they do not add strict internal edges.
- Witness-control dependencies remain in CausalDependency and WC, where they justify selected ε outcomes. They are not RequiredJOrder generators.
- Elaboration legality is represented by the explicit Sys_B^elab channels, E4 channel-realization semantics, and Π_elab / sigmaDUTProjection projection constraints. It is not a separate RequiredJOrder generator unless it is realized as one of the explicit strict channel or synchronization edges below.

```
def E2SignalAnchorAlignmentConstraint
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (C.Sigma)) : Prop :=
SignalAnchorCyclesAndLabelsAligned C side τ u v

def E3MessageStrictRequiredOrder
(C : ComparisonInstance)
```

```
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (C.Sigma)) : Prop :=
E3SameInterfaceMessageOrder C side τ u v

def E3DistinctInterfaceNoReverseLegality
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side)) : Prop :=
∀ u v,
DistinctMessageInterfaces C side τ u v →
SourceLexicallyBefore C side τ u v →
¬ RequiredReverseCommitOrder C side τ v u

-- E5 has two logically different parts.
--
-- The pref_S(s) side is non-strict legality:
--   q ∈ pref_S(s) gives issue(q) ≤ clk(s), and for committed blocking q
--   the commit cycle of q is ≤ clk(s), with equality allowed.
-- It is therefore not a strict RequiredJOrder generator.
--
-- The suff_S(s) side is the strict post-boundary side:
--   q ∈ suff_S(s) gives issue(q) > clk(s), and commit cannot precede issue.
-- Thus the synchronization unit for s is strictly before the observable
-- message unit for q. This is the only E5 contribution to RequiredJOrder.
def E5PrefSyncBoundaryLegality
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side)) : Prop :=
∀ s q uSync uMsg,
SyncUnitOf C side τ s uSync →
MessageUnitOf C side τ q uMsg →
q ∈ prefS s →
IssuedInPrefix C side τ q →
IssueTimeOf C side τ q ≤ CommitCycleOfUnit C side τ uSync ∧
(Blocking q →
  CommittedMessageUnit C side τ q uMsg →
  CommitCycleOfUnit C side τ uMsg ≤ CommitCycleOfUnit C side τ uSync)

def E5StrictSuffSyncBoundaryRequiredOrder
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (C.Sigma)) : Prop :=
```

```
∃ s q,
SyncUnitOf C side τ s u ∧
MessageUnitOf C side τ q v ∧
q ∈ suffS s ∧
IssuedInPrefix C side τ q ∧
CommitCycleOfUnit C side τ u < CommitCycleOfUnit C side τ v

-- Keep the old name as the RequiredJOrder generator name, but define it to be
-- exactly the strict successor-side E5 relation.
def E5SyncBoundaryRequiredOrder
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (C.Sigma)) : Prop :=
E5StrictSuffSyncBoundaryRequiredOrder C side τ u v

-- The actual generator is intentionally the narrow v42 generator below.
def RequiredJOrderGenerator
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (C.Sigma)) : Prop :=
E1SyncOrder C side τ u v ∨
E3MessageStrictRequiredOrder C side τ u v ∨
E4ChannelRequiredOrder C side τ u v ∨
E5SyncBoundaryRequiredOrder C side τ u v ∨
SynchronizationUnitRequiredOrder C side τ u v


structure RequiredJOrderWF
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side)) where
-- This is stated as a generator-level exclusion for proof hygiene. A more
-- minimal formalization could prove the same fact as a consequence of the
-- concrete generator definitions, but keeping it in RequiredJOrderWF is
-- harmless and matches the v42 intent that E2 contributes alignment only, not
-- strict observable order.
e2_alignment_only :
∀ u v,
E2SignalAnchorAlignmentConstraint C side τ u v →
¬ RequiredJOrderGenerator C side τ u v ∧
¬ RequiredJOrderGenerator C side τ v u
distinct_interface_no_positive_source_edge :
```

```
∀ u v,
DistinctMessageInterfaces C side τ u v →
SourceLexicallyBefore C side τ u v →
¬ RequiredJOrderGenerator C side τ u v
atomicity_is_grouping_not_order :
∀ u v,
AtomicUnitNonSplittingConstraint C side τ u v →
SameObservableUnit C side τ u v
witness_control_not_order :
∀ u v,
WitnessControlDependency C side τ u v →
¬ RequiredJOrderGenerator C side τ u v
elaboration_not_separate_order :
∀ u v,
ElaborationInformationFlow C side τ u v →
(RequiredJOrderGenerator C side τ u v →
 E1SyncOrder C side τ u v ∨
 E3MessageStrictRequiredOrder C side τ u v ∨
 E4ChannelRequiredOrder C side τ u v ∨
 E5SyncBoundaryRequiredOrder C side τ u v ∨
 SynchronizationUnitRequiredOrder C side τ u v)
```

RequiredJOrderWF is not intended to be a new external tool-flow assumption. It is a definitional sanity theorem about the concrete RequiredJOrderGenerator disjuncts once E1–E5, issue/commit well-formedness, observable-unit grouping, and elaboration well-formedness have been instantiated. The Lean development may keep it as a named structure for proof modularity, but J.1 should obtain it from the following lemma rather than expecting it to appear magically in JTraceAssumptions.

```
theorem RequiredJOrderWF_from_concrete_generators
(C : ComparisonInstance)
(W : WitnessBundle C)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(hE : ERules C)
(hIC : IssueCommitWF C W)
(hOU : ObservableUnitWF C side τ)
(hImpl : ImplementationAssumptions C)
(hElab : ElaborationProjectionWF C side τ) :
RequiredJOrderWF C side τ :=
by
-- Unfold RequiredJOrderGenerator and discharge each WF field by cases on the
-- concrete v42 generator disjunct:
```

```
-- • E2 contributes alignment only;
-- • distinct-interface E3 lexical order is no-reverse legality only;
-- • the pref_S(s) side of E5 is non-strict issue/commit legality only and
--   contributes no RequiredJOrder edge;
-- • the suff_S(s) side of E5 is the only E5 strict RequiredJOrder generator;
-- • atomicity is already enforced by ObservableUnitWF grouping;
-- • witness-control dependencies remain in WC and CausalDependency only; and
-- • elaboration information flow contributes order only when realized through
--   an explicit E1/E3-same-interface/E4-strict-channel/E5-suff/synchronization
--   generator.
sorry

def RequiredJOrder
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (C.Sigma)) : Prop :=
Relation.TransGen
(RequiredJOrderGenerator C side τ)
u v

theorem RequiredJOrder_acyclic_on_observable_units
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(hWF : RequiredJOrderWF C side τ) :
Acyclic (RequiredJOrder C side τ) :=
by
-- Strict RequiredJOrder edges are exactly the v42 strict observable-order
-- edges above. Each edge is justified by an explicit synchronization order,
-- same-interface message order, FIFO/channel order, rendezvous/channel order,
-- the strict successor side of synchronization-boundary separation, or paired
-- synchronization/barrier before-after constraints. Atomic multi-endpoint
-- transactions are single observable units, E2 signal anchors are alignment
-- constraints, distinct-interface lexical order is no-reverse legality rather
-- than a positive edge, and witness-control / elaboration information-flow
-- dependencies are not in this generator.
--
-- Issue/commit attribution is intentionally not part of RequiredJOrder. It is
-- used by RTLTimeNextObservableUnit and the fixed RTL-time ideal-chain
-- construction schedule to choose a realizable next unit, but that construction
-- schedule adds no observable order.
--
-- The acyclicity proof is the v42 Theorem J.1 argument instantiated per fixed
```

```
-- legal execution prefix, but with the E5 split made explicit. Assign each
-- observable unit γ its commit cycle clk(γ) in that prefix, with explicitly
-- atomic same-cycle units carrying the single common commit cycle of their
-- endpoints. Every strict generator edge γ_1 ≺_J γ_2 implies clk(γ_1) < clk(γ_2):
--
-- • E1 synchronization order is strict between successive synchronization units.
-- • E3 same-interface message order is strict for the same-interface required
--   order generator.
-- • E4 channel-order generators are strict for reset-empty, bounded-occupancy,
--   non-fall-through, and explicit rendezvous/channel ordering constraints.
-- • E5 contributes a strict edge only from the synchronization unit s to a
--   committed message unit for q ∈ suff_S(s). This edge is strict because
--   issue(q) > clk(s) and commit(q) cannot occur before issue(q).
-- • Paired synchronization/barrier before-after constraints are strict by their
--   own generator definitions.
--
-- The pref_S(s) half of E5 is deliberately absent from RequiredJOrder. For
-- q ∈ pref_S(s), E5 gives issue(q) ≤ clk(s), and for a committed blocking q
-- the commit cycle of q is ≤ clk(s), with equality allowed. Those facts are
-- E5 legality / boundary well-formedness facts used by issue/commit,
-- frontier, cutpoint, and compatibility proofs; they are not strict ≺_J edges
-- and are not used in the acyclicity measure.
--
-- Hence any hypothetical ≺_J cycle would force
-- clk(γ_0) < ... < clk(γ_0), which is impossible. The measure for the actual
-- strict RequiredJOrder graph is commit cycle alone. Issue timestamps are used
-- only to prove that the strict suff_S(s) E5 generator is indeed commit-cycle
-- increasing; they are not a second component of the acyclicity measure.
-- E2 alignment, atomic grouping, distinct-interface no-reverse legality,
-- pref_S(s) E5 legality, witness-control dependencies, and elaboration
-- information flow contribute no strict edges, per RequiredJOrderWF.
sorry
```

CompatibleP : ComparisonInstance → Process → Prefix C.Sys_ref → Prefix C.RTL_B → Prop

CompatibleP C P τ_ref τ_post means:
the P-local observable units of τ_ref and τ_post are related by a value-label-preserving bijection;
explicitly atomic units are mapped to explicitly atomic units and are not split;
the mapping preserves the P-local RequiredJOrder constraints induced by E1, E3 same-interface message order, E4, the strict suff_S side of E5, and the explicit synchronization constraints;
it preserves E2 signal-anchor equality/alignment constraints, but does not treat

E2 as an independent source of strict order;
it preserves the pref_S side of E5 as issue/commit legality and synchronization-boundary well-formedness, not as a strict RequiredJOrder edge;
it treats distinct-interface source-order no-reverse as legality, not as a strict edge; and
it does not require independent P-local units to occupy the same cycle bucket in both executions.

Issue/commit attribution and selected witness/request-attribution facts are preserved through RTLTimeNextObservableUnit, J.Abs, and J.AbsConf obligations; they are not additional RequiredJOrder edges.

CompatibleSys : ComparisonInstance → Prefix C.Sys_ref → Prefix C.RTL_B → Prop

CompatibleSys C τ_ref τ_post means:
∀ P, CompatibleP C P τ_ref τ_post;
all explicit abstract channels in the chosen Sys_ref / RTL_B comparison satisfy the applicable E4 channel semantics at the agreed abstract boundaries;
the system-level observable-unit mapping preserves the mandated global RequiredJOrder / ≺_J constraints and value labels;
explicitly atomic multi-endpoint units remain atomic on both sides;
E2 anchor alignment, distinct-interface no-reverse legality, and elaboration projection legality are preserved by their own well-formedness/projection obligations rather than by extra strict ≺_J edges;
no equality of CycleBucket lists, no equality of bucket cycle numbers, and no bucket-by-bucket equality of sigmaEvents is required.

Thus the bijection used by CompatibleSys is over ObservableUnitsOf, and its ordering obligation is preservation of the narrowed v42 RequiredJOrder. Independent observable units may be placed in different cycle buckets or grouped differently in Sys_ref and RTL_B, provided their Σ labels, values, atomicity requirements, FIFO/signal/synchronization obligations, no-reverse legality, and required ≺_J constraints are preserved.

The notation τ_ref,system ≈_sys τ_post,system is an abbreviation for CompatibleSys C τ_ref τ_post. It is a directed compatibility predicate, not a symmetric equivalence relation. The main construction direction is Post → Ref; the reverse direction is restricted to RTL-consistent source prefixes.

J.0:

```
theorem J0_pairwise_composition
  (C : ComparisonInstance)
  (W : WitnessBundle C)
  (hC : C.WellFormed)
  (hB : BRules C)
  (hR : RRules C)
  (hE : ERules C)
  (hIC : IssueCommitWF C W)
  (hContracts : ModelingContracts C)
  (τ_ref : Prefix C.Sys_ref)
  (τ_post : Prefix C.RTL_B)
  (hProc : ∀ P, CompatibleP C P τ_ref τ_post)
  (hChan : ChannelsWellFormedAndMatched C τ_ref τ_post) :
  CompatibleSys C τ_ref τ_post

-- Projected process compatibility for the outer DUT/testbench-visible alphabet.
-- This is weaker than CompatibleP over C.Sigma: it compares only the
-- C.sigmaDUTProjection image of the process-owned observable units, preserving
-- projected values, atomicity that remains visible at Σ_DUT, and required
-- projected RequiredJOrder / ≺_J constraints. It may ignore proof-internal units hidden

-- by the projection and may commute projected units that are not causally
-- comparable at the DUT boundary.
def CompatiblePDUT
(C : ComparisonInstance)
(P : Process)
(τ_ref : Prefix C.Sys_ref)
(τ_post : Prefix C.RTL_B) : Prop :=
CompatiblePOnProjection C P C.Sigma_DUT C.sigmaDUTProjection τ_ref τ_post

-- Projected system compatibility for the outer DUT/testbench-visible alphabet.
-- This is the formal target of the Appendix L Note 1 safety branch.
def CompatibleSysDUT
(C : ComparisonInstance)
(τ_ref : Prefix C.Sys_ref)
(τ_post : Prefix C.RTL_B) : Prop :=
CompatibleSysOnProjection C C.Sigma_DUT C.sigmaDUTProjection τ_ref τ_post

-- Projected pairwise composition. This is the Σ_DUT analogue of J.0 and is
-- intentionally weaker than J0_pairwise_composition: it requires projected
-- process compatibility, projected channel/accounting consistency, and projected
-- required-order preservation only at C.Sigma_DUT.
```

```
theorem J0_pairwise_composition_DUT
  (C : ComparisonInstance)
  (W : WitnessBundle C)
  (hC : C.WellFormed)
  (hB : BRules C)
  (hR : RRules C)
  (hE : ERules C)
  (hIC : IssueCommitWF C W)
  (hContracts : ModelingContracts C)
  (τ_ref : Prefix C.Sys_ref)
  (τ_post : Prefix C.RTL_B)
  (hProc : ∀ P, CompatiblePDUT C P τ_ref τ_post)
  (hChan : ChannelsWellFormedAndMatchedOnProjection
        C C.Sigma_DUT C.sigmaDUTProjection τ_ref τ_post) :
  CompatibleSysDUT C τ_ref τ_post
```

J.1:

```
-- The plain J.1 trace theorem uses TraceBRules and therefore does not import
-- B4/B5 merely by taking the full BRules bundle. NormalizationBRules enters
-- only through JCutpointAssumptions and Appendix K.

structure JTraceAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
wellformed : C.WellFormed
bRules : TraceBRules C
rRules : RRules C
eRules : ERules C
issueCommitWF : IssueCommitWF C W
implementation : ImplementationAssumptions C
witnessCompatible : WC C W
witnessUniverseWF : WitnessBundleWF C W
contracts : ModelingContracts C
environment : ReactiveEnvironmentWF C.env
-- Elaboration-projection well-formedness consumed by
-- RequiredJOrderWF_from_concrete_generators and by the
-- J1_fixed_RTL_time_selected_witness_construction proof.
-- Trivial (identity projection) for ordinary Sys_B; supplied by the
-- ElaborationPackage piSigma_*_wf projection obligations in Sys_B^elab cases.
elaborationProjectionWF :
∀ side τ, ElaborationProjectionWF C side τ

-- Assumption bundle for the projected Σ_DUT safety theorem. This intentionally
-- omits witnessCompatible : WC C W and replaces it with the weaker
```

```
-- SigmaDUTSafetyWitnessObligation C W. Like JTraceAssumptions, it omits
-- B4/B5. It is not suitable for cutpoint correspondence or Appendix K unless
-- those theorems separately import full WC and NormalizationBRules.
structure JSigmaDUTSafetyAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
wellformed : C.WellFormed
bRules : TraceBRules C
rRules : RRules C
eRules : ERules C
issueCommitWF : IssueCommitWF C W
implementation : ImplementationAssumptions C
sigmaDUTSafetyWitness : SigmaDUTSafetyWitnessObligation C W
witnessUniverseWF : WitnessBundleWF C W
contracts : ModelingContracts C
environment : ReactiveEnvironmentWF C.env

-- RTLTimeNextObservableUnit carries the fixed v42 enumeration discipline. It is
-- derived from the selected RTL_B witness prefix after grouping explicitly
-- atomic same-cycle transactions into single ObservableUnit objects. It refines
-- RTL_B commit-cycle order and, within one commit cycle, uses only the selected
-- witness/replay attribution needed to choose a realizable next unit. This
-- predicate is a construction-schedule predicate, not a RequiredJOrder
-- generator, and it adds no observable order among units that ≺_J leaves
-- unordered.
def RTLTimeNextObservableUnit
(C : ComparisonInstance)
(W : WitnessBundle C)
(τ_post : Prefix C.RTL_B)
(H : Finset (ObservableUnit C.Sigma))
(γ : ObservableUnit C.Sigma) : Prop :=
γ ∈ ObservableUnitsOf C .post τ_post ∧
γ ∉ H ∧
AllRequiredJPredecessorsIn C .post τ_post H γ ∧
RefinesSelectedRTLCommitCycleOrder C W τ_post H γ ∧
WithinCycleWitnessReplayAttributionCompatible C W τ_post H γ ∧
AtomicUnitAlreadyGrouped C .post τ_post γ

structure RTLTimeIdealChain
(C : ComparisonInstance)
(W : WitnessBundle C)
(τ_post : Prefix C.RTL_B) where
N : Nat
idealAt : Nat → Finset (ObservableUnit C.Sigma)
```

```
initial_empty :
idealAt 0 = ∅
terminal_units :
idealAt N = ObservableUnitsOf C .post τ_post
constant_after_terminal :
∀ n,
N ≤ n →
idealAt n = idealAt N
single_unit_step :
∀ n,
n < N →
∃ γ,
γ ∉ idealAt n ∧
RTLTimeNextObservableUnit C W τ_post (idealAt n) γ ∧
idealAt (n+1) = idealAt n ∪ {γ}
rtl_time_prefix_closed :
∀ n u v,
n ≤ N →
v ∈ idealAt n →
RequiredJOrder C .post τ_post u v →
u ∈ idealAt n

def NextJUnitRealizable
(C : ComparisonInstance)
(W : WitnessBundle C)
(τ_post : Prefix C.RTL_B)
(τ_ref_prefix : Prefix C.Sys_ref)
(u : ObservableUnit C.Sigma) : Prop :=
SourceOperationFrontierReached C W τ_ref_prefix u ∨
AlreadyIssuedPendingInReplayState C W τ_ref_prefix u

-- SelectedJWitness is produced by the Appendix J.1 construction. It is not an
-- unconstrained hypothesis and it is not inferred later from CompatibleSys.
-- The evidence records the fixed RTL-time ideal chain and the replay proof that
-- builds the particular Sys_ref prefix selected for τ_post and W.
structure SelectedJWitnessEvidence
(C : ComparisonInstance)
(W : WitnessBundle C)
(τ_post : Prefix C.RTL_B)
(τ_ref : Prefix C.Sys_ref) where
chain : RTLTimeIdealChain C W τ_post
terminal_replay :
ReplayPrefixForIdeal C W τ_post τ_ref (chain.idealAt chain.N)
```

```
terminal_immediate_replay :
ImmediateReplayStateForIdeal
C W τ_post τ_ref (chain.idealAt chain.N)
step_realizability :
∀ n,
n < chain.N →
∃ γ τ_ref_prefix,
γ ∉ chain.idealAt n ∧
RTLTimeNextObservableUnit C W τ_post (chain.idealAt n) γ ∧
chain.idealAt (n+1) = chain.idealAt n ∪ {γ} ∧
ReplayPrefixForIdeal C W τ_post τ_ref_prefix (chain.idealAt n) ∧
NextJUnitRealizable C W τ_post τ_ref_prefix γ ∧
ImmediateReplayStateForIdeal
C W τ_post τ_ref_prefix (chain.idealAt n)
exec_ref : Exec_ref C τ_ref
compatible : CompatibleSys C τ_ref τ_post

def SelectedJWitness
(C : ComparisonInstance)
(W : WitnessBundle C)
(τ_post : Prefix C.RTL_B)
(τ_ref : Prefix C.Sys_ref) : Prop :=
Nonempty (SelectedJWitnessEvidence C W τ_post τ_ref)
```

The J.1 induction must follow the fixed RTL-time ideal chain and must maintain the canonical immediate post-unit replay state for every chain prefix. It must not extend an arbitrary ≺_J-minimal observable unit, because v42’s proof relies on the fact that the next RTL-time unit is realizable in the replay state: its source operation is either frontier-reached or already issued and pending. Each chain step adds exactly one ObservableUnit γ. After realizing γ, invoke IDR_immediate_post_unit_deterministic_replay to establish ImmediateReplayStateForIdeal for the extended prefix. If γ is an explicitly atomic rendezvous, SyncChannel/ac_sync, or R2C fixed-point transaction, it has already been grouped as one ObservableUnit before the chain step is taken. No maximal ε-normalization is performed in this construction, and no NormalizationBRules premise is imported. Acyclicity of RequiredJOrder is still needed to show that each RTL-time prefix is an ideal, but acyclicity alone is not the construction schedule.

```
theorem J1_fixed_RTL_time_selected_witness_construction
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JTraceAssumptions C W)
(τ_post : Prefix C.RTL_B)
(hExec : Exec_post C τ_post) :
∃ chain : RTLTimeIdealChain C W τ_post,
∀ n,
n < chain.N →
∃ γ τ_ref_prefix,
γ ∉ chain.idealAt n ∧
RTLTimeNextObservableUnit C W τ_post (chain.idealAt n) γ ∧
chain.idealAt (n+1) = chain.idealAt n ∪ {γ} ∧
ReplayPrefixForIdeal C W τ_post τ_ref_prefix (chain.idealAt n) ∧
NextJUnitRealizable C W τ_post τ_ref_prefix γ ∧
ImmediateReplayStateForIdeal
C W τ_post τ_ref_prefix (chain.idealAt n) :=
by
-- First obtain observable-unit well-formedness for the finite post prefix.
have hOU_post :
  ObservableUnitWF C .post τ_post :=
  ObservableUnitWF_of_exec_post C W hJ τ_post hExec

-- RequiredJOrderWF is derived from the concrete v42 generator definitions; it
-- is not an additional JTraceAssumptions field.
have hWF_post :
  RequiredJOrderWF C .post τ_post :=
  RequiredJOrderWF_from_concrete_generators
    C W .post τ_post
    hJ.eRules
    hJ.issueCommitWF
    hOU_post
    hJ.implementation
    hJ.elaborationProjectionWF

have hAcyc_post :
  Acyclic (RequiredJOrder C .post τ_post) :=
  RequiredJOrder_acyclic_on_observable_units
    C .post τ_post hWF_post

-- Construct the fixed RTL-time enumeration from the selected RTL_B witness
-- prefix, after grouping explicitly atomic same-cycle transactions as single
-- ObservableUnit objects. The enumeration refines commit-cycle order and uses
```

```
-- within-cycle witness/replay attribution only to choose a realizable next unit.
-- The enumeration itself adds no observable order.
--
-- At each n < chain.N, add exactly one next unit γ. The replay-realizability
-- case analysis is performed against chain.idealAt n, so each within-cycle unit
-- is checked after all earlier enumerated same-cycle units have already been
-- incorporated into the replay prefix. Stop at the canonical immediate
-- post-unit semantic state and invoke IDR_immediate_post_unit_deterministic_replay
-- to populate ImmediateReplayStateForIdeal. Do not append a maximal ε-closure;
-- that operation belongs to J1_cutpoint_correspondence through
-- NormalizationBRules.



sorry


theorem J1_post_to_ref
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JTraceAssumptions C W)
(τ_post : Prefix C.RTL_B)
(hExec : Exec_post C τ_post) :
∃ τ_ref,
SelectedJWitness C W τ_post τ_ref ∧
Exec_ref C τ_ref ∧
CompatibleSys C τ_ref τ_post

theorem J1_selected_ref_prefix_is_liberal
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JTraceAssumptions C W)
(τ_post : Prefix C.RTL_B)
(τ_ref : Prefix C.Sys_ref)
(hSel : SelectedJWitness C W τ_post τ_ref)
(hExecRef : Exec_ref C τ_ref) :
LiberalReferencePrefix C τ_ref :=
by
-- Unfold RefAdmissible / the selected replay construction to obtain the
-- R4/E3/E5 and distinct-interface no-reverse facts for this exact selected
-- (τ_post, τ_ref, W) triple. No infinite reference execution is substituted for
-- the finite prefix produced by J.1.
sorry
-- Appendix L Note 1 projected safety theorem.
```

```
-- This theorem proves only the Σ_DUT-visible safety compatibility fragment.
-- Its proof follows the J.1 construction, but whenever an ε-modeled local timing
-- choice would normally require WC only to fix proof-internal timing, the proof
-- may instead use hJ.sigmaDUTSafetyWitness to show that the choice either is
-- covered by ordinary snooping/witness machinery or is certified
-- Σ_DUT-safety-inert. Such inert choices can add/remove/relabel no projected
-- Σ_DUT unit and can reorder only DUT-causally-independent projected units,
-- which CompatiblePDUT / CompatibleSysDUT already permit to commute.
theorem J1_post_to_ref_sigmaDUT_safety
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JSigmaDUTSafetyAssumptions C W)
(τ_post : Prefix C.RTL_B)
(hExec : Exec_post C τ_post) :
∃ τ_ref,
Exec_ref C τ_ref ∧
CompatibleSysDUT C τ_ref τ_post
```

Do not use J1_post_to_ref_sigmaDUT_safety as a substitute for J1_post_to_ref.
The full proof-internal trace theorem still requires JTraceAssumptions C W, including WC C W but not B4/B5, and concludes
CompatibleSys over the proof-internal alphabet C.Sigma. The projected theorem
is only the Appendix L Note 1 safety branch at the outer C.Sigma_DUT boundary.

Because hJ includes the standing RTL_B B1 field, RTLConsistentSourcePrefix may be read in the main theorem stack as matching the selected RTL_B execution prefix and, equivalently by B1, a prefix of the unique RTL_B Σ-trace for the fixed stimulus. The restriction remains a source-side restriction: arbitrary Sys_ref executions are not forced to have RTL_B matches merely because B1 holds on the post side.

Restricted reverse:

```
theorem J1_ref_to_post_restricted
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JTraceAssumptions C W)
(τ_ref : Prefix C.Sys_ref)
(hExec : Exec_ref C τ_ref)
(hConsistent : RTLConsistentSourcePrefix C W τ_ref) :
∃ τ_post,
Exec_post C τ_post ∧
CompatibleSys C τ_ref τ_post
```

Cutpoint abstraction and correspondence:

```
-- J.Abs is an RTL_B cutpoint-to-source-slice abstraction relation, not a
-- universal bit-level projection from arbitrary RTL registers to source states.
-- It is parameterized by the selected Appendix I/L witness and by the chosen
-- Sys_ref abstract boundaries.
def JAbs
(C : ComparisonInstance)
(W : WitnessBundle C)
(ρ_post : State)
(σ_ref : State) : Prop :=
ObservablePrefixAgreement C W ρ_post σ_ref ∧
PerProcessSourceFrontierSliceAgreement C W ρ_post σ_ref ∧
SourceVisibleValueSliceAgreement C W ρ_post σ_ref ∧
AbstractChannelStateSliceAgreement C W ρ_post σ_ref ∧
RendezvousEndpointRequestSliceAgreement C W ρ_post σ_ref ∧
BlockingRequestAttributionPendingSliceAgreement C W ρ_post σ_ref ∧
EnablementSliceAgreement C W ρ_post σ_ref

-- J.AbsConf is a compliance obligation for the selected RTL_B implementation,
-- abstract boundary, witness, and normalized cutpoints. It is required whenever
-- Corollary J.1 is used to transfer Pending_P, BoundaryReq, rendezvous endpoint
-- request, ChannelEnabled/ChannelDisabled, Front_P, or WFG facts. The selected
-- τ_ref used here is exactly the one marked by J1_post_to_ref through
-- SelectedJWitness; J.AbsConf is not applied to an arbitrary compatible prefix.
structure JAbsConf
(C : ComparisonInstance)
(W : WitnessBundle C) where
holds_for_selected_triple :
∀ τ_post τ_ref ρ_post σ_ref,
SelectedJWitness C W τ_post τ_ref →
TerminalCutpoint τ_post ρ_post →
TerminalCutpoint τ_ref σ_ref →
EpsilonNormalizedPair C τ_ref τ_post →
JAbs C W ρ_post σ_ref
-- "Preserved" here is preservation modulo exactly the changes already
-- accounted for by Issue/Commit linkage, E4 channel-realization semantics, and
-- the explicit boundary / sync-boundary rules. It must not be read as saying
-- that normalization can arbitrarily change request, pending, or attribution
```

-- facts while still satisfying J.AbsConf.
normalization_preserves_abs_slice :
∀ τ_post ρ_0 ρ_post,
EpsilonNormalizationSuffix C .post τ_post ρ_0 ρ_post →
SigmaHistoryUnchanged C .post ρ_0 ρ_post ∧
RequestPendingAttributionSlicePreservedModuloIssueCommitE4Boundary
C W ρ_0 ρ_post

-- B4/B5 normalization and J.AbsConf are cutpoint-only premises. They are not
-- fields of JTraceAssumptions and therefore do not narrow the plain J.1 trace
-- theorem.
structure JCutpointAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) extends JTraceAssumptions C W where
normalization : NormalizationBRules C
jAbsConf : JAbsConf C W

-- CutpointCorrespondence is the K/J bridge relation between corresponding
-- well-formed ε-quiescent cutpoints. The post-side cutpoint is named ρ_post to
-- emphasize that it is an RTL_B microstate related to σ_ref by J.Abs, not a raw
-- source-state cutpoint. J.Abs is the single source of truth for the transferred
-- source slices. Legacy fields such as NextFront agreement, buffered-channel
-- state agreement, raw rendezvous endpoint request agreement, and pending /
-- enablement agreement should be derived by projection lemmas from hCorr.j_abs,
-- rather than duplicated as independent fields that could drift.
structure CutpointCorrespondence
(C : ComparisonInstance)
(W : WitnessBundle C)
(σ_ref : State)
(ρ_post : State) where
j_abs : JAbs C W ρ_post σ_ref
ref_quiescent : EpsQuiescent σ_ref
post_quiescent : EpsQuiescent ρ_post
ref_wellformed : WellFormedCutpoint C .ref σ_ref
post_wellformed : WellFormedCutpoint C .post ρ_post
direct_input_anchor_agreement : DirectInputAnchorAgreement C W σ_ref ρ_post
array_mapping_agreement : ArrayMappingAgreement C W σ_ref ρ_post
r2c_fixed_point_agreement : R2CFixedPointAgreement C W σ_ref ρ_post

```
theorem cutpoint_nextfront_agreement
(C : ComparisonInstance)
(W : WitnessBundle C)
(σ_ref ρ_post : State)
(hCorr : CutpointCorrespondence C W σ_ref ρ_post) :
NextFrontAgreement C W σ_ref ρ_post :=
by
-- Projection from hCorr.j_abs.PerProcessSourceFrontierSliceAgreement.
sorry

theorem cutpoint_rendezvous_endpoint_request_agreement
(C : ComparisonInstance)
(W : WitnessBundle C)
(σ_ref ρ_post : State)
(hCorr : CutpointCorrespondence C W σ_ref ρ_post) :
RendezvousEndpointRequestSliceAgreement C W ρ_post σ_ref :=
by
-- Projection from hCorr.j_abs; no separate raw field is maintained.
sorry

theorem cutpoint_pending_enablement_agreement
(C : ComparisonInstance)
(W : WitnessBundle C)
(σ_ref ρ_post : State)
(hCorr : CutpointCorrespondence C W σ_ref ρ_post) :
BlockingRequestAttributionPendingSliceAgreement C W ρ_post σ_ref ∧
EnablementSliceAgreement C W ρ_post σ_ref :=
by
-- Projection from hCorr.j_abs; this is the fact consumed by K0/K2.
sorry

theorem J1_cutpoint_correspondence
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JCutpointAssumptions C W)
(τ_ref : Prefix C.Sys_ref)
(τ_post : Prefix C.RTL_B)
```

```
(hCompat : CompatibleSys C τ_ref τ_post)
(hNorm : EpsilonNormalizedPair C τ_ref τ_post)
(hSel : SelectedJWitness C W τ_post τ_ref) :
∃ σ_ref ρ_post,
TerminalCutpoint τ_ref σ_ref ∧
TerminalCutpoint τ_post ρ_post ∧
CutpointCorrespondence C W σ_ref ρ_post

-- Dependent package used by Appendix K. Every field refers to the same selected
-- post prefix, reference prefix, cutpoints, and witness.
structure SelectedMatchedCutpoint
(C : ComparisonInstance)
(W : WitnessBundle C) where
τ_post : Prefix C.RTL_B
τ_ref : Prefix C.Sys_ref
ρ_post : State
σ_ref : State
selected : SelectedJWitness C W τ_post τ_ref
post_exec : Exec_post C τ_post
ref_exec : Exec_ref C τ_ref
compatible : CompatibleSys C τ_ref τ_post
normalized : EpsilonNormalizedPair C τ_ref τ_post
post_terminal : TerminalCutpoint τ_post ρ_post
ref_terminal : TerminalCutpoint τ_ref σ_ref
ref_liberal : LiberalReferencePrefix C τ_ref
correspondence : CutpointCorrespondence C W σ_ref ρ_post

theorem J1_selected_matched_cutpoint
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JCutpointAssumptions C W)
(τ_post : Prefix C.RTL_B)
(hExecPost : Exec_post C τ_post) :
∃ M : SelectedMatchedCutpoint C W,
M.τ_post = τ_post :=
by
-- Apply J1_post_to_ref once using hJ.toJTraceAssumptions, normalize
-- that exact pair using hJ.normalization, use J1_selected_ref_prefix_is_liberal
```

```
-- on the same τ_post/τ_ref pair, and invoke J1_cutpoint_correspondence with
-- the same SelectedJWitness evidence and hJ.jAbsConf. Package all results
-- without reselecting either prefix or either cutpoint.
sorry
```

The proof of J1_cutpoint_correspondence must populate CutpointCorrespondence.j_abs from hJ.jAbsConf.holds_for_selected_triple for the same selected (τ_post, τ_ref, ρ_post, σ_ref, W) produced by J1_post_to_ref. It must populate CutpointCorrespondence.ref_quiescent and CutpointCorrespondence.post_quiescent from hNorm / the terminal normalized cutpoint construction, and must populate CutpointCorrespondence.ref_wellformed and CutpointCorrespondence.post_wellformed from the terminal cutpoint construction and the side-specific execution well-formedness facts supplied by hJ.wellformed and the compatible Sys_ref / RTL_B executions. Thus any later K theorem that receives hCorr also receives both the ε-quiescence, WellFormedCutpoint, and J.Abs source-slice facts needed to apply K0 and to transfer request/pending/endpoint-request facts. IDR_immediate_post_unit_deterministic_replay supplies the source-side replay slice used by plain J.1. After hNorm constructs the selected ε-quiescent pair, IDR_deterministic_replay supplies the corresponding normalized source-side replay slice used by the cutpoint theorem. J.AbsConf supplies the RTL_B-side request, pending, attribution, endpoint-request, and enablement slice. The latter is not inferred from the P-local Σ-label sequence alone. Legacy agreement facts, when needed, should be obtained through projection lemmas from hCorr.j_abs rather than through duplicate CutpointCorrespondence fields.

18. Appendix K WFG and liveness

Define WFG over the selected proof-internal Sys_ref / RTL_B boundaries, not over the outer Sigma_DUT projection. In ordinary cases this distinction may collapse to the identity projection. In elaborated cases, however, Sys_ref = Sys_B^elab(E), and the WFG must see the explicit staged/bundle/join/epoch-gate abstract channels introduced by E even when sigmaDUTProjection / Π_elab hides their events from the outer DUT/testbench-visible alphabet.

```
def FrontierItemChannelSide
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem)
(c : Channel) : Prop :=
∃ q,
```

MsgOfFrontierItemSide C side x = some q ∧
q.channel = c

def ComplementOwnerItemSide
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem)
(Q : Process) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
ComplementOwner C side q = Q

def InternalChannelOfFrontierItem
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
InternalChannel C side q.channel

def WFGProofInternalBoundaryItem
(C : ComparisonInstance)
(side : Side)
(x : FrontierItem) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
q.channel ∈ ProofInternalChannels C side ∧
ChannelRepresentedInSigma C.Sigma q.channel ∧
¬ ChannelVisibilityDefinedOnlyBySigmaDUT C q.channel

def WFG
(C : ComparisonInstance)
(side : Side)
(σ : State) : Digraph Process :=
{ edge := WFGEdge C side σ }

def WFGEdge
(C : ComparisonInstance)

```
(side : Side)
(σ : State)
(P Q : Process) : Prop :=
∃ x,
x ∈ Front C side P σ ∧
BlockingMessageFrontierItem x ∧
BoundaryReqItemSide C side x σ ∧
ChannelDisabledItemSide C side x σ ∧
ComplementOwnerItemSide C side x Q ∧
InternalChannelOfFrontierItem C side x ∧
WFGProofInternalBoundaryItem C side x

theorem WFG_ignores_sigmaDUTProjection
(C : ComparisonInstance)
(side : Side)
(σ : State)
(P Q : Process) :
WFGEdge C side σ P Q ↔
WFGEdgeComputedOverProofInternalSigma C side σ P Q :=
by
-- WFG, Front, BoundaryReqItemSide, ChannelEnabledItemSide, and
-- ChannelDisabledItemSide are interpreted over C.Sigma and the chosen
-- Sys_ref / RTL_B abstract channels through the owning dynamic message
-- instance of each message-operation frontier item. sigmaDUTProjection is used
-- only after the proof-internal comparison when projecting to the outer
-- DUT/testbench-visible observation boundary.
sorry
```

Edges are generated from Front_P(σ), not arbitrary non-frontier asserted requests. This intentionally makes the WFG sparser than a graph over all asserted boundary requests: only pending blocking frontier items generate wait-for edges. For message-operation frontier items, BoundaryReqItemSide and ChannelDisabledItemSide recover the owning DynMsgInstance and then apply the operation-attributive BoundaryReqSide / ChannelDisabledSide predicates. Failed non-blocking polls and already-asserted non-frontier requests may affect state or occupancy through other rules, but they do not themselves create WFG dependency edges.

The Lean development must include the Appendix K.2.2 / Appendix R.1 structural classification theorem as an explicit milestone. Front_P(σ) is not an independent primitive for liveness proofs: at ε-quiescent cutpoints it is derived from the blocking message-operation slice of NextFront_P together with the operation-attributive BoundaryReq predicate.

```
-- Message-operation frontier items whose owning dynamic message instance lies
-- in the Q_Ω slice for process P. This is a predicate on FrontierItem, not a
-- direct membership test in Q_Ω. The document's Q_Ω,P is a set of
-- DynMsgInstance, while Ω_msg,P / NextFront_P are FrontierItem-level objects.
-- Therefore this predicate relates the two universes through
-- MsgInstanceOfFrontierItem / MsgOfFrontierItemSide rather than by literal set
-- intersection.
def MessageFrontierItemInQOmega
(C : ComparisonInstance)
(W : WitnessBundle C)
(side : Side)
(P : Process)
(x : FrontierItem) : Prop :=
∃ q,
MsgOfFrontierItemSide C side x = some q ∧
q ∈ QOmegaOfProcess C W P
```

Q_Ω membership is therefore invariant as the proof moves among cutpoints of the selected witness. Only NextFront_P(σ), BoundaryReq, ChannelEnabled, and ChannelDisabled are state dependent. The side-indexed MsgOfFrontierItemSide projection must identify the same witness-owned dynamic operation instance on corresponding reference and post cutpoints; that agreement is supplied by the selected witness and J.Abs/J.AbsConf, not by constructing separate side-specific Q_Ω sets.

```
def BlockingMsgNextFront
(C : ComparisonInstance)
(W : WitnessBundle C)
(side : Side)
(P : Process)
(σ : State) : Set FrontierItem :=
{ x |
x ∈ NextFront C side P σ ∧
```

```
MessageFrontierItemInQOmega C W side P x ∧
BlockingMessageFrontierItem x ∧
MessageKindOfFrontierItem C side x ∈ {MsgKind.Push, MsgKind.Pop} }
```

This preserves the document's distinction between Ω_msg,P / NextFront_P, whose elements are FrontierItem objects, and Q_Ω,P, whose elements are DynMsgInstance objects. A message frontier item belongs to the blocking NextFront slice only when its owning dynamic message instance is defined and lies in Q_Ω,P.

```
def FrontFromNext
(C : ComparisonInstance)
(W : WitnessBundle C)
(side : Side)
(P : Process)
(σ : State) : Set FrontierItem :=
{ x |
x ∈ BlockingMsgNextFront C W side P σ ∧
BoundaryReqItemSide C side x σ }

theorem K0_front_nextfront_classification
(C : ComparisonInstance)
(W : WitnessBundle C)
(side : Side)
(P : Process)
(σ : State)
(hK : KStructuralAssumptions C W)
(hQ : EpsQuiescent σ)
(hWF : WellFormedCutpoint C side σ) :
Front C side P σ =
FrontFromNext C W side P σ :=
by
-- Appendix K.2.2 / Appendix R.1 proof target, aligned with the v42 expanded
-- proof of Lemma K.0. The forward direction must prove that a pending blocking
-- frontier item is not merely minimal among pending blocking items, but minimal
-- in the full Remain_P(σ) / NextFront_P sense. The proof therefore splits on
-- any hypothetical y <_P x:
--   1. y cannot be a preceding synchronization item, because the current
```

```
--     interval definition plus R3/E1 puts it in Done_P(σ);
--   2. y cannot be a preceding anchored sequential signal item, because R2/E2
--     realizes it at pred_S/succ_S, nor a preceding combinational item whose
--     emitted R2C unit has already been realized;
--   3. y cannot be an earlier same-interval message item, because R4/E3 gives
--     issue order; if it committed it is Done, if it is pending blocking it
--     contradicts Front-minimality, if it is successful NB it is Done, and if
--     it is failed NB it is not an Ω_P frontier item;
--   4. y cannot come from an earlier source interval, because the closing sync
--     has committed and R3/E5 forces earlier blocking pref_S work to have
--     committed no later than that sync.
-- The reverse direction uses Appendix C stable attribution: an asserted
-- BoundaryReq for x ∈ BlockingMsgNextFront is attributed to its owning q_x and
-- cannot be retagged to another same-endpoint operation without an intervening
-- commit. Younger already-asserted requests may exist as non-frontier protocol
-- state under Appendix B no-withdrawal/drain-only rules, but they do not enter
-- Front_P(σ) while an earlier pending blocker is frontier-minimal.
sorry

theorem K0_front_correspondence_from_cutpoint
(C : ComparisonInstance)
(W : WitnessBundle C)
(P : Process)
(σ_ref ρ_post : State)
(hK : KStructuralAssumptions C W)
(hCorr : CutpointCorrespondence C W σ_ref ρ_post) :
Front C .ref P σ_ref = Front C .post P ρ_post :=
by
-- Apply K0_front_nextfront_classification on both sides using hK, with
-- hCorr.ref_quiescent / hCorr.post_quiescent supplying the ε-quiescence
-- hypotheses and hCorr.ref_wellformed / hCorr.post_wellformed supplying the
-- WellFormedCutpoint hypotheses. Then use projection lemmas from hCorr.j_abs,
-- such as cutpoint_nextfront_agreement and
-- cutpoint_pending_enablement_agreement, for NextFront agreement and
-- operation-attributive BoundaryReqItemSide agreement on the common
-- witness-fixed BlockingMsgNextFront slice. The document-numbered A1-A4 and A6-A11
facts are accessed
-- through hK.docNoA5; the Appendix C Issue/Commit attribution premise is accessed
```

```
-- through hK.issue_commit_wf.
sorry
```

This theorem is intentionally stated for BlockingMsgNextFront_P, not for the broader MsgNextFront_P. Successful non-blocking message-operation frontier items may contribute committed Push_c / Pop_c Σ-labels, and failed non-blocking polls are ε-steps, but neither creates a pending blocking WFG cause.

```
def ProcessStalledAtFront
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State) : Prop :=
FrontierBlocked C side P σ

theorem process_stalled_at_front_iff_frontier_blocked
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State) :
ProcessStalledAtFront C side P σ ↔
FrontierBlocked C side P σ :=
Iff.rfl

-- ProcessStalledAtFront is now a document-facing alias, not a second
-- independently maintained conjunction. The Section 13 progress machinery,
-- the WFG construction, the canonical closed/drain-closed component, and the
-- Appendix K deadlock predicates therefore consume exactly the same
-- FrontierBlocked definition. This eliminates the former proof seam in which
-- equivalence of two textually duplicated bodies was only promised by a comment.

def WFGClosed
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(σ : State) : Prop :=
∀ P Q,
P ∈ D →
```

```
WFGEdge C side σ P Q →
Q ∈ D

def WFGTerminalSCC
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(σ : State) : Prop :=
D.Nonempty ∧
StronglyConnectedSubgraph (WFG C side σ) D ∧
WFGClosed C side D σ

-- A drain escape is not necessarily a WFG edge leaving D. It is an enabled
-- non-frontier blocking transfer/drain that would discharge already-issued
-- downstream work for some process in D. Such a drain may be intra-process or
-- may not appear as a graph edge at all, because WFG edges are generated only
-- from current minimal blocking frontier items Front_P(σ).
--
-- Important: "non-frontier" here means "not in the current minimal pending
-- blocking frontier Front_P(σ)." It does not mean that the operation instance is
-- absent from the broader dynamic/message-frontier universe Ω_msg,P / Q_Ω,P.
-- Conversely, it should not be characterized as a member of the current
-- BlockingMsgNextFront_P(σ) slice together with BoundaryReq, because K.0 proves
-- that, at ε-quiescent well-formed cutpoints, those items are exactly Front_P(σ).
--
-- Downstream automatic-flush work is already-issued downstream protocol state:
-- its owning message-operation frontier item may still be part of the broader
-- witness universe, may still have BoundaryReq asserted, and may commit as
-- drain-only progress, but its item is not currently minimal in Front_P(σ).
def OwnsCurrentFrontierMsgInstance
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State)
(q : DynMsgInstance) : Prop :=
∃ x,
x ∈ Front C side P σ ∧
MsgOfFrontierItemSide C side x = some q

def NonFrontierBlockingDrainCandidate
(C : ComparisonInstance)
(side : Side)
(P : Process)
```

```
(σ : State)
(q : DynMsgInstance) : Prop :=
∃ x,
DrainableDownstreamRequest C side P σ x ∧
MsgOfFrontierItemSide C side x = some q ∧
x ∉ Front C side P σ

def EnabledNonFrontierBlockingDrainForProcess
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State) : Prop :=
∃ q,
OwnsMessageEndpoint P q ∧
Blocking q ∧
NonFrontierBlockingDrainCandidate C side P σ q ∧
BoundaryReqPersistentDomainSide C side q σ ∧
BoundaryReqSide C side q σ ∧
ChannelEnabledSide C side q σ
def NoEnabledNonFrontierBlockingDrain
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State) : Prop :=
¬ EnabledNonFrontierBlockingDrainForProcess C side P σ

def DrainClosed
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(σ : State) : Prop :=
∀ P,
P ∈ D →
NoEnabledNonFrontierBlockingDrain C side P σ
```

The K.1 SCC contradiction should use this definition. The closed SCC supplies the internal frontier WFG cycle; DrainClosed separately rules out currently enabled non-frontier blocking drains for processes in D. In v42, however, DrainClosed is not misread as saying that outside-D or environment activity can never make such a non-frontier request ChannelEnabled later. The K.1 Exec bridge therefore uses fair drain-normalization: if a D-owned already-issued non-frontier request later becomes continuously serviceable under B2/B3, allow it to commit as drain-only progress, and use the finite-drain argument to reach a later ε-quiescent drain-closed cutpoint when needed.

```
def HasOutgoingWFGEdgeInD
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(P : Process)
(σ : State) : Prop :=
∃ Q,
Q ∈ D ∧
WFGEdge C side σ P Q

def ClosedDrainClosedWFGComponent
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(σ : State) : Prop :=
WFGTerminalSCC C side D σ ∧
DrainClosed C side D σ ∧
(∀ P, P ∈ D → FrontierBlocked C side P σ) ∧
(∀ P, P ∈ D → HasOutgoingWFGEdgeInD C side D P σ)

def ExistsClosedDrainClosedWFGCycle
(C : ComparisonInstance)
(side : Side)
(σ : State) : Prop :=
∃ D, ClosedDrainClosedWFGComponent C side D σ

def InternalMPDeadlockShapeAt
(C : ComparisonInstance)
(side : Side)
(σ : State) : Prop :=
EpsQuiescent σ ∧
ExistsClosedDrainClosedWFGCycle C side σ

-- Public document-level deadlock predicate: the shape above together with
-- reachability in the selected comparison side.
def ReachableInternalMPDeadlockAt
```

(C : ComparisonInstance)
(side : Side)
(σ : State) : Prop :=
Reachable (SystemOfSide C side) σ ∧
InternalMPDeadlockShapeAt C side σ

The K.1 SCC contradiction uses InternalMPDeadlockShapeAt after reachability has been supplied by an execution prefix or by a separate hypothesis. ClosedDrainClosedWFGComponent is the single canonical conjunction: the terminal SCC supplies the cycle of internal message-passing dependencies; DrainClosed rules out escape by downstream draining; FrontierBlocked ensures every process in D is stalled on its current blocking frontier, not merely carrying a non-frontier asserted request; and HasOutgoingWFGEdgeInD supplies the non-vacuity/internal-cause condition. Every process in the candidate deadlock set must have at least one WFG edge, generated from an internal proof-visible message-passing frontier item, to another process in the same set. This excludes the spurious singleton case where a process is merely waiting on an external environment or testbench channel and therefore has no internal WFG cause.

ProcessStalledAtFront is definitionally equal to FrontierBlocked, so older document-facing statements that use the former name reduce to this same conjunction without an additional semantic lemma. ExistsClosedDrainClosedWFGCycle, InternalMPDeadlockShapeAt, the per-process participation predicate, and the terminated-peer corollaries all quantify over ClosedDrainClosedWFGComponent directly.

ReachableInternalMPDeadlockAt is the public document-level predicate. The factorization prevents a theorem from accidentally treating an arbitrary state with the deadlock graph shape as a reachable deadlock. Finite-prefix and execution-scoped theorems may use InternalMPDeadlockShapeAt because their terminal-prefix premise already supplies reachability; Appendix K and Appendix R facade statements should expose the reachable wrapper where the document speaks about a reachable deadlock.

Terminated-process convention (v42 K.1). The scheduling-rules document additionally fixes a terminated-process convention: at an examined cutpoint, a process that is blocked solely on channels whose unique complementary endpoint process has terminated in its current reset epoch, with no intervening reset of that peer, is not counted as participating in an internal message-passing deadlock. Such a configuration is a protocol/termination modeling issue, not a K.1 deadlock. In this strategy that convention is not an extra assumption — it is a corollary of finish_process_idle_padding_until_next_reset together with the closed-SCC deadlock definition above, and Lean should prove it explicitly:

```
def ProcessParticipatesInInternalMPDeadlockShapeAt
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State) : Prop :=
EpsQuiescent σ ∧
∃ D,
P ∈ D ∧
ClosedDrainClosedWFGComponent C side D σ

theorem process_participates_iff_deadlock_component_member
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State) :
ProcessParticipatesInInternalMPDeadlockShapeAt C side P σ ↔
EpsQuiescent σ ∧
∃ D, P ∈ D ∧ ClosedDrainClosedWFGComponent C side D σ :=
Iff.rfl

theorem terminated_peer_waiting_not_in_deadlock_component
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State)
(hP2P : PointToPointChannels C)
(hWait : BlockedOnlyOnPeersTerminatedInCurrentEpoch C side P σ) :
¬ ∃ D,
P ∈ D ∧ ClosedDrainClosedWFGComponent C side D σ :=
by
-- A peer terminated in its current reset epoch, with no later reset before σ,
-- contributes no active complementary endpoint request at σ. Under point-to-point
-- ownership, a process waiting only on such peers cannot satisfy the outgoing-edge
-- and terminal-SCC conjuncts of the canonical internal-deadlock component.
-- A later reset creates a new epoch and is outside this theorem premise.
sorry

theorem terminated_peer_waiting_not_internal_MP_deadlock
(C : ComparisonInstance)
(side : Side)
(P : Process)
(σ : State)
(hP2P : PointToPointChannels C)
(hWait : BlockedOnlyOnPeersTerminatedInCurrentEpoch C side P σ) :
```

```
¬ ProcessParticipatesInInternalMPDeadlockShapeAt C side P σ :=
by
intro hPart
exact terminated_peer_waiting_not_in_deadlock_component
  C side P σ hP2P hWait hPart.2
```

Proof sketch. By A6 point-to-point channels, each channel in P's blocking frontier has a unique complementary endpoint process Q. The epoch-aware hWait premise says that, at the examined cutpoint σ, every such Q has a FINISH_Q occurrence in Q's current reset epoch and no reset affecting Q has occurred after that finish. By finish_process_idle_padding_until_next_reset, Q is padded with idle ε-steps from that FINISH_Q occurrence through σ: its Front is empty, it asserts no BoundaryReq, it commits no message operations, and it generates no blocking WFG cause at σ. Hence Q fails FrontierBlocked — its frontier is empty, violating the first conjunct — so Q cannot belong to a ClosedDrainClosedWFGComponent at σ. But every WFG edge out of P generated by P's blocking frontier targets such a Q. Therefore, for any D containing P, either P's outgoing WFG edges leave D, violating the HasOutgoingWFGEdgeInD/terminal-SCC conditions, or D would have to contain a terminated Q, violating the FrontierBlocked condition for Q. Either way no canonical closed drain-closed WFG component containing P exists at σ. A later reset of Q creates a new epoch and invalidates the hWait premise rather than being suppressed by this result. The per-process corollary follows directly by unfolding ProcessParticipatesInInternalMPDeadlockShapeAt; no unproved ProcessStalledAtFront/FrontierBlocked conversion is needed.

Scope warning (v42 K.1). This convention is a scoping rule for the deadlock definition only. It does not verify, and must not be read as verifying, intended termination behavior: explicit EOF, done, shutdown, or cancellation protocols, and any design intent that a consumer should observe a producer's termination, must be modeled explicitly as message-passing or signal protocol in Sys_ref and RTL_B and proved through the ordinary I/J/K obligations, or verified outside Appendix K. A process left waiting forever on a terminated peer is excluded from the K.1 deadlock alarm precisely because the document treats it as a protocol-level modeling defect, not as an internal message-passing deadlock; the exclusion must therefore never be used to argue that such a configuration is acceptable.

K assumptions should keep the document-numbered Appendix K assumptions visible, but the Lean proof must also separate the structural K premises from A5. Lemma K.2b is the bridge used before A5 is invoked, so K.2b and the K.0/K.2 structural lemmas must not take an assumption bundle that already contains A5. The DocKStructuralAssumptions structure below therefore stores A1 and A6–A11. The document-numbered A2–A4 names are derived projections of the B4, B2, and B3 fields already carried by A1's BRules component, rather than duplicate assumption fields. A5 is carried only by the final KAssumptions wrapper used for

K.1.

The v45 Appendix K update changes A5 substantially. A5 is no longer a scheduler-robust liveness premise over all source scheduling choices. It is the conservative serial-blocking source-liveness premise: for A5 only, a process may not issue a later same-interval blocking Push/Pop until every earlier same-interval blocking Push/Pop of that process has committed. The ordinary Appendix J witness, Corollary J.1 cutpoint correspondence, and RTL_B implementation may still contain overlapped / liberal issue. Lemma K.2b is the bridge from such a witness-relative overlapped Sys_ref deadlock to a serial-source deadlock that contradicts A5.

The J.1-to-K.2b path is finite-prefix throughout and uses Prefix C.Sys_ref as its semantic carrier throughout. Serial blocking is a restriction of the ordinary liberal finite source-prefix semantics, not a replacement for the R4/E3/E5/no-reverse discipline. Infinite executions remain the carrier for B2/B3 fairness and admissibility, but J.1 produces a semantic finite reference prefix, every K.2b one-step reduction constructs another semantic finite reference prefix, and A5 consumes the resulting finite conservative-serial prefix. ExecutionBucketed C.Sigma appears in K.2b only through RefPrefixBuckets as a derived measure/stream representation; no arbitrary edited bucket list is treated as realizable without constructing the corresponding Prefix C.Sys_ref.

def A11_Kepsilon_WFGInert
(C : ComparisonInstance) : Prop :=
∀ side P τ t_0 t σ_0 σt,
StateAtCycleInPrefix C side τ t_0 σ_0 →
StateAtCycleInPrefix C side τ t σt →
$t_0 \leq t$ →
InternalOrEpsilonOnlyBetween C side τ t_0 t →
FrontierBlocked C side P σ_0 →
FrontierRemainsBlockedOver C side P τ t_0 t →
WFGInertForProcess C side P σ_0 σt ∧
NoNewBlockingFrontierCause C side P σ_0 σt

def BarrierWellFormed
(C : ComparisonInstance)
(W : WitnessBundle C) : Prop :=
∀ b,
SyncChannelInstance C b →
DesignatedParticipantsReachSameBarrierCount C W b ∧
NoPermanentBarrierBypass C W b

inductive SourceBlockingIssuePolicy
| liberal

```
| serial

-- Every K.2b theorem below reduces semantic reference-system prefixes. The
-- bucket list is a derived observation of an already-realized Prefix C.Sys_ref;
-- it is never an independent carrier from which the proof later assumes a
-- semantic prefix can be reconstructed.
def RefPrefixBuckets
(C : ComparisonInstance)
(ρ_ref : Prefix C.Sys_ref) : ExecutionBucketed C.Sigma :=
ObservableBucketsOfSystemPrefix C .ref ρ_ref

-- Completion of an earlier blocking operation before a later same-interval issue.
-- In the document-aligned serial policy, serial means the earlier blocking
-- operation has committed before the later operation issues; no same-cycle
-- commit-then-issue exception is assumed. If a future document revision adds a
-- precise intra-bucket commit-before-issue convention, represent it as a
-- separately named optional extension theorem, not as part of A5 by default.
def EarlierBlockingOpCompletedBeforeLaterIssue
(C : ComparisonInstance)
(side : Side)
(τ : ExecutionBucketed C.Sigma)
(qEarlier qLater : DynMsgInstance)
(tIssueLater : Cycle) : Prop :=
CommitsBefore C side τ qEarlier tIssueLater

-- qLater is an eager issue if it is issued before some source-earlier blocking
-- Push/Pop in the same dynamic interval has completed in the sense above. Eager
-- issue is permitted by the liberal witness policy, but forbidden by the serial A5
-- policy.
def EagerBlockingIssueAt
(C : ComparisonInstance)
(side : Side)
(τ : ExecutionBucketed C.Sigma)
(qLater qEarlier : DynMsgInstance)
(t : Cycle) : Prop :=
Blocking qLater ∧ Blocking qEarlier ∧
SameOwningProcess qEarlier qLater ∧
SameDynamicSourceInterval C side τ qEarlier qLater ∧
SourceOrderBefore C side τ qEarlier qLater ∧
IssueAtCycle C side τ qLater t ∧
¬ EarlierBlockingOpCompletedBeforeLaterIssue C side τ qEarlier qLater t

structure EagerBlockingIssueEvent
(C : ComparisonInstance)
```

```
(side : Side)
(ρ : ExecutionBucketed C.Sigma) where
qLater : DynMsgInstance
qEarlier : DynMsgInstance
tIssue : Cycle
eager : EagerBlockingIssueAt C side ρ qLater qEarlier tIssue

-- The concrete implementation can choose any finite representation of the eager
-- issue events of the finite prefix ρ, provided membership is equivalent to the
-- EagerBlockingIssueAt witness above. This Finset is the measure used by K.2b.
def EagerBlockingIssueEvents
(C : ComparisonInstance)
(side : Side)
(ρ : ExecutionBucketed C.Sigma) : Finset (EagerBlockingIssueEvent C side ρ) :=
AllEagerBlockingIssueEventsInFinitePrefix C side ρ

-- IssuedBlockingEndpointSourceOrderStream is ordered by source/endpoint order,
-- not by issue time. Deferring one eager issue may move an event later in time,
-- so an issue-time-ordered stream would not be prefix-monotone. The prefix theorem
-- below is about the source-order endpoint stream: deferral may withhold a suffix
-- of the endpoint stream, but it never drops an earlier endpoint operation while
-- retaining a later one.
def IssuedBlockingEndpointSourceOrderStream
(C : ComparisonInstance)
(side : Side)
(ρ : ExecutionBucketed C.Sigma)
(c : Channel)
(dir : EndpointDirection) : List DynMsgInstance :=
BlockingEndpointInstancesIssuedInPrefixOrderedBySourceEndpointOrder C side ρ c dir

-- DefersOneEagerIssue is the run-surgery relation used by the one-step reduction.
-- It identifies a particular eager issue event in ρ, removes or defers that
-- operation to its serial slot, preserves all retained earlier committed transfers
-- and their values, and changes no endpoint stream except by suppressing a
-- source-order suffix on explicit WFG-relevant point-to-point channels. The relation
-- must also record that any new terminal cutpoint σ' used after the edit is reached
-- by finite ε/drain normalization from the edited prefix.
def DefersOneEagerIssue
(C : ComparisonInstance)
(side : Side)
(ρ ρ' : ExecutionBucketed C.Sigma) : Prop :=
∃ ev : EagerBlockingIssueEvent C side ρ,
RemovesOrDelaysExactlyEagerIssue C side ρ ρ' ev ∧
RetainedCommittedTransferPrefix C side ρ' ρ ∧
```

```
∀ c dir,
ChannelCanContributeWFGEdge C c →
IsPrefix
(IssuedBlockingEndpointSourceOrderStream C side ρ' c dir)
(IssuedBlockingEndpointSourceOrderStream C side ρ c dir)

def ConservativeSerialBlockingRun
(C : ComparisonInstance)
(side : Side)
(τ : ExecutionBucketed C.Sigma) : Prop :=
∀ qLater qEarlier t,
EagerBlockingIssueAt C side τ qLater qEarlier t → False

def LiberalBlockingRun
(C : ComparisonInstance)
(side : Side)
(τ : ExecutionBucketed C.Sigma) : Prop :=
IssueOrderConformsToR4E3E5 C side τ ∧
NoReverseDistinctInterfaceIssue C side τ

-- Prefix predicate for all finite prefixes of an infinite execution. The policy
-- restrictions must hold on every finite prefix, equivalently on the infinite run;
-- they are not checked only on one arbitrarily chosen prefix.
def LiberalReferencePrefix
(C : ComparisonInstance)
(ρ_ref : Prefix C.Sys_ref) : Prop :=
Exec_ref C ρ_ref ∧
IssueOrderConformsToR4E3E5 C .ref ρ_ref ∧
NoReverseDistinctInterfaceIssue C .ref ρ_ref

def ConservativeSerialReferencePrefix
(C : ComparisonInstance)
(ρ_ref : Prefix C.Sys_ref) : Prop :=
LiberalReferencePrefix C ρ_ref ∧
NoEagerBlockingIssue C .ref ρ_ref

theorem conservative_serial_reference_prefix_implies_liberal
(C : ComparisonInstance)
(ρ_ref : Prefix C.Sys_ref) :
ConservativeSerialReferencePrefix C ρ_ref →
LiberalReferencePrefix C ρ_ref :=
by
intro h
exact h.1
```

```
-- A5 is stated directly over finite admissible source prefixes.
def InternalMPDeadlockFreeOnSerialPrefixesRef
(C : ComparisonInstance) : Prop :=
∀ ρ_ref σ_ref,
ConservativeSerialReferencePrefix C ρ_ref →
TerminalCutpoint ρ_ref σ_ref →
¬ InternalMPDeadlockShapeAt C .ref σ_ref

-- Convenience alias only; it adds no infinite-execution domain.
def InternalMPDeadlockFreeOnSerialExecRef
(C : ComparisonInstance) : Prop :=
InternalMPDeadlockFreeOnSerialPrefixesRef C

-- Finite measure used by Lemma K.2b. For a finite prefix ρ, EagerIssueCount is
-- the number of dynamic blocking issue events in ρ that satisfy
-- EagerBlockingIssueAt. The count is finite because ρ is finite.
def EagerIssueCount
(C : ComparisonInstance)
(side : Side)
(ρ : ExecutionBucketed C.Sigma) : Nat :=
Finset.card (EagerBlockingIssueEvents C side ρ)

theorem zero_eager_iff_serial_blocking_prefix
(C : ComparisonInstance)
(side : Side)
(ρ : ExecutionBucketed C.Sigma) :
EagerIssueCount C side ρ = 0 ↔
ConservativeSerialBlockingRun C side ρ :=
by
-- Unfold EagerIssueCount, EagerBlockingIssueEvents, and
-- ConservativeSerialBlockingRun.
sorry

-- Semantic-prefix wrappers for the bucket-level eager-issue machinery. Because
-- both arguments are Prefix C.Sys_ref, DefersOneEagerIssueRefPrefix includes the
-- realizability fact that the edited bucket projection comes from an actual
-- admissible reference-system prefix.
def RefEagerIssueCount
(C : ComparisonInstance)
(ρ_ref : Prefix C.Sys_ref) : Nat :=
EagerIssueCount C .ref (RefPrefixBuckets C ρ_ref)

def RefEndpointIssuedBlockingStreamPrefix
```

```
(C : ComparisonInstance)
(ρ_ref' ρ_ref : Prefix C.Sys_ref)
(c : Channel)
(dir : EndpointDirection) : Prop :=
IsPrefix
(IssuedBlockingEndpointSourceOrderStream
  C .ref (RefPrefixBuckets C ρ_ref') c dir)
(IssuedBlockingEndpointSourceOrderStream
  C .ref (RefPrefixBuckets C ρ_ref) c dir)

structure DefersOneEagerIssueRefPrefix
(C : ComparisonInstance)
(ρ_ref ρ_ref' : Prefix C.Sys_ref) : Prop where
-- This field is the explicit semantic/admissibility obligation. It requires an
-- infinite RefAdmissible extension of the edited finite prefix, including the
-- applicable B2/B3 fairness and progress premises.
output_exec_ref : Exec_ref C ρ_ref'
output_issue_order : IssueOrderConformsToR4E3E5 C .ref ρ_ref'
output_no_reverse : NoReverseDistinctInterfaceIssue C .ref ρ_ref'
bucket_deferral :
  DefersOneEagerIssue
   C .ref
   (RefPrefixBuckets C ρ_ref)
   (RefPrefixBuckets C ρ_ref')

theorem DefersOneEagerIssueRefPrefix.output_liberal
(C : ComparisonInstance)
(ρ_ref ρ_ref' : Prefix C.Sys_ref)
(h : DefersOneEagerIssueRefPrefix C ρ_ref ρ_ref') :
LiberalReferencePrefix C ρ_ref' :=
⟨h.output_exec_ref, h.output_issue_order, h.output_no_reverse⟩
theorem zero_ref_eager_iff_serial_reference_prefix
(C : ComparisonInstance)
(ρ_ref : Prefix C.Sys_ref)
(hLiberal : LiberalReferencePrefix C ρ_ref) :
RefEagerIssueCount C ρ_ref = 0 ↔
ConservativeSerialReferencePrefix C ρ_ref :=
by
-- Unfold the semantic wrappers, use zero_eager_iff_serial_blocking_prefix on
-- RefPrefixBuckets C ρ_ref, and identify ConservativeSerialBlockingRun of that
-- projection with NoEagerBlockingIssue C .ref ρ_ref.
sorry

structure DocKStructuralAssumptions
```

```
(C : ComparisonInstance)
(W : WitnessBundle C) where
-- A1. R-rules and B-rules.
-- This is the document-numbered Appendix K assumption that the selected
-- Sys_ref / RTL_B comparison satisfies the semantic rule bundles needed by K.
-- E-rules are still required by the imported Appendix J compatibility theorem,
-- but they are not renamed as a document-numbered K assumption here.
A1_R_and_B_rule_conformance :
RRules C ∧ BRules C

-- A2–A4 are not stored a second time. Their document-facing names are
-- defined below as projections from A1_R_and_B_rule_conformance.2, preventing
-- drift between the Appendix K aliases and the underlying B4/B2/B3 obligations.

-- A5 is deliberately not included in this structural bundle. A5 is the
-- source-level serial liveness input consumed only by the final K.1 contradiction,
-- after K.2b has converted the witness-relative / overlapped reference deadlock
-- into a serial-source deadlock. This prevents K.2b from depending, even
-- accidentally, on the premise it is meant to help contradict.

-- A6. Point-to-point channels.
-- This is the document-numbered Appendix K A6 assumption. It records the
-- ordinary point-to-point channel premise. The stronger Lean-side requirement
-- that every WFG-relevant explicit abstract boundary, including introduced
-- Sys_B^elab staged, bundle, join, or epoch-gate boundary channels, be
-- point-to-point is carried by KElaborationPrerequisite below, not by A7.
A6_point_to_point_channels :
 PointToPointChannels C

-- A7. Determinism / witness selection / cutpoint abstraction.
-- RTL_B determinism is the B1 part used by Appendix K; WC records the selected
-- Sys_ref witness supplied by the Appendix I/J/L witness machinery; and
-- J.AbsConf records that the same selected RTL_B cutpoint, Sys_ref cutpoint,
-- and witness discharge the source-slice abstraction used by Corollary J.1 and
-- Appendix K.
--
-- Keep A7 aligned with the scheduling-rules document: A7 has these three
-- conjuncts only. Point-to-point/WFG-boundary obligations belong to A6 and to
-- the Lean-side KElaborationPrerequisite.
A7_determinism_witness_selection_cutpoint_abstraction :
 PostDeterministicSigmaTrace C ∧
 WC C W ∧
 JAbsConf C W
```

```
-- A8. Barrier well-formedness (SyncChannels).
-- Barrier stalls caused by mismatched or conditional participation are outside
-- Appendix K's internal message-passing deadlock analysis.
A8_barrier_well_formedness :
BarrierWellFormed C W

-- A9. Single-transfer-per-cycle endpoint constraint.
A9_single_transfer_per_cycle :
SingleTransferPerCycleEndpoints C

-- A10. Reset-empty FIFO assumption.
A10_reset_empty_fifo :
ResetEmptyFifoAssumption C

-- A11. WFG-inert ε / Kε.
A11_Kepsilon_WFG_inert :
A11_Kepsilon_WFGInert C

-- Document-numbered A2–A4 aliases, derived from the B-rule bundle already
-- carried by A1. These are definitions/projections, not additional assumptions.
def DocKStructuralAssumptions.A2_finite_pipeline_depth_B4
(C : ComparisonInstance)
(W : WitnessBundle C)
(h : DocKStructuralAssumptions C W) :
FiniteEpsilonDepth C :=
h.A1_R_and_B_rule_conformance.2.B4_finiteEpsilonDepth

def DocKStructuralAssumptions.A3_weak_fairness_B2
(C : ComparisonInstance)
(W : WitnessBundle C)
(h : DocKStructuralAssumptions C W) :
∀ side τ,
InfiniteExecutionOfSide C side τ →
WeakFairness C side τ :=
h.A1_R_and_B_rule_conformance.2.B2_weakFairness

def DocKStructuralAssumptions.A4_channel_progress_B3
(C : ComparisonInstance)
(W : WitnessBundle C)
(h : DocKStructuralAssumptions C W) :
∀ side τ c,
InfiniteExecutionOfSide C side τ →
P_push C side τ c ∧ P_pop C side τ c :=
h.A1_R_and_B_rule_conformance.2.B3_channelProgress
```

```
-- Lean-side prerequisite for applying Appendix K to the chosen proof-internal
-- Sys_ref boundary structure. This is not a new document-numbered A-assumption.
-- It refines the document's A6 point-to-point premise for the explicit abstract
-- boundaries over which WFG, BoundaryReq, ChannelEnabled, and ChannelDisabled
-- are actually evaluated.
def KElaborationPrerequisite
(C : ComparisonInstance) : Prop :=
∀ c,
  ChannelCanContributeWFGEdge C c →
  ChosenAbstractBoundaryOfSysRef C c ∧
  PointToPointAtWFGEvaluationBoundary C c ∧
  NoHiddenSyncBoundaryGatingAtOrdinaryBufferedBoundary C c ∧
  (RequiresSyncBoundaryWithholding C c →
    C.refChoice.kind = ReferenceKind.elaborated)
```

Lean-side additions:

```
structure KStructuralAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
docNoA5 : DocKStructuralAssumptions C W

-- Imported Appendix J compatibility uses E1–E5. Keep this as a Lean-side
-- theorem prerequisite rather than relabeling it as one of Appendix K's
-- document-numbered A1–A11 assumptions.
e_rules_for_imported_J :
ERules C

-- Appendix C Issue/Commit well-formedness is needed by K.0/K.1-style
-- frontier and request-attribution lemmas, but it is not one of the
-- document-numbered Appendix K A1–A11 assumptions. In particular, the current
-- document's A6 is the point-to-point channel assumption, not Issue/Commit WF.
issue_commit_wf :
IssueCommitWF C W
witness_universe_wf :
WitnessBundleWF C W

-- The K-stack invokes Appendix J / Corollary J.1 to obtain corresponding
-- Sys_ref and RTL_B cutpoints from a post-side deadlock cutpoint. Therefore the
-- non-K-numbered prerequisites required by JTraceAssumptions and
-- JCutpointAssumptions must also be available
-- to the K theorem instance. These are Lean-side import prerequisites, not
-- additional document-numbered Appendix K assumptions.
wellformed_for_imported_J :
C.WellFormed
```

```
implementation_for_imported_J :
ImplementationAssumptions C
environment_for_imported_J :
ReactiveEnvironmentWF C.env
elaboration_projection_wf_for_imported_J :
∀ side τ, ElaborationProjectionWF C side τ

contracts : ModelingContracts C
r2c? : Option (R2C_WF C)
elaboration_prerequisite :
KElaborationPrerequisite C

-- Final Appendix K theorem bundle. This is the only K bundle that carries A5.
-- Structural lemmas, including K.2b, take KStructuralAssumptions instead.
structure KAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
structural : KStructuralAssumptions C W
A5_source_liveness :
InternalMPDeadlockFreeOnSerialPrefixesRef C

-- Derived constructor used whenever a structural K proof imports Appendix J's
-- post-to-reference or cutpoint-correspondence theorem.
def KStructuralAssumptions.toJTraceAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W) : JTraceAssumptions C W :=
{
wellformed := hK.wellformed_for_imported_J
bRules := BRules.toTraceBRules C hK.docNoA5.A1_R_and_B_rule_conformance.2
rRules := hK.docNoA5.A1_R_and_B_rule_conformance.1
eRules := hK.e_rules_for_imported_J
issueCommitWF := hK.issue_commit_wf
witnessUniverseWF := hK.witness_universe_wf
implementation := hK.implementation_for_imported_J
witnessCompatible :=
hK.docNoA5.A7_determinism_witness_selection_cutpoint_abstraction.2.1
contracts := hK.contracts
environment := hK.environment_for_imported_J
elaborationProjectionWF := hK.elaboration_projection_wf_for_imported_J
}

def KAssumptions.toJTraceAssumptions
(C : ComparisonInstance)
```

```
(W : WitnessBundle C)
(hK : KAssumptions C W) : JTraceAssumptions C W :=
KStructuralAssumptions.toJTraceAssumptions C W hK.structural

-- Cutpoint transfer imports B4/B5 and J.AbsConf in addition to the trace bundle.
def KStructuralAssumptions.toJCutpointAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W) : JCutpointAssumptions C W :=
{
toJTraceAssumptions := KStructuralAssumptions.toJTraceAssumptions C W hK
normalization :=
  BRules.toNormalizationBRules C hK.docNoA5.A1_R_and_B_rule_conformance.2
jAbsConf :=
  hK.docNoA5.A7_determinism_witness_selection_cutpoint_abstraction.2.2
}

def KAssumptions.toJCutpointAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W) : JCutpointAssumptions C W :=
KStructuralAssumptions.toJCutpointAssumptions C W hK.structural
```

This avoids fictional A12/A13 labels and also avoids silently renumbering the document's A1–A11 assumptions inside Lean. It also makes the Appendix J import path explicit: structural K proofs use hK.docNoA5 for document-numbered facts other than A5, use KStructuralAssumptions.toJTraceAssumptions C W hK for the plain J.1 trace theorem, and use KStructuralAssumptions.toJCutpointAssumptions C W hK for normalized cutpoint transfer. The final K.1 theorem uses the wrapper KAssumptions C W;
A5 is then accessed as hK.A5_source_liveness and structural lemmas receive hK.structural. The JCutpointAssumptions projection packages the B4/B5 normalization premises and the J.AbsConf discharge for the same selected cutpoint/witness triple. Since A7 is
kept aligned with the scheduling-rules document, A7 has exactly three conjuncts; J.AbsConf is projected from A7, while the WFG-boundary point-to-point fact is projected from the Lean-side KElaborationPrerequisite.

```
def KStructuralAssumptions.jAbsConf
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W) : JAbsConf C W :=
hK.docNoA5.A7_determinism_witness_selection_cutpoint_abstraction.2.2
```

```
def KAssumptions.jAbsConf
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W) : JAbsConf C W :=
KStructuralAssumptions.jAbsConf C W hK.structural

def KStructuralAssumptions.pointToPointAtWFGBoundary
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W) :
∀ c, ChannelCanContributeWFGEdge C c →
PointToPointAtWFGEvaluationBoundary C c :=
fun c hc => (hK.elaboration_prerequisite c hc).2.1

def KAssumptions.pointToPointAtWFGBoundary
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W) :
∀ c, ChannelCanContributeWFGEdge C c →
PointToPointAtWFGEvaluationBoundary C c :=
KStructuralAssumptions.pointToPointAtWFGBoundary C W hK.structural
```

Standing B1 import for Appendix K. Since Appendix K uses the same main theorem stack as Appendix J, KStructuralAssumptions C W should import the standing RTL_B B1 field from the B-rule / JTraceAssumptions layer. K.1 may still use B4/B5 only at the points where ε-normalization, fair drain-normalization, ε-quiescent endpoint reasoning, or uniqueness up to finite ε-stutter is needed, but it should not re-state B1 as a merely conditional uniqueness premise.

K lemmas:

The single canonical K.0 interface is K0_front_nextfront_classification above, together with K0_front_correspondence_from_cutpoint. No second placeholder K0_frontier_blocking_classification declaration is retained.

```
theorem K0a_rendezvous_endpoint_request_agreement
(C : ComparisonInstance)
(W : WitnessBundle C)
(hCorr : CutpointCorrespondence C W σ_ref ρ_post)
(c : Channel)
(hB0 : capacity c = 0) :
Req_prod c σ_ref = Req_prod c ρ_post ∧
Req_cons c σ_ref = Req_cons c ρ_post
```

K0a is not proved from matched observable history alone. For B(c)=0 rendezvous, ChannelEnabled depends on the raw complementary endpoint request in the same ε-quiescent cycle, and a raw request may persist as protocol state while not being the current frontier item. The proof therefore projects the required raw producer-side and consumer-side endpoint-request equality from hCorr.j_abs, using the J.AbsConf discharge for the same selected (τ_post, τ_ref, ρ_post, σ_ref, W) triple.

```
-- Definitional unfolding lemma.
-- This is useful in proofs, but it is not the substantive Appendix K.1
-- channel-case characterization.
theorem InternalMPDeadlockShapeAt_unfold_at_quiescent
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(σ : State)
(hQ : EpsQuiescent σ) :
InternalMPDeadlockShapeAt C .post σ ↔
ExistsClosedDrainClosedWFGCycle C .post σ :=
by
-- Unfold InternalMPDeadlockShapeAt and use hQ for the ε-quiescence conjunct.
sorry

-- Channel-side cause exposed by Appendix K.1.
-- These cases are derived from ChannelDisabledItemSide plus E4 at the chosen
-- abstract Sys_ref / RTL_B boundary. For buffered channels, disabled Push means
-- no space and disabled Pop means no data. For rendezvous channels, disabled
-- Push/Pop means the unique complementary endpoint request is absent in the
-- same cycle.
inductive K1BlockedChannelCause
(C : ComparisonInstance)
(side : Side)
(σ : State)
(x : FrontierItem)
(q : DynMsgInstance)
(c : Channel) : Prop
| buffered_push_full :
q.channel = c →
```

IsPushKind q.kind →
0 < capacity C c →
occAt c σ.cycle = capacity C c →
K1BlockedChannelCause C side σ x q c
| buffered_pop_empty :
q.channel = c →
IsPopKind q.kind →
0 < capacity C c →
occAt c σ.cycle = 0 →
K1BlockedChannelCause C side σ x q c
| rendezvous_push_missing_pop :
q.channel = c →
IsPushKind q.kind →
capacity C c = 0 →
¬ ComplementaryPopRequestAt C side c σ →
K1BlockedChannelCause C side σ x q c
| rendezvous_pop_missing_push :
q.channel = c →
IsPopKind q.kind →
capacity C c = 0 →
¬ ComplementaryPushRequestAt C side c σ →
K1BlockedChannelCause C side σ x q c

-- Substantive Appendix K.1 characterization.
-- This is the theorem corresponding to the main document's Lemma K.1:
-- every process in the deadlocked SCC is blocked at its current blocking
-- message frontier, and every such disabled blocking channel operation has one
-- of the E4 channel-side causes below.
theorem K1_deadlock_characterization
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(σ : State)
(hReach : Reachable C.RTL_B σ)
(hDead : InternalMPDeadlockShapeAt C .post σ) :
∃ D,
WFGTerminalSCC C .post D σ ∧
DrainClosed C .post D σ ∧

```
∀ P,
P ∈ D →
ProcessStalledAtFront C .post P σ ∧
∀ x q c,
x ∈ Front C .post P σ →
BlockingMessageFrontierItem x →
MsgOfFrontierItemSide C .post x = some q →
q.channel = c →
BoundaryReqItemSide C .post x σ →
ChannelDisabledItemSide C .post x σ →
K1BlockedChannelCause C .post σ x q c :=
by
-- Proof outline:
-- 1. Unfold InternalMPDeadlockShapeAt using
-- InternalMPDeadlockShapeAt_unfold_at_quiescent to obtain an ε-quiescent
-- closed drain-closed WFG SCC D.
-- 2. Use ProcessStalledAtFront to show each P ∈ D is stalled on its current
-- blocking message frontier, not on an internal ε-step.
-- 3. Use MsgOfFrontierItemSide to recover the owning dynamic message instance q.
-- 4. Use ChannelDisabledItemSide, hK.docNoA5.A1_R_and_B_rule_conformance, and
-- hK.e_rules_for_imported_J / E4 channel semantics at σ.cycle:
-- • disabled buffered Push_c implies occAt c σ.cycle = capacity C c;
-- • disabled buffered Pop_c implies occAt c σ.cycle = 0;
-- • disabled rendezvous Push_c implies the complementary Pop request is
-- absent in σ;
-- • disabled rendezvous Pop_c implies the complementary Push request is
-- absent in σ.
-- 5. Use hK.elaboration_prerequisite to ensure that c is the chosen explicit
-- abstract boundary of the Sys_ref / RTL_B comparison after any required
-- Sys_B^elab elaboration. Synchronization-boundary ramp-down / epoch-gating
-- is therefore represented at explicit abstract channels, not as a hidden
-- exception to E4 on an otherwise ordinary buffered channel.
sorry

theorem K2_dependency_refinement
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
```

(σ_ref ρ_post : State)
(hCorr : CutpointCorrespondence C W σ_ref ρ_post) :
WFG C .ref σ_ref = WFG C .post ρ_post

theorem K2a_drain_closure_transfer
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(D : Finset Process)
(σ_ref ρ_post : State)
(hCorr : CutpointCorrespondence C W σ_ref ρ_post)
(hDrainPost : DrainClosed C .post D ρ_post) :
DrainClosed C .ref D σ_ref

K2_dependency_refinement and K2a_drain_closure_transfer intentionally do not take separate hQref / hQpost or hWFref / hWFpost parameters. Their cutpoint quiescence facts are part of hCorr, via hCorr.ref_quiescent and hCorr.post_quiescent, and their cutpoint well-formedness facts are part of hCorr, via hCorr.ref_wellformed and hCorr.post_wellformed. Thus these theorems can invoke K0_front_correspondence_from_cutpoint without any extra side hypotheses.

K.2b serial-source dominance. The scheduling-rules v45 proof no longer tries to rebuild a fresh serial replay toward the same frontier. Lean should formalize K.2b as a well-founded reduction on the number of eager blocking issues in a finite liberal semantic reference prefix. Every theorem that creates or edits a run quantifies over Prefix C.Sys_ref. RefPrefixBuckets is used only to compute finite eager-event measures and endpoint streams from those semantic prefixes; the proof may not first construct an arbitrary ExecutionBucketed value and then assume that it lifts to a legal Sys_ref prefix.

structure DrainClosedDeadlockingRefPrefix
(C : ComparisonInstance)
(ρ_ref : Prefix C.Sys_ref)
(σ : State) where
liberal : LiberalReferencePrefix C ρ_ref
terminal : TerminalCutpoint ρ_ref σ
deadlocked : InternalMPDeadlockShapeAt C .ref σ
component :
∃ D, ClosedDrainClosedWFGComponent C .ref D σ

-- Prefix-suppression is not arbitrary deletion. It may remove or defer only a
-- suffix/order-ideal of each endpoint’s issued blocking-operation stream. This
-- prefix property is the reason occupancy domination is true; naive “fewer

```
-- Pushes and fewer Pops” is not monotone.
theorem K2b_endpoint_prefix_suppression
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(ρ_ref ρ_ref' : Prefix C.Sys_ref)
(hDef : DefersOneEagerIssueRefPrefix C ρ_ref ρ_ref') :
∀ c dir,
ChannelCanContributeWFGEdge C c →
RefEndpointIssuedBlockingStreamPrefix C ρ_ref' ρ_ref c dir :=
by
-- Project both semantic prefixes through RefPrefixBuckets, then use A6
-- point-to-point ownership, same-endpoint source order, E3 no-reverse
-- issue/commit order and hDef.bucket_deferral. The hDef.output_exec_ref field
-- is the separately constructed admissibility evidence for ρ_ref'; this theorem
-- projects the endpoint-prefix consequence and does not reconstruct Exec_ref.
sorry

theorem K2b_joint_value_control_consistency
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(ρ_ref ρ_ref' : Prefix C.Sys_ref)
(hPref : ∀ c dir,
 RefEndpointIssuedBlockingStreamPrefix C ρ_ref' ρ_ref c dir)
(hRetained : RetainedCommittedTransferPrefix
 C .ref
 (RefPrefixBuckets C ρ_ref')
 (RefPrefixBuckets C ρ_ref)) :
JointSourceReplayAgreementUpToRetainedCommits
 C W
 (RefPrefixBuckets C ρ_ref')
 (RefPrefixBuckets C ρ_ref) :=
by
-- This is a simultaneous source/dataflow theorem, not merely a per-process
-- deterministic replay lemma. Fixed stimulus, fixed NB-CF / witness-selected NB
-- outcomes, matched retained Push payloads, and matched retained Pop-return
-- values imply the same operation identities, guards, payloads, and branches up
-- to the retained commit prefix.
sorry

theorem K2b_occupancy_disabledness_domination
(C : ComparisonInstance)
(W : WitnessBundle C)
```

```
(hK : KStructuralAssumptions C W)
(ρ_ref ρ_ref' : Prefix C.Sys_ref)
(σ σ' : State)
(x : FrontierItem)
(hPrefix : ∀ c dir,
  ChannelCanContributeWFGEdge C c →
  RefEndpointIssuedBlockingStreamPrefix C ρ_ref' ρ_ref c dir)
(hFront : CorrespondingBlockedFrontierItem
  C .ref
  (RefPrefixBuckets C ρ_ref)
  (RefPrefixBuckets C ρ_ref')
  σ σ' x)
(hDisabled : ChannelDisabledItemSide C .ref x σ) :
ChannelDisabledItemSide C .ref x σ' :=
by
-- Evaluate on the explicit abstract channel where ChannelDisabled is defined.
-- For a blocked Pop, endpoint-prefix suppression preserves the relevant empty
-- condition; for a blocked Push, it preserves the relevant full condition; for
-- rendezvous, suppressing eager issue cannot create the missing unique
-- complementary endpoint request. Sys_B^elab cases lift only when the introduced
-- boundary channel is point-to-point at the WFG evaluation boundary.
sorry

-- This monotonicity fact prevents the eager-reduction proof from asserting,
-- without proof, that serialization cannot dissolve the deadlock. The two run
-- arguments are semantic Sys_ref prefixes; their bucket projections expose the
-- occupancy edit, while admissibility and fixed-stimulus facts remain attached
-- to the prefixes themselves.
theorem K2b_no_third_party_enable_under_eager_deferral
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(ρ_ref ρ_ref' : Prefix C.Sys_ref)
(σ σ' : State)
(D : Finset Process)
(hDef : DefersOneEagerIssueRefPrefix C ρ_ref ρ_ref')
(hPrefix : ∀ c dir,
  ChannelCanContributeWFGEdge C c →
  RefEndpointIssuedBlockingStreamPrefix C ρ_ref' ρ_ref c dir)
(hPTP : ∀ c,
  ChannelCanContributeWFGEdge C c →
  PointToPointAtWFGEvaluationBoundary C c)
(hComponent : ClosedDrainClosedWFGComponent C .ref D σ) :
NoThirdPartyEnablementOrOutgoingEscape
```

```
  C .ref
  (RefPrefixBuckets C ρ_ref)
  (RefPrefixBuckets C ρ_ref')
  σ σ' D :=
by
-- Split on whether the deferred eager operation is still pending at σ or
-- committed in ρ_ref. Pending eager requests are non-frontier protocol state
-- and are inert under Kε. If it committed, point-to-point plus endpoint-prefix
-- occupancy domination shows that withholding the early commit cannot enable a
-- third process or create an outgoing escape from the canonical component.
sorry

theorem fair_drain_normalize_closed_component
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(side : Side)
(ρ : Prefix (SystemOfSide C side))
(σ : State)
(D : Finset Process)
(hTerm : TerminalCutpoint ρ σ)
(hClosed : ClosedWFGComponent C side D σ)
(hBlocked : ∀ P, P ∈ D → FrontierBlocked C side P σ)
(hNoEscape : NoThirdPartyEnablementOrOutgoingEscapeAt C side ρ σ D) :
∃ ρ' σ',
ExtendsFinitePrefix C side ρ' ρ ∧
TerminalCutpoint ρ' σ' ∧
ReachableByFiniteDrainNormalization C side D σ σ' ∧
EpsQuiescent σ' ∧
DrainClosed C side D σ' ∧
∃ D', ClosedDrainClosedWFGComponent C side D' σ' :=
by
-- This is the side-parametric finite fair drain-normalization construction
-- referenced by scheduling-rules K.2b and by the "Exec_post applicability at a
-- deadlock cutpoint" part of Theorem K.1. It allows only already-issued
-- non-frontier drain commits, interleaves fair outside-D/environment progress,
-- and issues no new work across the blocked frontier.
sorry

-- Once disabledness and no-third-party-enable are known, invoke
-- fair_drain_normalize_closed_component; the edited semantic prefix then has a
-- canonical terminal closed drain-closed WFG component.
theorem K2b_terminal_scc_persists_after_eager_deferral
(C : ComparisonInstance)
```

```
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(ρ_ref ρ_ref' : Prefix C.Sys_ref)
(σ σ' : State)
(D : Finset Process)
(hDef : DefersOneEagerIssueRefPrefix C ρ_ref ρ_ref')
(hOld : ClosedDrainClosedWFGComponent C .ref D σ)
(hNoEscape : NoThirdPartyEnablementOrOutgoingEscape
  C .ref
  (RefPrefixBuckets C ρ_ref)
  (RefPrefixBuckets C ρ_ref')
  σ σ' D)
(hBlocked' : ∀ P, P ∈ D → FrontierBlocked C .ref P σ')
(hDrain' : DrainClosed C .ref D σ') :
∃ D', ClosedDrainClosedWFGComponent C .ref D' σ' :=
by
-- Follow WFG edges from blocked processes in D at σ'. Point-to-point ensures the
-- complementary owner is unique and no third-party arbitration can be newly
-- enabled by removing the eager operation. Finiteness of the process set yields
-- a nonempty terminal SCC. Drain closure rules out downstream drain escapes.
sorry

theorem K2b_construct_admissible_eager_deferral
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(ρ_ref : Prefix C.Sys_ref)
(σ : State)
(hDead : DrainClosedDeadlockingRefPrefix C ρ_ref σ)
(hEager : 0 < RefEagerIssueCount C ρ_ref) :
∃ ρ_ref',
DefersOneEagerIssueRefPrefix C ρ_ref ρ_ref' :=
by
-- This is the explicit Exec_ref-preservation obligation. Choose the eager issue
-- to defer, construct the edited finite Sys_ref prefix under the same fixed
-- stimulus/witness/capacity/boundary instance, and then construct an infinite
-- RefAdmissible extension of that edited prefix. The extension proof must
-- re-establish the applicable B2/B3 fairness and progress conditions; it is not
-- obtained merely by editing RefPrefixBuckets or by asserting that the output is
-- a semantic Prefix C.Sys_ref. The output_issue_order and output_no_reverse
-- fields establish the remaining LiberalReferencePrefix conjuncts.
sorry

theorem K2b_one_step_eager_reduction
```

```
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(ρ_ref : Prefix C.Sys_ref)
(σ : State)
(hDead : DrainClosedDeadlockingRefPrefix C ρ_ref σ)
(hEager : 0 < RefEagerIssueCount C ρ_ref) :
∃ ρ_ref' σ',
DefersOneEagerIssueRefPrefix C ρ_ref ρ_ref' ∧
DrainClosedDeadlockingRefPrefix C ρ_ref' σ' ∧
RefEagerIssueCount C ρ_ref' < RefEagerIssueCount C ρ_ref :=
by
-- First invoke K2b_construct_admissible_eager_deferral. This is where the
-- edited finite prefix and its fair/progress-satisfying RefAdmissible extension
-- are constructed. Then use the record's bucket_deferral field. Endpoint-prefix
-- suppression and occupancy-disabledness domination preserve the
-- relevant blockers; no-third-party-enable and fair_drain_normalize_closed_component
recover
-- a canonical component at an ε-quiescent drain-closed cutpoint.
sorry

theorem K2b_minimal_eager_deadlock_is_serial
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(ρ_ref : Prefix C.Sys_ref)
(σ : State)
(hDead : DrainClosedDeadlockingRefPrefix C ρ_ref σ)
(hMin : ∀ ρ_ref' σ',
  DrainClosedDeadlockingRefPrefix C ρ_ref' σ' →
  RefEagerIssueCount C ρ_ref ≤ RefEagerIssueCount C ρ_ref') :
ConservativeSerialReferencePrefix C ρ_ref :=
by
-- If RefEagerIssueCount is positive, K2b_one_step_eager_reduction constructs an
-- actual semantic prefix with a smaller count, contradicting minimality. Then
-- zero_ref_eager_iff_serial_reference_prefix and hDead.liberal yield seriality.
sorry

theorem K2b_serial_source_dominance_for_internal_deadlock
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(ρ_ref : Prefix C.Sys_ref)
(σ_ref : State)
```

```
(hLiberal : LiberalReferencePrefix C ρ_ref)
(hTerm : TerminalCutpoint ρ_ref σ_ref)
(hDead : InternalMPDeadlockShapeAt C .ref σ_ref) :
∃ ρ_ser σ_ser,
ConservativeSerialReferencePrefix C ρ_ser ∧
TerminalCutpoint ρ_ser σ_ser ∧
InternalMPDeadlockShapeAt C .ref σ_ser :=
by
-- Package the selected prefix as DrainClosedDeadlockingRefPrefix, extracting its
-- canonical component from hDead. Among all semantic finite liberal deadlocking
-- prefixes reachable under the same fixed stimulus, witness choices, capacities,
-- and explicit boundaries, choose one with minimal RefEagerIssueCount. The
-- one-step theorem produces another Prefix C.Sys_ref whenever the count is
-- positive, so the minimum is zero. The zero-count theorem yields a
-- ConservativeSerialReferencePrefix, which is returned directly to A5.
sorry
```

Implementation note. K.2b is now the single proof bridge that replaces the former scheduler-robust A5 quantification. Its public theorem and every proof-producing reduction lemma use Prefix C.Sys_ref. Bucketed execution values are derived only by RefPrefixBuckets to compute eager counts, endpoint streams, and local occupancy facts. Consequently the zero-eager reduction result already is the semantic ρ_ser returned to A5; no InfiniteExecutionBucketed substitution or hidden realizability step is allowed. Exec_ref membership is nevertheless a real proof obligation: K2b_construct_admissible_eager_deferral must construct the RefAdmissible infinite extension for every edited finite prefix, and the later K.2b lemmas consume that explicit evidence rather than assuming it automatically. Exec membership for K.1 should distinguish three cases.

First, for a globally fixed-point cutpoint, B6 idle-stutter may be used to build an infinite execution extension:

```
theorem fixedpoint_cutpoint_in_Exec_post
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(ρ_post : State)
(hReach : Reachable C.RTL_B ρ_post)
(hFP : FixedPointState C .post ρ_post)
(hIdle : C.assumptions.B.B6_idleStutterTimeDivergence) :
∃ τ_post,
TerminalCutpoint τ_post ρ_post ∧
Exec_post C τ_post
```

Second, a partial internal-MP deadlock is not justified by B6 idle-stutter alone, because processes outside D or environment endpoints may still have enabled observable actions. The v42 proof supplies the missing bridge by fair drain-normalization:

```
def DrainNormalizationStep
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(σ σ' : State) : Prop :=
∃ P q,
P ∈ D ∧
OwnsMessageEndpoint P q ∧
Blocking q ∧
NonFrontierBlockingDrainCandidate C side P σ q ∧
BoundaryReqSide C side q σ ∧
ChannelEnabledSide C side q σ ∧
CommitDrainOnly C side q σ σ' ∧
NoNewWorkCrossesDisabledInternalFrontier C side D σ σ'
```

Thus drain-normalization is intentionally stated over already-issued downstream blocking requests, not over the current BlockingMsgNextFront_P(σ) slice. At ε-quiescent well-formed cutpoints, K.0 identifies Front_P(σ) with the BoundaryReq-enabled blocking NextFront slice; drain-only progress is for already-issued downstream requests outside that current minimal frontier, while NoNewWorkCrossesDisabledInternalFrontier prevents such commits from advancing new source work past the blocked frontier.

```
def DrainNormalizedDeadlockCutpoint
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
(σ₀ σ' : State) : Prop :=
ReachableByFiniteDrainNormalization C side D σ₀ σ' ∧
EpsQuiescent σ' ∧
DrainClosed C side D σ' ∧
∀ P,
P ∈ D → ProcessStalledAtFront C side P σ'
```

```
theorem deadlock_cutpoint_in_Exec_post_via_fair_drain_normalization
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KStructuralAssumptions C W)
(ρ_post : State)
(hReach : Reachable C.RTL_B ρ_post)
(hDead : InternalMPDeadlockShapeAt C .post ρ_post) :
∃ τ_post ρ_post' D,
TerminalCutpoint τ_post ρ_post' ∧
Exec_post C τ_post ∧
DrainNormalizedDeadlockCutpoint C .post D ρ_post ρ_post' ∧
InternalMPDeadlockShapeAt C .post ρ_post' :=
by
-- Use K1_deadlock_characterization to obtain the closed SCC D. B3 cannot force
-- progress of D-owned internal frontier blockers without contradicting
-- disabled-frontier, drain-closure, or WFG closure; rendezvous blockers are
-- handled by absence of the same-cycle complementary endpoint request.
-- Environment-facing frontier items are not WFG edges and are handled as
-- ordinary fair continuation, not as WFG-closure contradictions. For
-- already-issued D-owned non-frontier requests, drain-closure only says they are
-- not ChannelEnabled at the current cutpoint; if outside-D or environment
-- progress later makes one continuously serviceable, allow it to commit as
-- drain-only progress. Automatic flush, finite explicit capacities, finite
-- ε-depth, and no new work across the disabled internal frontier imply only
-- finitely many such D-owned drain commits before a new ε-quiescent
-- drain-closed cutpoint is reached, unless a frontier item commits and the
-- proof uses the corresponding reachable cutpoint according to the v45 case
-- split. Interleave this finite drain-normalization with fair outside-D and
-- environment continuation to obtain Exec_post.
sorry
```

Third, the main K theorem can be exposed in both execution-scoped and reachable forms. The execution-scoped post-side form is convenient internally; the reachable form is what Appendix K states after applying the fair drain-normalization bridge. The source-side liveness premise used to contradict a reference deadlock is now serial, not liberal/witness-relative.

```
def InternalMPDeadlockFreeOnExecPost
```

```
(C : ComparisonInstance) : Prop :=
∀ τ_post ρ_post,
Exec_post C τ_post →
TerminalCutpoint τ_post ρ_post →
¬ InternalMPDeadlockShapeAt C .post ρ_post
```

```
theorem K1_liveness_preservation_on_exec
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W) :
InternalMPDeadlockFreeOnExecPost C
```

```
def InternalMPDeadlockFreeReachablePost
(C : ComparisonInstance) : Prop :=
∀ ρ_post,
¬ ReachableInternalMPDeadlockAt C .post ρ_post
```

Main K theorem:

```
theorem K1_liveness_preservation
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W) :
InternalMPDeadlockFreeReachablePost C
```

Proof dependency note. Prove the reachable post-side conclusion by contradiction. Assume ReachableInternalMPDeadlockAt C .post ρ_post and apply deadlock_cutpoint_in_Exec_post_via_fair_drain_normalization to obtain one specific Exec_post prefix τ_post ending at the ε-quiescent drain-normalized cutpoint ρ_post'. Import Appendix J with hJCut := KAssumptions.toJCutpointAssumptions C W hK and invoke J1_selected_matched_cutpoint on that exact τ_post. The B4/B5 normalization premises and the selected J.AbsConf evidence are packaged in hJCut rather than supplied to the plain trace theorem. Destruct the resulting package M : SelectedMatchedCutpoint C W and retain the equality M.τ_post = τ_post.

All subsequent facts are projections from M: M.selected, M.ref_exec, M.compatible, M.normalized, M.post_terminal, M.ref_terminal, M.ref_liberal, and M.correspondence. Consequently J.AbsConf, the selected witness, and the cutpoint transfer cannot silently refer to a different post prefix, reference prefix, or normalized pair. Use

K2_dependency_refinement, K2a_drain_closure_transfer, and K0a_rendezvous_endpoint_request_agreement with M.correspondence to transfer the closed drain-closed internal WFG deadlock from M.ρ_post to M.σ_ref.

Apply K2b_serial_source_dominance_for_internal_deadlock to M.τ_ref, M.σ_ref, M.ref_liberal, M.ref_terminal, and the transferred deadlock. It returns ρ_ser and σ_ser satisfying ConservativeSerialReferencePrefix C ρ_ser, TerminalCutpoint ρ_ser σ_ser, and InternalMPDeadlockShapeAt C .ref σ_ser. These three facts contradict hK.A5_source_liveness directly. The proof never converts the selected finite J.1 prefix into an InfiniteExecutionBucketed argument. No additional fair-extension assumption is introduced at the K.1 contradiction step: K.2b returns a ConservativeSerialReferencePrefix whose LiberalReferencePrefix component already contains Exec_ref, and K2b_construct_admissible_eager_deferral is the explicit theorem that constructs and proves the required RefAdmissible extension after each eager deferral.

This is intentionally one-way. Do not state:
InternalMPDeadlockFreeOnSerialPrefixesRef C ↔ InternalMPDeadlockFreeReachablePost C

A5 separation and K.1 deduplication. A5 is the finite-prefix predicate InternalMPDeadlockFreeOnSerialPrefixesRef C, matching the scheduling-rules formulation over reachable cutpoints of admissible conservative-serial source prefixes. It is deliberately not part of KStructuralAssumptions, because K.2b and the structural K.0/K.2 lemmas are used to reach the serial-source deadlocking prefix before A5 is invoked. The final KAssumptions wrapper adds exactly one A5 field, hK.A5_source_liveness. Consequently K1_liveness_preservation and K1_liveness_preservation_on_exec no longer take a second explicit source-liveness premise; they consume hK.A5_source_liveness at the final contradiction step. InternalMPDeadlockFreeOnSerialExecRef remains only a definitional convenience alias for the finite-prefix A5 predicate; it introduces no second execution domain. This removes duplicate assumptions and prevents circular dependence of K.2b on A5. It does not erase the admissibility obligation: because ConservativeSerialReferencePrefix contains LiberalReferencePrefix and therefore Exec_ref, the fair/progress-satisfying extension of each edited prefix must be constructed explicitly inside K2b_construct_admissible_eager_deferral before the serial prefix can be returned to A5.

19. Appendix R facade

Appendix R should be formalized last as aliases/wrappers over earlier theorems.

Use Lean names that avoid the doc's R.2 collision:

```
theorem R_lemma1_front_nextfront_classification := ...
theorem R_theorem1_prefix_matching := ...
theorem R_corollary1_cutpoint_correspondence := ...
theorem R_corollary1a_rendezvous_endpoint_request_agreement := ...
theorem R_lemma2_dependency_refinement := K2_dependency_refinement
theorem R_theorem2_closed_cycle_and_one_way_drain_transfer := ...
```

Appendix R uses per-type numbering. The Lean identifiers include the item type and therefore remain unambiguous: the closed-cycle/drain-transfer result is the scheduling document's Theorem R.2 and is named R_theorem2_closed_cycle_and_one_way_drain_transfer. Do not renumber it to R.3 merely to avoid cross-type reuse of the numeral R.2.

Do not call cutpoint correspondence R1_cutpoint_correspondence; in the doc structure, it corresponds to a corollary, not Theorem R.1.

Appendix R cutpoint correspondence should expose the v42 naming: the post-side terminal cutpoint is ρ_post, the relation σ_ref ≡ ρ_post is the compact notation for the J.Abs/J.AbsConf source-slice correspondence ρ_post ⇓_{Abs,J,w} σ_ref, and it is not equality of arbitrary RTL microstate.

20. What remains as hypotheses

These remain theorem parameters or design/tool obligations:
1. R-rules R1, R2, R2C, R3, R4, R5.
2. E-rules E1-E5, with the v42 RequiredJOrder restriction made explicit: E2 is alignment only, distinct-interface E3 source order is no-reverse legality rather than a positive strict edge, atomic multi-endpoint operations are single observable units rather than internal order edges, and E5 contributes strict RequiredJOrder only on its suff_S side. The pref_S side of E5 remains a non-strict issue/commit legality and sync-boundary well-formedness obligation: q ∈ pref_S(s) gives issue(q) ≤ clk(s), and for committed blocking q the commit cycle is ≤ clk(s), with equality allowed. The sanity package RequiredJOrderWF is not a new external assumption; it should be proved inside Lean as RequiredJOrderWF_from_concrete_generators once the concrete E1-E5,

IssueCommitWF, ObservableUnitWF, R2C unit, and elaboration/projection definitions are instantiated.

3. B-rule semantic obligations are theorem-scoped rather than imported through one undifferentiated J bundle. TraceBRules carries B1, B2, B3, and B6 for the plain Appendix J trace theorem and its admissible execution construction. NormalizationBRules carries B4 and B5 and is imported only by JCutpointAssumptions and by Appendix K, where ε-normalization, ε-quiescent endpoint reasoning, or finite-stutter closure is actually used. The full BRules bundle remains the document-numbered Appendix K A1 component. B1 is the standing RTL_B deterministic-Σ-trace assumption PostDeterministicSigmaTrace C for the fixed comparison instance; it is not a source-side determinism premise. B2/B3 are encoded temporally, and B5 is limited to genuine quiescence closure under the no-external-change qualification, including bounded_system_quiescence. B2 must encode the split in which ChannelEnabled means eligibility, selection is the fairness event, and the selected enabled action commits in the selected cycle. B3 remains the channel-progress obligation that can discharge persistent full/empty/rendezvous-blocked premises before a pending transfer becomes ChannelEnabled. The max and least-upper-bound facts for SystemQuiescenceBound C are Lean lemmas proved from Finset.sup, not hypotheses. The Appendix K A2, A3, and A4 names are projections from the B4, B2, and B3 fields of A1's full BRules component. Appendix K A5 remains the separate finite-prefix serial-source liveness premise InternalMPDeadlockFreeOnSerialPrefixesRef C; the witness-relative reference deadlock is connected to it only by K2b_serial_source_dominance_for_internal_deadlock.

4. Well-formedness of the selected comparison instance C.WellFormed.

5. ImplementationAssumptions C, including E4 channel-realization conformance and the mechanically carried per-channel field ∀ c, BackAnnotatedExplicitAbstractChannel C c → BFaithful C c, plus the other named implementation obligations such as I.A0.

6. Appendix C Issue/Commit well-formedness and request-attribution discipline IssueCommitWF C W.

7. Reactive-environment well-formedness ReactiveEnvironmentWF C.env.

8. Witness compatibility WC C W for the full proof-internal C.Sigma theorem, obtained from SideConditionDischargeRecord.wc through WCDischarge.proof. Lemma L.2 snooping, Corollary L.3 global NB-CF plus its identity-witness and local-protocol premises, or a complete mixed per-site partition may provide the proof-producing provenance, but none replaces the actual WC premise.

8a. For the Appendix L Note 1 Σ_DUT-projected safety theorem only, the weaker SigmaDUTSafetyWitnessObligation C W may replace WC. This obligation is discharged choice-by-choice: each Σ_DUT-relevant ε-modeled local timing choice is either covered by ordinary snooping/witness machinery or belongs to a certified SigmaDUTSafetyInertInterface. This does not discharge full WC and does not discharge Appendix K WFG-inertness / Kε obligations.

8b. J.AbsConf C W for the selected RTL_B implementation, abstract boundary, witness, and normalized cutpoints. The theorem-scoped audit makes this a RequiredDischarge for j1Cutpoint and k1Liveness scopes and permits inapplicability only for a j1Trace-only scope. This is a tool/RTL-flow compliance obligation alongside E4 channel-realization conformance and B-faithfulness; it is not proved by deterministic replay alone.

8c. B-faithfulness of each back-annotated capacity B(c), as a named tool/RTL-flow compliance hypothesis BFaithful C c per explicit abstract channel: (i) boundary completeness — every Σ-visible transfer of c in RTL_B crosses the agreed abstract boundary as a committed Push_c or Pop_c, with no hidden side entrance or exit; (ii) exact boundary behavior — the concrete boundary permits exactly the Push/Pop actions permitted by E4, with occupancy bounded by B(c), no hidden storage or credit, and no hidden grant, arbitration, or backpressure condition unless that condition is represented by explicit Sys_B^elab boundary structure; (iii) ε-opacity of internal movement — internal movement of data within the realized channel between commit points is ε and contributes no Σ-labels and no WFG causes; and (iv) an evidence obligation — each B(c) is supported by a trusted back-annotation algorithm, a per-channel certificate, a structural RTL check, an interface monitor, or a formal flow, recorded in the theorem-scoped SideConditionDischargeRecord C W scope of Section 1.1 and carried by ImplementationAssumptions.bFaithful into I/J/K theorem instances. Like 8b, this is an obligation on the tool/RTL flow and the chosen comparison instance; it is not derived by E4, Proposition J.0, or Theorem J.1, all of which assume it.
9. Direct-input contracts.
10. Array access mapping rules.
11. R2C fixed-point well-formedness R2C_WF C when combinational-process signal IO is in scope, including selected Obs(C_comb) slices, stutter-canonical unit emission, component-local occurrence-ordinal matching, and either a single anchoring domain or a confluence proof for multi-input R2C components; cycle-locked R2C components must be declared explicitly.
12. Automatic-flush / drain-only blocked-frontier discipline for pipelined or overlapped implementations: while a minimal blocking frontier remains fully disabled, no new later source work may issue past that frontier; already asserted blocking requests are not withdrawn before commit; and any observable work by the blocked process during the interval is attributable only to already-issued downstream drainable work.
13. Elaboration package E, when Sys_ref = Sys_B^elab(E).
14. K elaboration prerequisite KElaborationPrerequisite C: every WFG-relevant channel is represented at the chosen abstract Sys_ref / RTL_B comparison boundary; hidden sync-boundary gating is not allowed at ordinary buffered boundaries; and sync-boundary withholding that affects WFG reasoning is represented through explicit Sys_B^elab channels.
15. Appendix K structural assumptions not already covered above: point-to-point channels, point-to-point WFG evaluation boundaries for any introduced Sys_B^elab channels, SyncChannel/barrier well-formedness, single-transfer-per-cycle endpoint constraints, reset-empty FIFO/accounted initial occupancy, and WFG-inert ε / Kε.
16. Source-level internal message-passing deadlock freedom for K.1 under A5's conservative serial-blocking source semantics: InternalMPDeadlockFreeOnSerialPrefixesRef C.
This premise quantifies over finite admissible serial prefixes that are also ordinary liberal/R4/E3/E5/no-reverse reference prefixes; it is not a scheduler-robust

liveness premise over all source scheduling choices and it is not derived from
local R-rules or from RequiredJOrderWF.
K2b_serial_source_dominance_for_internal_deadlock
is the bridge from the witness-relative liberal reference deadlock to this serial
A5 premise.
17. Fair drain-normalization for K.1 Exec_post applicability: a reachable partial internal-MP deadlock cutpoint is not extended by pure idle-stutter unless it is a global fixed point; the proof must construct a fair continuation that allows finite D-owned non-frontier drain-only commits when B2/B3 later makes them continuously serviceable, while preserving the disabled internal frontier used for the WFG contradiction.
18. The side-condition discharge record of Section 1.1. This is audit bookkeeping, not an additional semantic assumption. Its WC entry proves the actual WC C W premise and records whether that proof came from NB-CF, Appendix L snooping, direct witness construction, or complete mixed per-site coverage; NB-CF and snooping are provenance routes rather than separate audited theorem propositions. The record separately covers Sys_B^elab(E), per-channel B-faithfulness, J.AbsConf, and Kε according to theorem scope. Each constructor carries evidence of the declared method; inapplicability requires a NotRequired proof, and required per-channel B-faithfulness has no inapplicable constructor. Every theorem instance claimed against the document should be accompanied by a fully populated SideConditionDischargeRecord.

21. What should be proved inside Lean

Lean should prove:

• bucketed execution infrastructure;

• WCDischarge.proof and the theorem-scope discharge facts showing that J.AbsConf cannot be marked inapplicable for j1Cutpoint/k1Liveness and Kε cannot be marked inapplicable for k1Liveness;

• finite ObservableUnitsOf extraction for prefixes, together with ObservableUnitWF grouping/non-splitting facts for rendezvous, SyncChannel/ac_sync, stutter-canonical R2C fixed-point signal-observation units, and any other explicitly atomic same-cycle units;

• temporal lemmas for AlwaysFrom / EventuallyFrom;

• BRules.toTraceBRules and BRules.toNormalizationBRules, together with theorem-scope lemmas showing that J1_post_to_ref consumes no B4/B5 premise while J1_cutpoint_correspondence and J1_selected_matched_cutpoint consume JCutpointAssumptions;

• the elementary Finset.sup facts for SystemQuiescenceBound C: every MainB4Bound C P for P ∈ ProcessUniverse C is bounded by SystemQuiescenceBound C, and SystemQuiescenceBound C is the least such

finite upper bound. These are proved from the definition of SystemQuiescenceBound C, not assumed as part of B5.
• channel-capacity lemmas for rendezvous and buffered cases;
• the BFaithful behavioral-refinement structure, including capacity_upper_bound and boundary_behavior_exact, its per-channel evidence projections, and the ImplementationAssumptions.bFaithful projection used by I/J/K theorem instances;
• ChannelEnabled-as-eligibility lemmas, FairActionSelected/FairActionCommitsAt lemmas, and E4 safety lemmas showing that committed transfers are legal at the chosen abstract boundary;
• issue existence/uniqueness and derived IssueTime;
• post_deterministic_process_projection, deriving each process-owned/process-attributed post-side Σ-trace projection from system-level B1 without adding isolated per-process determinism;
• the A2_finite_pipeline_depth_B4, A3_weak_fairness_B2, and A4_channel_progress_B3 projection definitions from DocKStructuralAssumptions.A1_R_and_B_rule_conformance, with no duplicate assumption fields;
• reset-epoch-indexed START_P / FINISH_P occurrence uniqueness, anchoring, and clock lookup, plus finish_process_idle_padding_until_next_reset and the current-epoch terminated-peer corollaries;
• RequiredJOrderWF_from_concrete_generators, proving that the concrete v42 RequiredJOrderGenerator satisfies the E2-alignment-only, distinct-interface no-positive-edge, atomicity-as-grouping, witness-control-not-order, R2C-unit-as-atomic-observation, and elaboration-not-separate-order restrictions;
• RequiredJOrder / ≺_J acyclicity over observable units, separately from the informational CausalDependency graph, proved by the v42 per-execution commit-cycle measure clk(γ) over the fixed legal prefix;
• B3_ordinary_boundary_from_B2_E4, discharging the B3 P_push / P_pop obligation from B2 plus E4 at ordinary single explicit abstract boundaries (Section 6), so that for OrdinaryE4Boundary instances B3 is a theorem rather than an assumption;
• ProcessStalledAtFront as a literal alias of FrontierBlocked; ClosedDrainClosedWFGComponent as the single conjunction used by ExistsClosedDrainClosedWFGCycle, InternalMPDeadlockShapeAt, ProcessParticipatesInInternalMPDeadlockShapeAt, terminated_peer_waiting_not_in_deadlock_component, and terminated_peer_waiting_not_internal_MP_deadlock;
• K.2b serial-source dominance lemmas over semantic Prefix C.Sys_ref objects: RefPrefixBuckets as a derived projection, RefEagerIssueCount, zero_ref_eager_iff_serial_reference_prefix, source-order endpoint-prefix suppression, the

DefersOneEagerIssueRefPrefix record with explicit output_exec_ref evidence, K2b_construct_admissible_eager_deferral constructing a RefAdmissible fair/progress-satisfying infinite extension of each edited finite prefix, joint value/control consistency over retained committed transfers, occupancy/disabledness domination at point-to-point WFG evaluation boundaries, no-third-party-enable monotonicity, canonical-component persistence after eager deferral, one-step semantic-prefix reduction, minimal-eager deadlock implies serial, and K2b_serial_source_dominance_for_internal_deadlock;
• direct-input late-sampling preservation from primitive environment contracts;
• array sync-boundary commit-order preservation;
• R2C fixed-point observation lemmas from R2C_WF, including selected observable slices, stutter-canonical unit generation, component-local occurrence-ordinal matching, cycle-locked components, and multi-input anchoring/confluence obligations;
• automatic-flush frontier lemmas;
• Lemma L.2: snooping establishes WC, including the latency-sensitive local-protocol handling premise for cadence-sensitive retry/timeout/poll-arbiter sites;
• Corollary L.3: NB-CF identity witness, with the global/per-boundary contract obligations needed by the updated WC definition;
• Appendix L Note 1 projected-safety branch. These prove only the Σ_DUT-visible safety fragment and do not produce WC or Appendix K deadlock/liveness obligations. The global SigmaDUTSafetyWitnessObligation C W remains an explicit projected-safety assumption; certified inert-interface choices are discharged locally by LNote1_sigmaDUT_safety_inert_interface, while non-inert choices require ordinary snooping/witness coverage:
o SigmaDUTSafetyInertInterface,
o SigmaDUTSafetyWitnessObligation,
o LNote1_sigmaDUT_safety_inert_interface,
o CompatiblePDUT,
o CompatibleSysDUT,
o J0_pairwise_composition_DUT, and
o J1_post_to_ref_sigmaDUT_safety.
• IObs_congruence_one_immediate_observable_unit over concrete ObsSigma and TraceBRules, using singleton units for ordinary events and indivisible ObservableUnitWF-grouped rendezvous, synchronization, and R2C units;
• IObs_congruence_one_observable_unit as the explicitly normalized variant using NormalizationBRules;
• IDR_immediate_post_unit_deterministic_replay for the canonical immediate post-unit states used by plain J.1, and IDR_deterministic_replay for ε-quiescent cutpoints;
• Appendix I finite and countably-infinite local witness construction, including

I5_finite_local_projection_from_admissible_global_continuation and I5_finite_local_projection_from_global_fixedpoint, with B6 used only in the latter global-fixed-point branch;

- I.3 using ChannelEnabled → eventual selection → commit rather than treating enablement as immediate commit;
- Proposition J.0;
- RTLTimeNextObservableUnit and RTLTimeIdealChain, including the single-unit fixed RTL-time enumeration of the selected RTL_B witness prefix and the proof that every prefix idealAt n is down-closed under RequiredJOrder;
- J1_fixed_RTL_time_selected_witness_construction, producing the selected single-unit RTL-time ideal chain and the per-step replay-realizability facts;
- Theorem J.1 post-to-reference;
- restricted reverse theorem for RTL-consistent source prefixes;
- Corollary J.1 cutpoint correspondence over J.Abs/J.AbsConf, using ρ_post ⇓_{Abs,J,w} σ_ref rather than an implicit RTL_B-to-source projection;
- J1_selected_ref_prefix_is_liberal with both τ_post and τ_ref arguments, plus SelectedMatchedCutpoint and J1_selected_matched_cutpoint so K.1 carries one exact selected tuple through J.AbsConf and K.2b;
- K.0/K.0a/K.1/K.2/K.2a, using the single canonical K0_front_nextfront_classification interface, with K.0 proving full NextFront minimality and K.0a using raw rendezvous endpoint-request equality from J.Abs/J.AbsConf;
- fixedpoint_cutpoint_in_Exec_post as the B6 idle-stutter special case for globally fixed-point post-side cutpoints;
- fair_drain_normalize_closed_component as the side-parametric finite fair drain-normalization lemma shared by K.2b and Theorem K.1;
- deadlock_cutpoint_in_Exec_post_via_fair_drain_normalization for the v42/v45 K.1 reachable-deadlock proof;
- the separated InternalMPDeadlockShapeAt and ReachableInternalMPDeadlockAt predicates;
- LiberalReferencePrefix, ConservativeSerialReferencePrefix, finite-prefix A5 / InternalMPDeadlockFreeOnSerialPrefixesRef, the convenience alias InternalMPDeadlockFreeOnSerialExecRef, and InternalMPDeadlockFreeOnExecPost;
- Theorem K.1 one-way liveness preservation, exposed both as an execution-scoped theorem and as the reachable post-side theorem obtained using fair drain-normalization, with Appendix K A5 stated over finite conservative serial-blocking source prefixes and the witness-relative reference deadlock bridged directly to such a prefix by K2b_serial_source_dominance_for_internal_deadlock;
- Appendix R aliases.

Optional non-core formalization. The Lean development may additionally prove CausalDependencyGenerator coverage, showing that the informational relation mirrors the scheduling-rules causal-dependency definition: message information flow, signal information flow, synchronization information flow, intra-process causal trace/source order, declared memory-serialization/shared-storage dependence, witness-control dependence, and elaboration information flow. This optional coverage theorem is not a premise of Theorem J.1, Corollary J.1, Appendix K, or Appendix R. Those theorem instances depend on RequiredJOrder / ≺_J, ObservableUnitWF, E1–E5, witness/cutpoint correspondence, and the relevant progress/liveness assumptions, not on acyclicity or completeness of the broader CausalDependency graph.

22. Development milestones

Milestone A — Finite sequential, blocking-only, non-elaborated core

Scope:
Sys_ref = Sys_B;
sequential processes only;
blocking Push/Pop only;
no NB calls;
no snooping;
no external arrays;
no direct-input late sampling;
no combinational R2C;
finite bucketed prefixes and finite bucketed execution fragments;
R1–R5 and E1–E5 infrastructure for the sequential/blocking subset;
E4 channel-capacity legality for rendezvous and buffered channels;
Issue/Commit existence, uniqueness, and request-attribution discipline;
conditional finite extension lemmas stated with explicit enabledness or progress witnesses, not with B2/B3 fairness/progress conclusions;
finite-prefix observable-unit extraction and blocking-only pairwise composition.

Targets:
E4_channel_capacity_lemmas_blocking
IssueCommitWF_blocking
I3_conditional_finite_extension_blocking_given_enabledness
J0_pairwise_composition_blocking

Milestone A deliberately does not target Theorem J.1, Corollary J.1, or

Theorem K.1. Those results are execution-level / temporal results: they quantify over Exec_post / Exec_ref, rely on FairProgressSatisfying infinite executions, and use B6 idle-stutter/time-divergence only for globally fixed-point cutpoints. General partial internal-MP deadlock reasoning is execution-scoped: the relevant deadlock cutpoint is already on an Exec_post / Exec_ref execution. These results therefore belong after the temporal B2/B3/B6 machinery is available.

Milestone B — Blocking temporal, cutpoint, and liveness stack

Milestone B1 — Temporal trace construction without B4/B5

Add: infinite bucketed executions; AlwaysFrom and EventuallyFrom; concrete FairAction; P_push and P_pop; WeakFairness and side-parametric B2; side-parametric B3 channel progress; B6 idle-stutter/time-divergence restricted to global fixed points; FairProgressSatisfying; finite-prefix Exec_post and Exec_ref; I5_countably_infinite_local_limit_construction; I5_finite_local_projection_from_admissible_global_continuation; I5_finite_local_projection_from_global_fixedpoint; TraceBRules and JTraceAssumptions; LiberalReferencePrefix; IObs_congruence_one_immediate_observable_unit; IDR_immediate_post_unit_deterministic_replay; and the fixed RTL-time selected-witness construction with ImmediateReplayStateForIdeal. The plain J.1 theorem does not import NormalizationBRules.

Targets:
RequiredJOrderWF_from_concrete_generators_blocking
IObs_congruence_one_immediate_observable_unit_blocking
IDR_immediate_post_unit_deterministic_replay_blocking
I5_countably_infinite_local_limit_construction_blocking
I5_finite_local_projection_from_admissible_global_continuation_blocking
I5_finite_local_projection_from_global_fixedpoint_blocking
RTLTimeIdealChain_single_unit_blocking
J1_fixed_RTL_time_selected_witness_construction_blocking
J1_post_to_ref_blocking
J1_selected_ref_prefix_is_liberal_blocking
B3_from_B2_E4_ordinary_boundary_blocking

Milestone B2 — ε-normalization and selected cutpoint correspondence

Add: constructive B4/B5 normalization; NormalizationBRules;

IObs_congruence_one_observable_unit as the normalized replay step; IDR_deterministic_replay at ε-quiescent cutpoints; JCutpointAssumptions; ε-quiescent terminal-cutpoint construction; the J.AbsConf implementation interface and theorem-scoped discharge; SelectedMatchedCutpoint; and fixed-point-only Exec_post extension. J.AbsConf is an implementation obligation consumed by the cutpoint theorem, not a theorem to be proved from J.1 replay.

Targets:
I1_bounded_epsilon_closure_blocking
IObs_congruence_one_observable_unit_blocking
IDR_deterministic_replay_blocking
J1_cutpoint_correspondence_blocking
J1_selected_matched_cutpoint_blocking
fixedpoint_cutpoint_in_Exec_post_blocking

Milestone B3 — WFG, automatic flush, K.2b, and K.1

Add: automatic-flush/drain-only semantics; Kε; canonical closed drain-closed WFG components; fair_drain_normalize_closed_component; ConservativeSerialReferencePrefix; finite-prefix InternalMPDeadlockFreeOnSerialPrefixesRef and its convenience alias; the DefersOneEagerIssueRefPrefix record; K2b_construct_admissible_eager_deferral with explicit Exec_ref preservation; eager-count reduction; and InternalMPDeadlockFreeOnExecPost.

Targets:
InternalMPDeadlockFreeOnSerialPrefixesRef_blocking
fair_drain_normalize_closed_component_blocking
K2b_construct_admissible_eager_deferral_blocking
K2b_endpoint_prefix_suppression_blocking
K2b_no_third_party_enable_blocking
K2b_terminal_scc_persists_blocking
K2b_serial_source_dominance_blocking
K1_liveness_preservation_blocking
terminated_peer_not_internal_MP_deadlock_blocking

Milestone C — Non-blocking under NB-CF

Add:

Qexec;

failed NB polls as ε;

successful NB transfers as Σ;

Corollary L.3.

Target:

J1_post_to_ref_nb_cf

Milestone D — Witness selector, snooping, and Appendix L Note 1 projected safety

Add:

partial-prefix WitnessSelector;

Lemma L.2;

SigmaDUTSafetyInertInterface;

SigmaDUTSafetyWitnessObligation;

CompatiblePDUT and CompatibleSysDUT;

projected pairwise composition J0_pairwise_composition_DUT;

Appendix L Note 1 projected-safety branch / J1_post_to_ref_sigmaDUT_safety for interfaces whose ε-modeled local timing choices cannot add/remove/relabel Σ_DUT-visible units and can onlycommute DUT-causally-independent projected units.

Targets:

J1_post_to_ref_with_snooping

J1_post_to_ref_sigmaDUT_safety

Milestone E — Direct inputs

Add:

- DirectInputContract;
- derived lateSamplingPreservesLogicalAnchor;
- bucket placement for logical anchor vs physical late sample.

Target:

J1_post_to_ref_direct_inputs

Milestone F — External arrays

Add:

- MemoryEvent;
- ArrayAccessMapping;
- sync-relative array commit rules.

Target:

J1_post_to_ref_external_arrays

Milestone G — R2C

Add:
· combModel?;
· LocalCombNetwork;
· CombNetwork;
· CombComposition;
· R2C_WF;
· selected observable slices Obs(C_comb);
· R2C fixed-point signal-observation units;
· stutter-canonical unit emission and explicit cycle-locked component mode;
· component-local occurrence-ordinal matching;
· multi-input anchoring-domain / interleaving-confluence obligations;
· per-process combinational component obligations;
· composed fixed-point observation semantics.
Target:
J1_post_to_ref_with_R2C_v42_units

Milestone H — Elaboration

Add:

- ElaborationPackage Sys_B Chan_outer Σ_DUT;
- Π_elab;
- π_Σ,E;
- A_E;

- explicit-channel interpretation of B, occ, BoundaryReq, ChannelEnabled, Front, and WFG.

Targets:

J1_post_to_ref_elab
K1_liveness_preservation_elab

Milestone I — Appendix R facade

Add compact R theorem wrappers after the full proof stack is available.

23. Expected difficulty

The hardest pieces remain:

1. Bucketed execution and ideal induction.
2. Temporal B2/B3 over infinite executions.
3. Concrete ObsSigma and I.DR.
4. Appendix J finite-ideal construction.
5. Direct-input late-sampling contracts.
6. External array access mapping.
7. R2C fixed-point semantics.
8. Appendix Q elaboration obligations.
9. Appendix K serial-A5 liveness/deadlock preservation, including K.2b eager-count reduction / serial-source dominance, explicit construction of RefAdmissible extensions after eager deferral, and fixed-point-only Exec_post extension.

A polished full formalization is plausibly in the 18k–30k Lean line range if direct inputs, arrays, R2C, temporal fairness/progress, non-blocking witness selection, snooping, automatic flush, Issue/Commit attribution, A11/Kε, and elaboration are all included. A core sequential/blocking/non-elaborated proof would be substantially smaller, plausibly 6k–10k lines; an intermediate blocking-plus-temporal-WFG proof would likely fall around 10k–16k lines.